

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

Introduction to Kubernetes & Container Orchestration

Chapter 01 — from absolute beginner to
interview-ready foundations.

👑 Chapter 01

Chapter Purpose

Docker helps us package and run applications as containers.

But in production, running one or two containers manually is not enough.

Real production systems need to run hundreds or thousands of containers across many servers. They need automatic scheduling, scaling, healing, rolling updates, service discovery, networking, storage, security, monitoring, and disaster recovery.

Kubernetes was created to solve this problem.

This chapter introduces Kubernetes from absolute beginner level.

Learning Objectives

By the end of this chapter, you should be able to explain:

- What Kubernetes is.
- Why Kubernetes was created.
- What container orchestration means.
- Why Docker alone is not enough for production at scale.
- What problems Kubernetes solves.
- What a Kubernetes cluster is.
- What control plane and worker nodes are.
- What pods are at a high level.
- What desired state means.
- Why Kubernetes is important for DevOps, SRE, and platform engineering interviews.

1. What Is Kubernetes?

Kubernetes is an open-source container orchestration platform.

It is used to deploy, run, manage, scale, and heal containerized applications.

SIMPLE DEFINITION

Kubernetes manages containers across multiple machines.

BETTER INTERVIEW DEFINITION

Kubernetes is a container orchestration platform that maintains the desired state of applications by scheduling containers onto worker nodes, monitoring their health, restarting failed workloads, scaling replicas, managing networking, storage, configuration, and deployments.

Kubernetes is often abbreviated as:

K8S

The name K8s means: K + 8 letters + s. Kubernetes has 8 letters between K and s.

2. Why Was Kubernetes Created?

Before Kubernetes, teams could run containers using Docker.

Example:

```
docker run -d -p 8080:80 nginx
```

SHELL

This works for one container.

But production systems need much more.

Imagine you have:

- 50 frontend containers
- 100 backend containers
- 20 worker containers
- 10 database-related services
- 5 cache services
- Multiple environments
- Multiple servers
- Multiple teams

Managing these manually becomes impossible.

3. Problems Before Kubernetes

Without Kubernetes, teams had to solve many problems manually.

Problem	Manual Challenge
Scheduling	Which server should run each container?
Scaling	How to increase or decrease replicas?
Self-healing	What if a container crashes?
Service discovery	How do containers find each other?
Load balancing	How is traffic distributed?
Rolling updates	How to deploy without downtime?
Rollbacks	How to return to previous version?
Storage	How does data persist?
Configuration	How are configs injected?
Secrets	How are passwords managed?
Security	Who can do what?
Monitoring	How do we observe health?

Kubernetes automates many of these responsibilities.

4. Why Docker Alone Is Not Enough

Docker is excellent for:

- Building images
- Running containers
- Local development
- Simple single-host setups
- CI/CD image packaging

But Docker alone does not fully solve:

- Multi-node scheduling
- Automatic failover
- Cluster-level scaling
- Service discovery across many nodes
- Declarative desired state management
- Rolling updates across replicas
- Production-grade orchestration

- Cluster-wide networking
- Advanced workload management

DOCKER ANSWERS

How do I package and run a container?

KUBERNETES ANSWERS

How do I run many containers reliably across many machines in production?

5. Real-World Analogy

Think of a large hospital.

Docker is like a medical equipment box. Each box contains everything needed for one task.

Kubernetes is like the hospital operations manager. It decides:

- Which doctor goes to which room
- Which patient needs attention
- Which machine is available
- What to do if a machine fails
- How to route patients
- How to scale staff during rush hours
- How to replace broken equipment
- How to maintain expected service level

Docker packages the application.

Kubernetes operates the application at scale.

6. What Is Container Orchestration?

Container orchestration means automatically managing containers across infrastructure.

It includes:

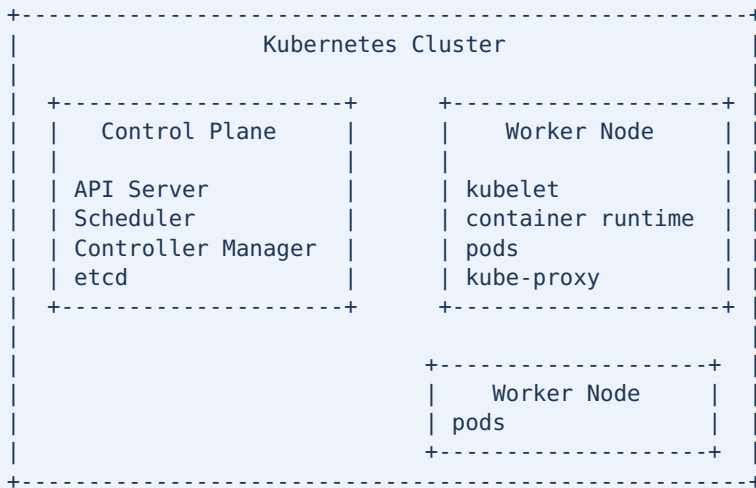
- Scheduling containers
- Starting containers
- Restarting failed containers
- Scaling containers
- Connecting containers
- Exposing applications
- Managing configuration
- Managing storage
- Rolling out new versions

- Rolling back failed versions
- Monitoring health

SIMPLE DEFINITION

Container orchestration is automated container management at scale.

7. Kubernetes in One Diagram



A Kubernetes cluster has two major parts:

1. Control plane
2. Worker nodes

8. What Is a Kubernetes Cluster?

A Kubernetes cluster is a group of machines that work together to run containerized applications.

A cluster contains:

- Control plane nodes
- Worker nodes
- Networking
- Storage integrations
- Container runtime
- Kubernetes components
- Application workloads

SIMPLE EXPLANATION

A Kubernetes cluster is a group of servers managed as one platform for running containers.

9. Control Plane

The control plane is the brain of the Kubernetes cluster.

It makes decisions. It stores cluster state. It accepts API requests. It schedules workloads. It watches whether the actual state matches the desired state.

Main control plane components:

- API Server
- etcd
- Scheduler
- Controller Manager
- Cloud Controller Manager

10. Worker Nodes

Worker nodes are machines where application containers actually run.

A worker node runs:

- kubelet
- container runtime
- kube-proxy
- pods

SIMPLE EXPLANATION

Control plane makes decisions. Worker nodes run the applications.

11. Kubernetes Cluster Analogy

Think of a delivery company.

Kubernetes Concept	Delivery Company Analogy
Cluster	Whole delivery company
Control plane	Head office
API server	Reception/front desk
Scheduler	Dispatch manager
etcd	Company database
Controller manager	Operations supervisors
Worker node	Delivery branch
Pod	Delivery package/team
Container	Item inside package
kubelet	Branch manager
Service	Customer-facing phone number/load balancer

The head office decides what should happen. Branches do the actual work.

12. What Is Desired State?

Desired state means the state you want Kubernetes to maintain.

Example:

I want 3 replicas of my web application running.

You tell Kubernetes this desired state.

Kubernetes continuously checks the actual state.

If actual state differs, Kubernetes tries to fix it.

Desired State Example

You define:

```
replicas: 3
```

YAML

Kubernetes sees:

Only 2 pods are running.

Kubernetes creates one more pod.

If there are 4 pods running, Kubernetes removes one.

This is the core Kubernetes idea:

CORE IDEA You declare what you want. Kubernetes works continuously to make reality match that declaration.

13. Desired State Diagram

```
Desired State
3 web pods should run
    |
    v
Kubernetes checks actual state
    |
    v
Actual State: only 2 pods running
    |
    v
Kubernetes creates 1 more pod
    |
    v
Actual State becomes 3 pods
```

14. Declarative vs Imperative

Kubernetes is mainly declarative.

Imperative

You tell the system exactly what command to execute.

Example: run this container now

Declarative

You describe the final desired state.

Example: I want 3 replicas of this application running.

Kubernetes figures out how to achieve and maintain it.

15. Imperative Example

```
kubectl run nginx --image=nginx
```

SHELL

This directly tells Kubernetes to run something.
Useful for quick testing.

16. Declarative Example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
```

YAML

Apply:

```
kubectl apply -f deployment.yaml
```

SHELL

This is preferred for real environments.

17. Why Declarative Is Better for Production

Declarative configuration supports:

- Version control
- Code review
- GitOps
- Repeatability
- Auditability
- Rollbacks
- Automation
- Environment consistency

Production Kubernetes should be managed using YAML, Helm, Kustomize, Terraform, or GitOps tools.

18. What Is a Pod?

A pod is the smallest deployable unit in Kubernetes.

A pod contains one or more containers.

SIMPLE DEFINITION A pod is a wrapper around one or more containers.

Most pods contain one main application container.

Some pods contain multiple containers, such as:

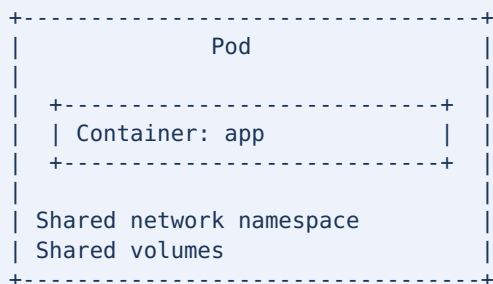
- Sidecar container
- Init container
- Helper container
- Proxy container

19. Pod vs Container

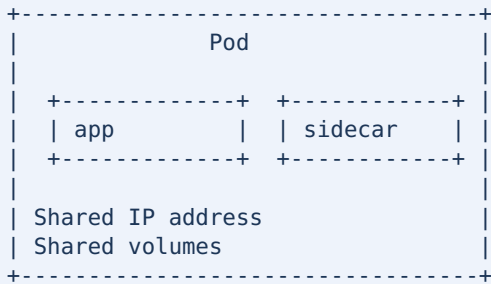
Concept	Meaning
Container	Runs application process
Pod	Kubernetes object that holds container(s)
Deployment	Manages replicas of pods
Service	Exposes pods over stable network name/IP

Docker runs containers directly. Kubernetes runs pods. Pods run containers.

20. Pod Diagram



For multi-container pod:



21. Why Kubernetes Uses Pods Instead of Containers Directly

Kubernetes uses pods because sometimes containers need to work closely together.

Containers inside the same pod share:

- Network namespace
- IP address
- localhost communication
- Volumes
- Lifecycle relationship

Examples:

- App container + logging sidecar
- App container + service mesh proxy
- App container + file sync sidecar

A pod gives Kubernetes a higher-level unit to schedule and manage.

22. What Is a Deployment?

A Deployment manages replicated stateless applications.

It ensures the desired number of pod replicas are running.

Example: Run 3 copies of nginx.

A Deployment provides:

- Replica management
- Rolling updates
- Rollbacks
- Self-healing
- Declarative app management

23. Deployment Flow

```
Deployment
  |
  v
ReplicaSet
  |
  v
Pods
  |
  v
Containers
```

The Deployment manages a ReplicaSet. The ReplicaSet manages pods. Pods run containers.

24. What Is a Service?

Pods are temporary. Pod IPs can change.

A Service provides a stable network endpoint for accessing pods.

SIMPLE DEFINITION A Service gives stable networking to a group of pods.

Example: frontend calls backend-service

Even if backend pods change, the Service remains stable.

25. Kubernetes Solves Dynamic Infrastructure

In Kubernetes:

- Pods can be created
- Pods can be deleted
- Pods can move to another node
- Pod IPs can change
- Nodes can fail
- New replicas can appear

Applications need stable ways to communicate.

Services solve this problem.

26. Kubernetes High-Level Request Flow

```
graph TD;
    UR[User Request] --> ILB[Ingress / Load Balancer];
    ILB --> S[Service];
    S --> P[Pod];
    P --> C[Container];
    C --> A[Application];
```

This is a common production traffic pattern.

27. Kubernetes Basic Object Model

Common Kubernetes objects:

Object	Purpose
Pod	Runs container(s)
Deployment	Manages stateless replicated pods
ReplicaSet	Maintains pod replicas
Service	Provides stable networking
ConfigMap	Stores non-secret configuration
Secret	Stores sensitive data
Namespace	Logical isolation
Ingress	HTTP/HTTPS routing
PersistentVolumeClaim	Requests storage
Job	Runs task to completion
CronJob	Runs scheduled task

28. What Is kubectl?

kubectl is the Kubernetes command-line tool.

It talks to Kubernetes API Server.

Examples:

```
kubectl get pods
kubectl get nodes
kubectl apply -f deployment.yaml
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl exec -it <pod-name> -- sh
```

SHELL

kubectl is similar to Docker CLI, but for Kubernetes.

29. kubectl Flow

```
User
|
v
kubectl
|
v
Kubernetes API Server
|
v
Cluster state changes
```

kubectl does not directly start containers on nodes. It sends API requests to the API Server. The control plane coordinates the rest.

30. What Happens When You Deploy an App?

When you apply a Deployment YAML:

```
kubectl apply -f deployment.yaml
```

SHELL

Internally:

1. kubectl sends YAML to API Server.
2. API Server validates request.
3. Desired state is stored in etcd.
4. Controller Manager notices new Deployment.
5. Deployment controller creates ReplicaSet.

6. ReplicaSet controller creates Pods.
7. Scheduler assigns Pods to nodes.
8. kubelet on each node sees assigned Pods.
9. Container runtime pulls images.
10. Containers start.
11. Kubernetes monitors health.

31. Deployment Internal Flow Diagram

```
kubectl apply
  |
  v
API Server
  |
  v
etcd stores desired state
  |
  v
Controller Manager creates ReplicaSet/Pods
  |
  v
Scheduler selects worker node
  |
  v
kubelet starts pod using container runtime
  |
  v
Container running
```

32. Why Kubernetes Is Important in DevOps

Kubernetes is important because modern DevOps teams need:

- Automated deployment
- Scaling
- Rollbacks
- Infrastructure abstraction
- Self-healing
- Cloud portability
- Standard deployment platform
- CI/CD integration
- Observability integration
- Security policies
- GitOps workflows

Kubernetes became a core platform engineering technology.

33. Why Kubernetes Is Important in SRE

SRE focuses on reliability.

Kubernetes helps reliability through:

- Self-healing
- Health checks
- Replica management
- Rolling updates
- Resource limits
- Autoscaling
- Failure isolation
- Declarative state
- Automated recovery

But Kubernetes also adds complexity.

Good SREs must understand both the benefits and failure modes.

34. Why Kubernetes Is Important in Platform Engineering

Platform engineering teams use Kubernetes to provide internal developer platforms.

They create reusable patterns for:

- Deployments
- Networking
- Secrets
- Monitoring
- Logging
- CI/CD
- Security
- Cost controls
- Multi-environment releases

Kubernetes becomes the foundation for a platform.

35. Kubernetes vs Docker

Feature	Docker	Kubernetes
Primary purpose	Build/run containers	Orchestrate containers
Scope	Single host mostly	Cluster
Object model	Container/image/network/volume	Pod/Deployment/Service/etc.
Scaling	Manual/basic	Declarative and automated
Self-healing	Limited	Strong
Networking	Docker networks	Cluster networking
Production orchestration	Limited	Standard
CLI	docker	kubectl

SHORT ANSWER

Docker packages and runs containers. Kubernetes manages containers at scale.

36. Kubernetes vs Docker Compose

Feature	Docker Compose	Kubernetes
Main use	Local multi-container apps	Production orchestration
Scope	Single machine	Cluster
Scheduling	No advanced scheduler	Built-in scheduler
Self-healing	Basic	Advanced
Scaling	Basic	Advanced
Networking	Compose network	Cluster networking
Storage	Docker volumes	PV/PVC/CSI
Complexity	Lower	Higher

Compose is excellent for local development. Kubernetes is designed for production scale.

37. Kubernetes Basic Example

Deployment:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello-nginx
  template:
    metadata:
      labels:
        app: hello-nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

YAML

Service:

```
apiVersion: v1
kind: Service
metadata:
  name: hello-nginx
spec:
  selector:
    app: hello-nginx
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

YAML

38. Beginner Explanation of YAML

Deployment

```
apiVersion: apps/v1
```

YAML

Uses Kubernetes apps API version.

```
kind: Deployment
```

YAML

Creates a Deployment object.

```
metadata:  
  name: hello-nginx
```

YAML

Names the Deployment.

```
replicas: 2
```

YAML

Requests two pod replicas.

```
selector:  
  matchLabels:  
    app: hello-nginx
```

YAML

Tells Deployment which pods it manages.

```
image: nginx:1.27
```

YAML

Runs nginx container image.

Service

```
kind: Service
```

YAML

Creates stable networking object.

```
selector:  
  app: hello-nginx
```

YAML

Sends traffic to pods with this label.

```
port: 80  
targetPort: 80
```

YAML

Service port 80 forwards to container port 80.

39. Common Beginner Mistakes

Mistake	Explanation
Thinking Kubernetes replaces Docker images	Kubernetes still runs container images
Thinking Pod equals container	Pod is wrapper around container(s)
Using pod IP directly	Pod IPs are temporary
Not understanding desired state	Kubernetes continuously reconciles
Manually editing running pods	Changes are not durable
Ignoring labels/selectors	Services and controllers depend on them
Using latest image tag	Bad for production
Not setting resources	Scheduling and stability issues

40. Troubleshooting Basics

Important commands:

```
kubectl get nodes
kubectl get pods
kubectl get pods -o wide
kubectl describe pod <pod>
kubectl logs <pod>
kubectl exec -it <pod> -- sh
kubectl get events
kubectl get deployments
kubectl get services
```

SHELL

Basic troubleshooting flow:

```
Pod issue
  |
  v
kubectl get pods
  |
  v
kubectl describe pod
  |
  v
kubectl logs
  |
  v
Check events, image, resources, config, networking
```

41. Production Best Practices Introduction

At a high level, production Kubernetes should include:

- Version-controlled manifests
- Resource requests and limits
- Health probes
- Non-root containers
- Proper image tags
- Secrets management
- Network policies
- RBAC
- Monitoring
- Logging
- Autoscaling
- Backup and recovery
- GitOps deployment
- Upgrade strategy
- Cost controls

Each will be explained deeply in later chapters.

42. Interview Questions and Answers

Beginner

1. What is Kubernetes?

Kubernetes is an open-source container orchestration platform used to deploy, manage, scale, and heal containerized applications.

2. Why was Kubernetes created?

It was created to manage containers at scale across many machines and automate scheduling, scaling, networking, recovery, and deployments.

3. What is container orchestration?

Container orchestration is automated management of containers across infrastructure.

4. What is a Kubernetes cluster?

A Kubernetes cluster is a group of machines that run containerized workloads under Kubernetes control.

5. What is a pod?

A pod is the smallest deployable unit in Kubernetes and contains one or more containers.

Intermediate

1. What is desired state?

Desired state is the state you declare, such as “run 3 replicas.” Kubernetes continuously works to make actual state match desired state.

2. What is the difference between Docker and Kubernetes?

Docker builds and runs containers. Kubernetes orchestrates containers across a cluster.

3. What is the control plane?

The control plane is the brain of Kubernetes. It includes API Server, etcd, Scheduler, and Controller Manager.

4. What is a worker node?

A worker node is a machine that runs application pods.

5. Why do we need Services?

Pods are temporary and IPs change. Services provide stable networking to reach pods.

Advanced

1. What happens when you apply a Deployment?

kubectl sends the manifest to API Server, state is stored in etcd, controllers create ReplicaSet and Pods, scheduler assigns nodes, kubelet starts containers through runtime.

2. Why is Kubernetes declarative?

Because users define desired state and Kubernetes controllers continuously reconcile actual state to match it.

3. Why does Kubernetes use pods instead of containers directly?

Pods allow one or more tightly coupled containers to share network, storage, and lifecycle.

4. Why is Kubernetes powerful but complex?

It automates many production concerns, but introduces distributed systems complexity in networking, storage, scheduling, security, and operations.

5. Why is Kubernetes important for platform engineering?

It provides a common foundation for building internal platforms, deployment standards, automation, and self-service developer workflows.

43. Scenario-Based Interview Questions

Scenario 1: Docker Works, Why Kubernetes?

QUESTION If Docker can run containers, why do we need Kubernetes?

EXPECTED ANSWER Docker can run containers on a host, but Kubernetes manages containers across many hosts with scheduling, scaling, self-healing, service discovery, rolling updates, and declarative state management.

Scenario 2: Pod Deleted Manually

QUESTION A pod managed by Deployment is manually deleted. What happens?

EXPECTED ANSWER The ReplicaSet controller notices actual replicas are below desired count and creates a replacement pod.

Scenario 3: Pod IP Changed

QUESTION A backend pod was recreated and its IP changed. How should frontend reach it?

EXPECTED ANSWER Frontend should use a Kubernetes Service, not direct pod IP.

Scenario 4: Need Zero-Downtime Deployment

QUESTION How does Kubernetes help deploy a new version without downtime?

EXPECTED ANSWER Deployments support rolling updates, gradually replacing old pods with new pods while maintaining availability.

44. Hands-On Labs

Lab 1: Check Cluster

```
kubectl get nodes
kubectl cluster-info
```

SHELL

Lab 2: Run a Pod Quickly

```
kubectl run nginx --image=nginx:1.27
kubectl get pods
kubectl delete pod nginx
```

SHELL

Lab 3: Apply Deployment

Create deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello-nginx
  template:
    metadata:
      labels:
        app: hello-nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

YAML

Apply:

```
kubectl apply -f deployment.yaml  
kubectl get pods  
kubectl get deployments
```

Lab 4: Delete One Pod

```
kubectl get pods  
kubectl delete pod <pod-name>  
kubectl get pods
```

Observe Kubernetes creates a replacement pod.

45. Chapter Revision Notes

- Kubernetes is a container orchestration platform.
- Docker runs containers; Kubernetes manages containers at scale.
- A Kubernetes cluster has a control plane and worker nodes.
- The control plane makes decisions.
- Worker nodes run pods.
- A pod is the smallest deployable unit.
- Desired state is the core Kubernetes idea.
- Kubernetes continuously reconciles actual state to desired state.
- Deployments manage replicated stateless applications.
- Services provide stable networking to pods.
- kubectl communicates with the API Server.
- Kubernetes is foundational for DevOps, SRE, and platform engineering.

46. Key Interview Statement

Memorize this:

KEY STATEMENT

Kubernetes is a declarative container orchestration platform. We define the desired state of applications, and Kubernetes continuously reconciles the actual state by scheduling pods, restarting failed workloads, scaling replicas, managing networking, storage, configuration, and rolling deployments across a cluster.

End of Volume 02 — Chapter 01

Next chapter:

Chapter 02 — Kubernetes Architecture Deep Dive

Topics:

- Control plane
- API Server
- etcd
- Scheduler
- Controller Manager
- Cloud Controller Manager
- Worker node
- kubelet
- kube-proxy
- container runtime
- CRI
- Cluster communication flow

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

Kubernetes Architecture Deep Dive

Chapter 02 — control plane, worker nodes, and
how every component works together.

👑 Chapter 02

API Server · etcd · Scheduler · kubelet

Chapter Purpose

In Chapter 01, you learned what Kubernetes is and why it exists.

Now you will learn the internal architecture of Kubernetes.

This chapter explains how the Kubernetes control plane, worker nodes, and supporting components work together to run containers reliably across a cluster.

This chapter is extremely important for interviews because many Kubernetes questions are architecture-based.

Learning Objectives

By the end of this chapter, you should be able to explain:

- Kubernetes cluster architecture.
- Control plane components.
- Worker node components.
- API Server.
- etcd.
- Scheduler.
- Controller Manager.
- Cloud Controller Manager.
- kubelet.
- kube-proxy.
- Container runtime.
- CRI.
- CNI.
- CSI.
- How Kubernetes deploys a pod internally.
- How components communicate.
- Common architecture troubleshooting scenarios.
- Production architecture best practices.

1. Kubernetes Architecture Overview

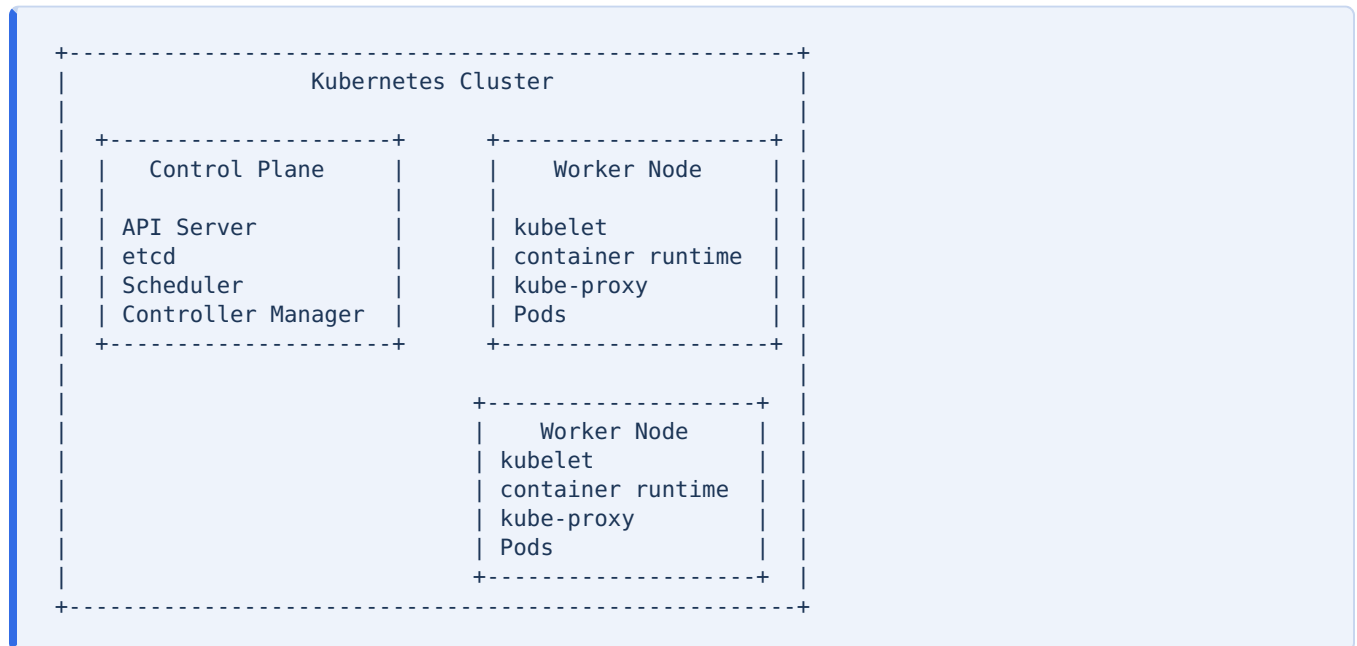
A Kubernetes cluster has two major sections:

1. Control Plane
2. Worker Nodes

The control plane manages the cluster.

Worker nodes run application workloads.

High-level architecture:



2. Control Plane

What Is the Control Plane?

The control plane is the management layer of Kubernetes.

It is responsible for:

- Accepting API requests.
- Storing cluster state.
- Scheduling pods.
- Running controllers.
- Watching cluster health.
- Reconciling desired state.
- Coordinating cluster operations.

SIMPLE DEFINITION

The control plane is the brain of the Kubernetes cluster.

Control Plane Components

Component	Purpose
API Server	Entry point to Kubernetes
etcd	Cluster database
Scheduler	Assigns pods to nodes
Controller Manager	Runs controllers
Cloud Controller Manager	Integrates cloud provider resources

3. API Server

What Is API Server?

The Kubernetes API Server is the front door of the cluster.

Every operation goes through the API Server.

Examples:

```
kubectl get pods
kubectl apply -f deployment.yaml
kubectl delete service app
kubectl scale deployment web --replicas=5
```

SHELL

All these commands talk to the API Server.

Why API Server Exists

Kubernetes needs one central, consistent interface for all operations.

The API Server provides:

- Authentication
- Authorization
- Admission control
- API validation
- Object persistence
- Cluster state access
- Watch mechanism

No component should directly modify etcd except API Server.

API Server Responsibilities

Responsibility	Explanation
Authentication	Who are you?
Authorization	Are you allowed?
Admission control	Should this request be modified or rejected?
Validation	Is the object valid?
Persistence	Store accepted state in etcd
Watch API	Notify components about changes

API Server Flow

```
kubectl / client
  |
  v
API Server
  |
  +--> Authentication
  |
  +--> Authorization
  |
  +--> Admission Controllers
  |
  +--> Validation
  |
  v
etcd
```

4. etcd

What Is etcd?

etcd is a distributed key-value database used by Kubernetes to store cluster state.

It stores information such as:

- Nodes
- Pods
- Deployments
- Services
- ConfigMaps
- Secrets
- RBAC objects

- Cluster configuration

SIMPLE DEFINITION

etcd is the source of truth for Kubernetes cluster state.

Why etcd Is Important

If etcd is lost, the cluster loses its stored state.

Running pods may continue temporarily, but Kubernetes control and recovery are severely affected.

That is why etcd backup is critical.

What etcd Stores

Examples:

- Pods
- Deployments
- ReplicaSets
- Services
- Secrets
- ConfigMaps
- Namespaces
- RBAC rules
- Node information
- Endpoint information

etcd does not store container image layers or application logs.

It stores Kubernetes object state.

etcd Production Notes

Production etcd should have:

- Regular backups
- Encryption at rest
- Restricted access
- High availability
- Monitoring
- Low disk latency
- Secure TLS communication

etcd is very sensitive to disk performance.

Slow disks can impact Kubernetes API performance.

5. Scheduler

What Is Scheduler?

The scheduler decides which worker node should run a pod.

When a pod is created, it may not yet have a node assigned.

The scheduler watches for unscheduled pods and selects a suitable node.

SIMPLE DEFINITION

The scheduler assigns pods to worker nodes.

Why Scheduler Exists

In a cluster with many nodes, Kubernetes must decide where each pod should run.

The scheduler considers:

- CPU requests
- Memory requests
- Node capacity
- Taints and tolerations
- Node selectors
- Node affinity
- Pod affinity
- Pod anti-affinity
- Topology spread constraints
- Volume constraints
- Node readiness

Scheduler Flow

```
graph TD; A[New Pod created] --> B[Pod has no node assigned]; B --> C[Scheduler watches unscheduled pod]; C --> D[Filters unsuitable nodes]; D --> E[Scores remaining nodes]; E --> F[Selects best node]; F --> G[Writes binding through API Server];
```

Filtering and Scoring

The scheduler works in two high-level stages:

Filter → Score

Filter

Remove nodes that cannot run the pod.

Examples:

- Not enough CPU.
- Not enough memory.
- Node is not ready.
- Taint not tolerated.
- Required node selector not matched.

Score

Rank remaining nodes.

Examples:

- Prefer less loaded node.
- Prefer topology balance.
- Prefer affinity match.

6. Controller Manager

What Is Controller Manager?

The Controller Manager runs Kubernetes controllers.

A controller is a loop that watches actual state and compares it with desired state.

If actual state differs, the controller takes action.

SIMPLE DEFINITION Controllers are reconciliation loops that make actual state match desired state.

Controller Loop

```
graph TD
    A[Watch desired state] --> B[Observe actual state]
    B --> C[Compare desired vs actual]
    C --> D[Take corrective action]
    D --> E[Repeat forever]
```

Common Controllers

Controller	Responsibility
Deployment Controller	Manages Deployments
ReplicaSet Controller	Maintains pod replica count
Node Controller	Watches node health
Job Controller	Manages Jobs
EndpointSlice Controller	Updates service endpoints
Namespace Controller	Handles namespace deletion
ServiceAccount Controller	Creates default service accounts

Example: ReplicaSet Controller

Desired state:

```
replicas: 3
```

YAML

Actual state:

2 pods running

ReplicaSet controller creates one more pod.

Actual state becomes:

3 pods running

7. Cloud Controller Manager

What Is Cloud Controller Manager?

Cloud Controller Manager integrates Kubernetes with cloud provider APIs.

In cloud environments, Kubernetes often needs to manage external cloud resources.

Examples:

- Load balancers
- Routes
- Node metadata
- Volumes
- Public IPs

Cloud Controller Manager separates cloud-specific logic from core Kubernetes.

Example

When you create a Service of type LoadBalancer in AWS:

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: LoadBalancer
  ports:
    - port: 80
  selector:
    app: web
```

YAML

The cloud integration creates an AWS load balancer.

In EKS, AWS controllers handle cloud-specific resources.

8. Worker Node

What Is a Worker Node?

A worker node is a machine that runs application workloads.

A worker node may be:

- Physical server
- Virtual machine
- Cloud instance
- EC2 instance in EKS

Worker nodes run pods.

Worker Node Components

Component	Purpose
kubelet	Node agent
container runtime	Runs containers
kube-proxy	Handles service networking
CNI plugin	Pod networking
CSI plugin	Storage integration

9. kubelet

What Is kubelet?

kubelet is the Kubernetes agent running on every worker node.

It communicates with the API Server.

It ensures pods assigned to that node are running.

SIMPLE DEFINITION

kubelet is the node-level agent that makes sure containers run as Kubernetes requested.

kubelet Responsibilities

- Register node with cluster.
- Watch assigned pods.
- Ask container runtime to start containers.

- Report pod status.
- Report node status.
- Run liveness/readiness/startup probes.
- Mount volumes.
- Manage pod lifecycle on the node.

kubelet Flow

```
API Server says:
"Run this pod on node-1"
  |
  v
kubelet on node-1 sees assignment
  |
  v
kubelet asks container runtime to pull image
  |
  v
container runtime starts containers
  |
  v
kubelet reports status back to API Server
```

10. Container Runtime

What Is Container Runtime?

The container runtime is responsible for running containers.

Common Kubernetes runtimes:

- containerd
- CRI-O

Older clusters sometimes used Docker Engine through Dockershim, but modern Kubernetes commonly uses containerd.

Runtime Responsibilities

- Pull images
- Unpack images
- Create containers
- Start containers
- Stop containers
- Report container status
- Manage container lifecycle

Kubernetes Runtime Flow

```
graph TD
    kubelet --> CRI
    CRI --> containerd["containerd / CRI-O"]
    containerd --> runc
    runc --> Linux_kernel["Linux kernel"]
    Linux_kernel --> container_process["container process"]
```

11. CRI

What Is CRI?

CRI stands for Container Runtime Interface.

It is the Kubernetes interface for communicating with container runtimes.

SIMPLE DEFINITION CRI allows kubelet to talk to different container runtimes in a standard way.

Why CRI Exists

Without CRI, Kubernetes would need custom integration for every runtime.

CRI allows Kubernetes to work with:

- containerd
- CRI-O
- other compatible runtimes

This makes Kubernetes more modular.

12. kube-proxy

What Is kube-proxy?

kube-proxy is a networking component that runs on each node.

It helps implement Kubernetes Service networking.

When you create a Service, kube-proxy configures rules so traffic can reach the correct pods.

kube-proxy Responsibilities

- Watches Services and EndpointSlices.
- Programs networking rules.
- Helps route service traffic to pods.
- Supports ClusterIP behavior.
- Uses iptables, IPVS, or other modes depending on configuration.

Service Traffic Concept

```
Client
  |
  v
Service IP
  |
  v
kube-proxy rules
  |
  v
One of the backend pods
```

13. CNI

What Is CNI?

CNI stands for Container Network Interface.

It defines how pod networking is configured.

CNI plugins provide pod IPs and connectivity.

Examples:

- Calico
- Cilium
- Flannel
- Weave Net
- AWS VPC CNI
- Azure CNI

Why CNI Exists

Kubernetes does not implement all networking itself.

It defines networking requirements and uses CNI plugins to configure actual networking.

CNI is responsible for:

- Assigning pod IPs

- Creating network interfaces
- Connecting pods to network
- Enforcing network policies depending on plugin
- Routing pod traffic

14. CSI

What Is CSI?

CSI stands for Container Storage Interface.

It allows Kubernetes to work with different storage systems.

Examples:

- AWS EBS CSI Driver
- AWS EFS CSI Driver
- Azure Disk CSI
- GCE Persistent Disk CSI
- Ceph CSI
- NetApp CSI

CSI provides standard storage integration.

CSI Responsibilities

- Provision volumes
- Attach volumes
- Mount volumes
- Resize volumes
- Snapshot volumes
- Delete volumes

Storage is covered deeply in later chapters.

15. Kubernetes Component Communication

Kubernetes components primarily communicate through the API Server.

IMPORTANT PRINCIPLE

The API Server is the central communication hub.

Example:

- kubectl talks to API Server.
- Scheduler watches API Server.

- Controllers watch API Server.
- kubelet reports to API Server.
- kube-proxy watches Services through API Server.

Communication Diagram

```

kubectll
|
v
API Server <-----> etcd
^
|
+----- Scheduler
|
+----- Controller Manager
|
+----- kubelet
|
+----- kube-proxy

```

16. What Happens When a Pod Is Created?

Example:

```
kubectl apply -f pod.yaml
```

SHELL

Internal flow:

1. kubectl sends request to API Server.
2. API Server authenticates user.
3. API Server authorizes request.
4. Admission controllers process request.
5. API Server validates object.
6. API Server stores pod in etcd.
7. Scheduler sees unscheduled pod.
8. Scheduler selects node.
9. API Server records node binding.
10. kubelet on selected node sees pod assignment.
11. kubelet asks container runtime to pull image.
12. Runtime starts container.
13. CNI configures pod networking.
14. Volumes are mounted if needed.
15. kubelet reports pod status.
16. Pod becomes Running.

Pod Creation Flow Diagram

```
kubectl apply
  |
  v
API Server
  |
  v
etcd stores pod
  |
  v
Scheduler assigns node
  |
  v
kubelet notices assigned pod
  |
  v
Container runtime pulls image
  |
  v
CNI sets up networking
  |
  v
Volumes mounted
  |
  v
Container starts
  |
  v
Pod Running
```

17. Kubernetes API Objects

Everything in Kubernetes is represented as an API object.

Examples:

- Pod
- Deployment
- Service
- ConfigMap
- Secret
- Namespace
- Node
- Job
- CronJob
- Ingress
- PersistentVolumeClaim

Each object has:

```
apiVersion:  
kind:  
metadata:  
spec:  
status:
```

18. spec vs status

spec

spec means desired state.

Example:

```
spec:  
  replicas: 3
```

You define this.

status

status means observed actual state.

Example:

```
status:  
  availableReplicas: 2
```

Kubernetes updates this.

spec/status Concept

```
graph TD  
    A[User writes spec] --> B[Kubernetes works]  
    B --> C[Kubernetes updates status]
```

This is fundamental to Kubernetes architecture.

19. Reconciliation

Reconciliation is the process of making actual state match desired state.

Example:

Desired: 3 replicas

Actual: 2 replicas

Controller action: Create 1 more pod

This loop runs continuously.

20. Kubernetes Is Eventually Consistent

Kubernetes does not always make changes instantly.

It works through asynchronous controllers.

Example:

You apply a Deployment.

Pods may take time to appear because:

- Controller must create ReplicaSet.
- ReplicaSet must create Pods.
- Scheduler must assign nodes.
- Images must pull.
- Containers must start.
- Probes must pass.

This is normal.

21. High Availability Control Plane

Production clusters usually run multiple control plane replicas.

Why?

- Avoid single point of failure.
- Improve availability.
- Support upgrades.
- Survive node failure.

Managed Kubernetes services like EKS manage control plane availability for you.

Self-managed clusters require careful HA design.

22. Single Control Plane vs HA Control Plane

Area	Single Control Plane	HA Control Plane
Availability	Lower	Higher
Complexity	Lower	Higher
Production suitability	Poor	Good
etcd resilience	Lower	Higher
Upgrade safety	Lower	Better

Production clusters should use HA control planes.

23. Managed Kubernetes Architecture

In managed services like Amazon EKS:

AWS manages:

- Control plane
- API Server availability
- Control plane scaling
- Control plane patching
- etcd management
- Control plane high availability

You manage:

- Worker nodes
- Add-ons
- Workloads
- IAM/RBAC
- Networking configuration
- Security policies
- Application deployments

24. Architecture Troubleshooting

Problem: kubectl Cannot Connect

Check:

```
kubectl cluster-info
kubectl config current-context
kubectl get nodes
```

Possible causes:

- Wrong kubeconfig
- API Server unreachable
- Expired credentials
- Network/VPN issue
- IAM/RBAC issue
- Cluster down

Problem: Pod Stuck Pending

Possible causes:

- No available node resources
- Taints not tolerated
- Node selector mismatch
- PVC not bound
- Image pull not started because unscheduled
- Scheduler cannot find suitable node

Commands:

```
kubectl describe pod <pod>
kubectl get events
kubectl get nodes
```

Problem: Node NotReady

Possible causes:

- kubelet down
- container runtime down
- network plugin issue
- node resource pressure
- node unreachable
- disk pressure
- kernel/container runtime problem

Commands:


```
kubectl describe node <node>
kubectl get nodes
```

SHELL

On node:

```
systemctl status kubelet
systemctl status containerd
journalctl -u kubelet -xe
```

SHELL

25. Production Best Practices

- Use managed Kubernetes when possible.
- Use HA control plane for production.
- Back up etcd in self-managed clusters.
- Monitor API Server latency.
- Monitor etcd health.
- Monitor scheduler and controller errors.
- Keep worker nodes healthy.
- Patch nodes regularly.
- Use supported container runtime.
- Keep CNI add-on healthy.
- Use RBAC and least privilege.
- Use GitOps or declarative deployment.
- Avoid manual changes to live objects.
- Understand spec/status and reconciliation.

26. Common Mistakes

Mistake	Explanation
Thinking kubectl talks to nodes directly	kubectl talks to API Server
Thinking etcd stores logs/images	etcd stores cluster state
Ignoring scheduler events	Scheduling failures are visible in events
Not understanding controllers	Controllers drive desired state
Treating pods as permanent	Pods are replaceable
Ignoring kubelet	kubelet is critical on each node
Ignoring CNI	Pod networking depends on CNI
Not backing up etcd	Dangerous for self-managed clusters

27. Interview Questions and Answers

Beginner

1. What are the main parts of Kubernetes architecture?

Control plane and worker nodes.

2. What is API Server?

The API Server is the entry point to Kubernetes and handles API requests.

3. What is etcd?

etcd is the key-value database that stores Kubernetes cluster state.

4. What is kubelet?

kubelet is the node agent that ensures assigned pods run on the worker node.

5. What is kube-proxy?

kube-proxy helps implement Kubernetes Service networking on nodes.

Intermediate

1. What does the scheduler do?

The scheduler assigns unscheduled pods to suitable worker nodes based on resources, constraints, and policies.

2. What does Controller Manager do?

It runs controllers that continuously reconcile actual state with desired state.

3. What is CRI?

CRI is the Container Runtime Interface used by kubelet to communicate with container runtimes.

4. What is CNI?

CNI is the Container Network Interface used to configure pod networking.

5. What is CSI?

CSI is the Container Storage Interface used to integrate storage systems with Kubernetes.

Advanced

1. What happens when you create a pod?

The request goes to API Server, is stored in etcd, scheduler assigns a node, kubelet on that node starts the pod using the container runtime, CNI configures networking, and kubelet reports status.

2. Why should components communicate through API Server?

The API Server provides authentication, authorization, validation, admission control, consistency, and central state access.

3. What is reconciliation?

Reconciliation is the continuous process where controllers make actual cluster state match desired state.

4. Why is etcd critical?

etcd stores all cluster state. Losing etcd can mean losing the cluster's desired state.

5. Why is Kubernetes eventually consistent?

Changes are processed asynchronously by controllers, scheduler, kubelet, and other components.

28. Scenario-Based Interview Questions

Scenario 1: Pod Stuck Pending

QUESTION A pod is stuck in Pending. Which component is involved?

EXPECTED ANSWER The scheduler is involved. It may not find a suitable node due to resource shortage, taints, node selectors, affinity rules, or unbound PVC. Check `kubectl describe pod` and events.

Scenario 2: Node NotReady

QUESTION A node becomes NotReady. What components do you check?

EXPECTED ANSWER Check kubelet, container runtime, node network, CNI, disk pressure, memory pressure, and node connectivity to API Server.

Scenario 3: kubectl Not Working

QUESTION kubectl cannot connect to cluster. What do you check?

EXPECTED ANSWER Check kubeconfig context, credentials, network/VPN, API Server endpoint, RBAC/IAM, and cluster status.

Scenario 4: Deployment Not Creating Pods

QUESTION Deployment exists but pods are not appearing. What do you check?

EXPECTED ANSWER Check Deployment events, ReplicaSet, controller manager health, labels/selectors, quotas, and API events.

29. Hands-On Labs

Lab 1: View Cluster Components

```
kubectl get nodes
kubectl get pods -A
kubectl cluster-info
```

SHELL

Lab 2: Inspect Node

```
kubectl describe node <node-name>
```

SHELL

Look for:

- Conditions
- Capacity
- Allocatable
- Taints
- Events

Lab 3: Watch Scheduling

Create pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-arch-demo
spec:
  containers:
    - name: nginx
      image: nginx:1.27
```

YAML

Apply:

```
kubectl apply -f pod.yaml
kubectl get pod nginx-arch-demo -o wide
```

SHELL

Observe which node it was scheduled on.

Lab 4: View Events

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

SHELL

Events often reveal scheduling, image pull, and runtime problems.

30. Chapter Revision Notes

- Kubernetes has a control plane and worker nodes.
- API Server is the cluster entry point.
- etcd stores cluster state.
- Scheduler assigns pods to nodes.
- Controller Manager runs reconciliation loops.
- kubelet runs on every worker node.
- Container runtime runs containers.

- kube-proxy helps implement Service networking.
- CNI provides pod networking.
- CSI provides storage integration.
- Kubernetes components communicate mainly through API Server.
- spec is desired state.
- status is observed state.
- Controllers reconcile actual state with desired state.

31. Key Interview Statement

Memorize this:

KEY STATEMENT

Kubernetes architecture is based on a control plane and worker nodes. The API Server is the central entry point, etcd stores cluster state, the scheduler assigns pods to nodes, controllers reconcile desired state, kubelet runs pods on worker nodes through a container runtime, CNI provides networking, CSI provides storage, and kube-proxy helps implement Service networking.

End of Volume 02 — Chapter 02

Next chapter:

Chapter 03 — Pods Deep Dive

Topics:

- What is a pod?
- Why pods exist
- Pod lifecycle
- Pod phases
- Container states
- Init containers
- Sidecars
- Multi-container pods
- Ephemeral containers
- Pod networking
- Pod storage
- Pod restart policies
- Pod troubleshooting

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

Pods Deep Dive

Chapter 03 — the smallest deployable unit, from basics to production troubleshooting.

👑 Chapter 03

[Lifecycle](#) · [Probes](#) · [Sidecars](#) · [Troubleshooting](#)

Chapter Purpose

A Pod is the smallest deployable unit in Kubernetes.

You cannot understand Kubernetes properly without understanding Pods.

Deployments create Pods.

StatefulSets create Pods.

DaemonSets create Pods.

Jobs create Pods.

Services send traffic to Pods.

Schedulers place Pods on nodes.

kubelet runs Pods on worker nodes.

This chapter explains Pods clearly from beginner level to production troubleshooting level.

Learning Objectives

By the end of this chapter, you should be able to explain:

- What a Pod is.
- Why Kubernetes uses Pods instead of containers directly.
- Pod lifecycle.
- Pod phases.
- Container states.
- Init containers.
- Sidecar containers.
- Multi-container Pods.
- Ephemeral containers.
- Pod networking.
- Pod storage.
- Restart policies.
- Pod health checks.
- Pod troubleshooting.
- Pod interview answers.

1. What Is a Pod?

A Pod is the smallest deployable unit in Kubernetes.

A Pod wraps one or more containers.

SIMPLE DEFINITION A Pod is a Kubernetes object that runs one or more containers together.

Most Pods run one main application container.

Example:

```
Pod
└─ Container: nginx
```

Some Pods run multiple containers.

Example:

```
Pod
├─ Container: app
└─ Container: log-sidecar
```

2. Why Kubernetes Uses Pods

Docker runs containers directly.

Kubernetes does not schedule containers directly.

Kubernetes schedules Pods.

Why?

Because some containers must work closely together and share:

- Network namespace
- IP address
- localhost communication
- Volumes
- Lifecycle
- Scheduling location

A Pod provides a shared environment for related containers.

3. Real-World Analogy

Think of a Pod as an apartment.

Inside the apartment, people share:

- Same address

- Same rooms
- Same kitchen
- Same utilities
- Same storage

A container is like a person living inside the apartment.

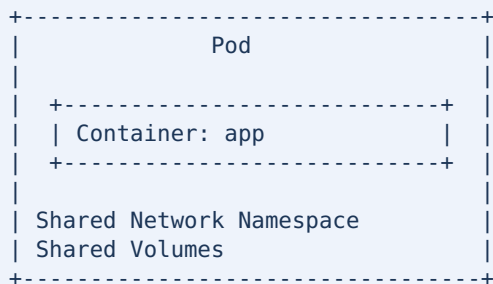
Most apartments may have one person.

Some apartments may have roommates.

Similarly, most Pods have one container, but some Pods have multiple containers that work together.

4. Pod Architecture

Single-container Pod:



Multi-container Pod:



5. Pod vs Container

Pod	Container
Kubernetes object	Runtime process environment
Can contain one or more containers	Runs one application process
Has one IP address	Shares Pod IP
Scheduled on node	Runs inside Pod
Can define volumes	Mounts Pod volumes
Managed by Kubernetes	Managed through container runtime

IMPORTANT ANSWER

Kubernetes schedules Pods, not individual containers.

6. Pod YAML Example

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

YAML

Apply:

```
kubectl apply -f pod.yaml
```

SHELL

Check:

```
kubectl get pods
```

SHELL

7. YAML Line-by-Line Explanation

```
apiVersion: v1
```

YAML

Uses Kubernetes core API version.

```
kind: Pod
```

YAML

Creates a Pod object.

```
metadata:  
  name: nginx-pod
```

YAML

Sets Pod name.

```
labels:  
  app: nginx
```

YAML

Adds label used for selection and organization.

```
spec:
```

YAML

Defines desired Pod configuration.

```
containers:
```

YAML

Lists containers inside the Pod.

```
  name: nginx
```

YAML

Container name.

```
    image: nginx:1.27
```

YAML

Container image.

```
      containerPort: 80
```

YAML

Documents that container listens on port 80.

8. Pod Lifecycle

A Pod goes through lifecycle phases.

Common phases:

Phase	Meaning
Pending	Accepted but not fully running
Running	At least one container running
Succeeded	All containers completed successfully
Failed	Containers terminated with failure
Unknown	State cannot be determined

Pod Lifecycle Diagram

```
Pod Created
|
v
Pending
|
v
Scheduled to Node
|
v
Images Pulled
|
v
Containers Created
|
v
Running
|
+--> Succeeded
|
+--> Failed
```

9. Pending Phase

A Pod in Pending means Kubernetes accepted the Pod, but it is not fully running yet.

Reasons:

- Scheduler has not assigned node.
- No node has enough CPU/memory.
- Taints not tolerated.
- PVC not bound.

- Image is still pulling.
- Container is not created yet.

Troubleshoot:

```
kubectl describe pod <pod-name>
kubectl get events
```

SHELL

10. Running Phase

A Pod is Running when it has been bound to a node and at least one container is running.

Check:

```
kubectl get pod <pod-name>
kubectl get pod <pod-name> -o wide
```

SHELL

-o wide shows the node and Pod IP.

11. Succeeded Phase

A Pod is Succeeded when all containers completed successfully and will not be restarted.

Common for:

- Jobs
- One-time tasks
- Batch workloads

Example:

```
backup job completed successfully
```

12. Failed Phase

A Pod is Failed when one or more containers terminated in failure and will not be restarted.

Common causes:

- Application error
- Bad command
- Permission issue
- Failed batch job

- Non-zero exit code

Check logs:

```
kubectl logs <pod-name>
```

SHELL

13. Container States

Inside a Pod, containers have states:

State	Meaning
Waiting	Container not started yet
Running	Container is running
Terminated	Container stopped

Examples of Waiting reasons:

- ContainerCreating
- ImagePullBackOff
- CrashLoopBackOff
- ErrImagePull

14. ContainerCreating

ContainerCreating means Kubernetes is preparing the container.

It may be:

- Pulling image
- Creating container
- Mounting volumes
- Setting up networking

If it stays too long, check:

```
kubectl describe pod <pod-name>
```

SHELL

15. ImagePullBackOff

ImagePullBackOff means Kubernetes failed to pull the container image and is backing off before retrying.

Common causes:

- Wrong image name
- Wrong tag
- Private registry auth missing
- Registry unavailable
- Network problem
- Rate limit

Check:

```
kubectl describe pod <pod-name>
```

SHELL

Look at Events.

16. CrashLoopBackOff

CrashLoopBackOff means container starts, crashes, restarts, and repeats.

Common causes:

- Application bug
- Missing environment variable
- Wrong command
- Config error
- Database unavailable
- Permission issue
- Failed startup probe
- App exits immediately

Commands:

```
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
kubectl describe pod <pod-name>
```

SHELL

--previous shows logs from the previous crashed container instance.

17. Pod Restart Policy

Pod restart policy controls restart behavior for containers.

Values:

Policy	Meaning
Always	Always restart containers
OnFailure	Restart only on non-zero exit
Never	Never restart

Example:

```
spec:
  restartPolicy: Always
```

YAML

Deployments usually use Always.

Jobs often use OnFailure or Never.

18. Pod Networking

Every Pod gets its own IP address.

Containers inside the same Pod share the same network namespace.

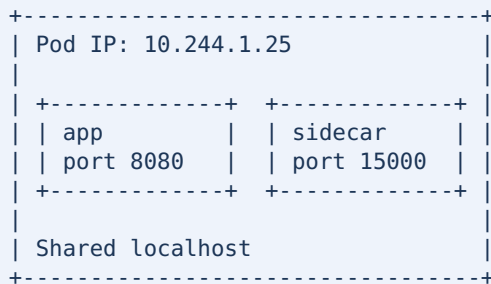
This means containers in one Pod can communicate using:

```
localhost
```

Example:

```
app container → localhost:15000 → sidecar container
```

Pod Networking Diagram



19. Pod Storage

Pods can define volumes.

Containers inside the Pod can mount those volumes.

Example:

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-demo
spec:
  containers:
    - name: app
      image: busybox
      command: ["sh", "-c", "echo hello > /data/message.txt && sleep 3600"]
      volumeMounts:
        - name: shared-data
          mountPath: /data
  volumes:
    - name: shared-data
      emptyDir: {}
```

YAML

emptyDir creates temporary storage for the Pod lifecycle.

20. emptyDir

emptyDir is created when a Pod is assigned to a node.

It exists as long as the Pod exists.

If the Pod is deleted, the data is deleted.

Use cases:

- Temporary files
- Shared files between containers

- Scratch space
- Sidecar communication

Not for persistent data.

21. Multi-Container Pods

Use multi-container Pods only when containers are tightly coupled.

Good use cases:

- App + log sidecar
- App + service mesh proxy
- App + config reloader
- App + local helper proxy

Bad use case:

- Frontend + backend + database in one Pod

Those should usually be separate Deployments/Pods.

22. Sidecar Container

A sidecar container supports the main application container.

Examples:

- Logging agent
- Proxy
- Metrics exporter
- File sync
- Service mesh proxy
- Config reloader

Sidecar example:

```
Pod
├─ app container
└─ log sidecar
```

The sidecar is not the main business app but helps it.

23. Init Containers

Init containers run before main containers.

They must complete successfully before app containers start.

Use cases:

- Wait for dependency
- Run database migration
- Prepare config
- Download files
- Set permissions

Example:

```
apiVersion: v1
kind: Pod
metadata:
  name: init-demo
spec:
  initContainers:
    - name: init
      image: busybox
      command: ["sh", "-c", "echo preparing && sleep 5"]
  containers:
    - name: app
      image: nginx:1.27
```

YAML

24. Init Container Flow

```
Pod starts
|
v
Init container 1 runs
|
v
Init container 1 succeeds
|
v
Main containers start
```

If an init container fails, Kubernetes restarts it according to Pod restart policy.

25. Ephemeral Containers

Ephemeral containers are temporary containers added to a running Pod for debugging.

They are useful when the main container image is minimal and has no shell or debugging tools.

Example command:

```
kubectl debug -it <pod-name> --image=busybox --target=<container-name>
```

SHELL

Use cases:

- Debug distroless containers
- Network troubleshooting
- Process inspection
- Temporary diagnostics

Ephemeral containers are not for normal application workloads.

26. Pod Labels

Labels are key-value pairs attached to Pods.

Example:

```
metadata:
  labels:
    app: nginx
    env: dev
```

YAML

Labels are used by:

- Services
- Deployments
- ReplicaSets
- NetworkPolicies
- Monitoring
- Cost allocation
- Organization

27. Pod Annotations

Annotations are key-value metadata like labels, but not used for selection.

Example:

```
metadata:
  annotations:
    owner: "platform-team"
    description: "nginx test pod"
```

YAML

Use annotations for:

- Tool metadata
- Observability metadata
- Build information
- Sidecar injection settings
- Descriptions

28. Pod Health Probes

Kubernetes supports three main probes:

Probe	Purpose
livenessProbe	Should container be restarted?
readinessProbe	Should Pod receive traffic?
startupProbe	Has slow-starting app finished startup?

Example:

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
```

YAML

Probes are critical for production workloads.

29. Pod Resource Requests and Limits

Example:

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

YAML

Requests are used for scheduling.

Limits control maximum usage.

If memory exceeds limit, container may be OOMKilled.

30. Pod Security Context

Security context controls security settings.

Example:

```
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
```

YAML

Use it to harden Pods.

31. Static Pods

Static Pods are managed directly by kubelet, not API Server.

They are usually used for control plane components in kubeadm clusters.

Static Pod manifests are stored on the node.

kubelet watches a directory and starts Pods from manifests.

Most application teams do not create static Pods.

32. Mirror Pods

For static Pods, kubelet creates mirror Pod objects in the API Server so they are visible through kubectl.

But the actual source of truth is the manifest file on the node.

33. Pod Troubleshooting Commands

SHELL

```
kubectl get pods
kubectl get pods -o wide
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
kubectl exec -it <pod-name> -- sh
kubectl get events
kubectl delete pod <pod-name>
```

For multi-container Pods:

SHELL

```
kubectl logs <pod-name> -c <container-name>
kubectl exec -it <pod-name> -c <container-name> -- sh
```

34. Common Pod Problems

Problem	Common Cause
Pending	Scheduling issue
ImagePullBackOff	Image pull failure
CrashLoopBackOff	App repeatedly crashing
OOMKilled	Memory limit exceeded
ContainerCreating stuck	Network/volume/image issue
Running but not ready	Readiness probe failing
Terminating stuck	Finalizers, node issue, graceful shutdown
Error	Container exited with failure

35. Troubleshooting: Pending Pod

Check:

SHELL

```
kubectl describe pod <pod-name>
kubectl get events
```


Common causes:

- Not enough CPU/memory
- Node selector mismatch
- Taints not tolerated
- PVC unbound
- No available nodes

36. Troubleshooting: ImagePullBackOff

Check:

```
kubectl describe pod <pod-name>
```

SHELL

Look at Events.

Fix:

- Correct image name/tag
- Add imagePullSecret
- Check registry permissions
- Check network
- Check registry availability

37. Troubleshooting: CrashLoopBackOff

Commands:

```
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
kubectl describe pod <pod-name>
```

SHELL

Common fixes:

- Fix app config
- Add missing env vars
- Correct command/args
- Fix permissions
- Increase startup delay/probe config
- Ensure dependencies are reachable

38. Troubleshooting: OOMKilled

Check:

```
kubectl describe pod <pod-name>
kubectl top pod <pod-name>
```

SHELL

Cause:

Container exceeded memory limit.

Fix:

- Increase memory limit
- Fix memory leak
- Tune application memory
- Reduce workers
- Add monitoring

39. Production Best Practices

- Do not create naked Pods for production apps; use Deployments, StatefulSets, Jobs, or DaemonSets.
- Use specific image tags.
- Define resource requests and limits.
- Add liveness/readiness/startup probes.
- Run as non-root.
- Avoid privileged Pods.
- Use ConfigMaps and Secrets for configuration.
- Use Services instead of direct Pod IPs.
- Use labels consistently.
- Avoid putting unrelated containers in one Pod.
- Use ephemeral containers for debugging minimal images.
- Keep Pods replaceable and stateless unless using proper storage.

40. Common Mistakes

Mistake	Why Bad
Creating standalone Pods in production	No self-healing controller
Using latest tag	Unpredictable deployments
No resource requests	Poor scheduling
No readiness probe	Traffic may go to unready Pod
Using Pod IP directly	IP changes
Too many containers in one Pod	Tight coupling
Running as root	Security risk
Ignoring events	Troubleshooting becomes harder

41. Interview Questions and Answers

Beginner

1. What is a Pod?

A Pod is the smallest deployable unit in Kubernetes and contains one or more containers.

2. Why does Kubernetes use Pods?

Pods allow containers to share network, storage, and lifecycle when they must work closely together.

3. Can a Pod have multiple containers?

Yes, but most Pods have one main container. Multiple containers are used for tightly coupled helper patterns such as sidecars.

4. What is Pod IP?

Each Pod gets its own IP address in the cluster network.

5. What happens if a standalone Pod dies?

If it is not managed by a controller, Kubernetes may not recreate it automatically.

Intermediate

1. What is an init container?

An init container runs before app containers and must complete successfully before the main containers start.

2. What is a sidecar container?

A sidecar is a helper container that supports the main application container, such as logging, proxying, or metrics.

3. What is CrashLoopBackOff?

It means a container repeatedly starts, crashes, and Kubernetes backs off before restarting it.

4. What is ImagePullBackOff?

It means Kubernetes failed to pull the container image and is retrying with backoff.

5. Difference between liveness and readiness probe?

Liveness decides whether to restart a container. Readiness decides whether the Pod should receive traffic.

Advanced

1. Why should unrelated containers not be placed in one Pod?

Because containers in a Pod share lifecycle and scheduling. Unrelated services should be independently deployed and scaled.

2. What is an ephemeral container?

A temporary debugging container added to an existing Pod.

3. What is emptyDir?

A temporary volume created for a Pod that exists as long as the Pod exists.

4. Why is direct Pod IP usage discouraged?

Pod IPs are temporary and change when Pods are recreated. Services provide stable access.

5. What causes OOMKilled?

The container exceeded its memory limit and was killed by the system.

42. Scenario-Based Interview Questions

Scenario 1: Pod Stuck Pending

QUESTION A Pod stays Pending. What do you check?

EXPECTED ANSWER Check `kubectl describe pod`, events, node resources, taints/tolerations, node selectors, affinity rules, and PVC binding.

Scenario 2: Pod CrashLoopBackOff

QUESTION A Pod is in CrashLoopBackOff. What do you check?

EXPECTED ANSWER Check current and previous logs, describe output, environment variables, command, config, permissions, dependencies, and probes.

Scenario 3: Pod Running but Not Receiving Traffic

QUESTION Pod is Running but Service does not send traffic.

EXPECTED ANSWER Check readiness probe, Service selector, Pod labels, endpoints, container port, and application listening address.

Scenario 4: Debug Distroless Container

QUESTION A distroless container has no shell. How do you debug it?

EXPECTED ANSWER Use ephemeral containers with `kubectl debug` or inspect logs/events/metrics externally.

43. Hands-On Labs

Lab 1: Create a Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

YAML

Apply:

```
kubectl apply -f pod.yaml
kubectl get pods -o wide
```

SHELL

Lab 2: View Logs

```
kubectl logs nginx-pod
```

SHELL

Lab 3: Exec Into Pod

```
kubectl exec -it nginx-pod -- sh
```

SHELL

Lab 4: Describe Pod

```
kubectl describe pod nginx-pod
```

SHELL

Lab 5: Delete Pod

```
kubectl delete pod nginx-pod
```

SHELL

Because this is a standalone Pod, it will not be recreated automatically.

44. Chapter Revision Notes

- A Pod is the smallest deployable unit in Kubernetes.
- A Pod contains one or more containers.
- Kubernetes schedules Pods, not individual containers.
- Containers in a Pod share network and volumes.
- Pod phases include Pending, Running, Succeeded, Failed, and Unknown.
- Container states include Waiting, Running, and Terminated.
- Init containers run before app containers.
- Sidecars support the main container.
- Ephemeral containers help debugging.
- Pod IPs are temporary.
- Services should be used for stable networking.
- Use Deployments, StatefulSets, Jobs, or DaemonSets instead of standalone Pods in production.

45. Key Interview Statement

Memorize this:

KEY STATEMENT A Pod is the smallest deployable unit in Kubernetes. It wraps one or more containers that share the same network namespace, Pod IP, localhost communication, and volumes. Kubernetes schedules Pods onto nodes, and controllers like Deployments or StatefulSets manage Pods for production workloads.

End of Volume 02 — Chapter 03

Next chapter:

Chapter 04 — ReplicaSets, Deployments, Rolling Updates, and Rollbacks

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

ReplicaSets, Deployments, Rolling Updates & Rollbacks

Chapter 04 — running stateless applications
reliably with self-healing, rollouts, and rollbacks.

👑 Chapter 04

ReplicaSet · Deployment · Rollout · Rollback

Chapter Purpose

In the previous chapter, you learned about Pods.

Pods are the smallest deployable unit in Kubernetes, but in production you usually do not create standalone Pods.

Why?

Because standalone Pods are not self-healing.

If a standalone Pod is deleted, Kubernetes does not automatically recreate it.

In production, applications should be managed by controllers such as Deployments, StatefulSets, DaemonSets, Jobs, or CronJobs.

This chapter focuses on ReplicaSets and Deployments, which are the most common way to run stateless applications in Kubernetes.

Learning Objectives

By the end of this chapter, you should be able to explain:

- What a ReplicaSet is.
- Why ReplicaSets exist.
- What a Deployment is.
- Why Deployments are used in production.
- How Deployments manage ReplicaSets.
- How ReplicaSets manage Pods.
- Rolling updates.
- Rollbacks.
- Deployment strategies.
- Deployment YAML structure.
- Labels and selectors.
- Scaling Deployments.
- Troubleshooting Deployments.
- Production best practices.
- Interview answers.

1. Why Standalone Pods Are Not Enough

A standalone Pod can run a container.

Example:

YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: nginx:1.27
```

This works for learning.

But if this Pod is deleted:

SHELL

```
kubectl delete pod nginx-pod
```

Kubernetes will not recreate it automatically.

That is a problem in production.

Production applications need:

- Self-healing
- Replicas
- Rolling updates
- Rollbacks
- Scaling
- Version management

Deployments provide these features.

2. What Is a ReplicaSet?

A ReplicaSet is a Kubernetes controller that ensures a specified number of Pod replicas are running.

SIMPLE DEFINITION

A ReplicaSet maintains the desired number of identical Pods.

Example desired state:

Run 3 nginx Pods.

If one Pod dies, ReplicaSet creates another.

If there are too many Pods, ReplicaSet deletes extra Pods.

3. Why ReplicaSet Exists

ReplicaSet solves the problem of keeping a stable number of Pods running.

Without ReplicaSet:

Pod deleted → application replica lost

With ReplicaSet:

Pod deleted → ReplicaSet creates replacement

ReplicaSet continuously compares:

Desired replicas vs actual replicas

Then it reconciles the difference.

4. ReplicaSet Analogy

Think of a restaurant manager.

The restaurant needs 3 chefs working.

Desired state:

3 chefs should be working.

Actual state:

Only 2 chefs are working.

Manager action:

Call 1 more chef.

If 4 chefs are working:

Send 1 chef home.

ReplicaSet works similarly for Pods.

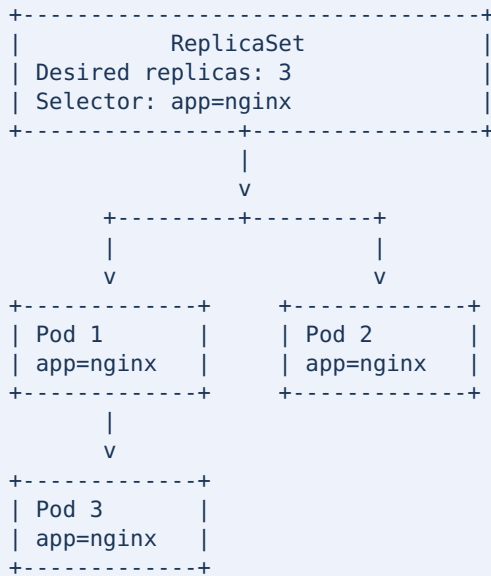
5. ReplicaSet Reconciliation

```
Desired replicas: 3
Actual replicas: 2
    |
    v
ReplicaSet creates 1 Pod
    |
    v
Actual replicas: 3
```

If actual state changes, ReplicaSet reacts.

This is Kubernetes reconciliation in action.

6. ReplicaSet Architecture



ReplicaSet uses labels and selectors to find which Pods it owns.

7. ReplicaSet YAML Example

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

YAML

Apply:

```
kubectl apply -f replicaset.yaml
```

SHELL

Check:

```
kubectl get rs
kubectl get pods
```

SHELL

8. ReplicaSet YAML Explanation

```
apiVersion: apps/v1
```

YAML

Uses the apps API group.

```
kind: ReplicaSet
```

YAML

Creates a ReplicaSet object.

```
metadata:
  name: nginx-rs
```

YAML

Names the ReplicaSet.

```
replicas: 3
```

YAML

Requests 3 Pods.

```
selector:
  matchLabels:
    app: nginx
```

YAML

ReplicaSet manages Pods with label app=nginx.

```
template:
```

YAML

Defines the Pod template used to create Pods.

```
labels:
  app: nginx
```

YAML

Labels applied to created Pods.

IMPORTANT

The selector must match the Pod template labels.

9. ReplicaSet Selector Importance

This is very important.

ReplicaSet uses selectors to decide which Pods it owns.

Selector:

```
selector:
  matchLabels:
    app: nginx
```

YAML

Pod template label:

```
labels:
  app: nginx
```

YAML

These must match.

If they do not match, the ReplicaSet cannot properly manage Pods.

10. What Happens If a Pod Is Deleted?

Run:

```
kubectl delete pod <pod-name>
```

SHELL

ReplicaSet notices:

Desired replicas: 3

Actual replicas: 2

Then it creates a replacement Pod.

Check:

```
kubectl get pods
```

SHELL

You will see a new Pod.

11. What Is a Deployment?

A Deployment is a higher-level Kubernetes controller used to manage stateless applications.

A Deployment manages ReplicaSets.

ReplicaSets manage Pods.

SIMPLE DEFINITION

A Deployment manages application releases by creating and updating ReplicaSets and Pods.

Deployment hierarchy:

```
Deployment
  |
  v
ReplicaSet
  |
  v
Pods
  |
  v
Containers
```

12. Why Deployment Exists

ReplicaSets maintain Pod count, but Deployments add production features:

- Rolling updates
- Rollbacks
- Revision history
- Declarative updates
- Scaling
- Self-healing through ReplicaSets
- Safer application releases

In production, you usually create Deployments, not ReplicaSets directly.

13. Deployment Analogy

Think of a Deployment as a release manager.

The release manager decides:

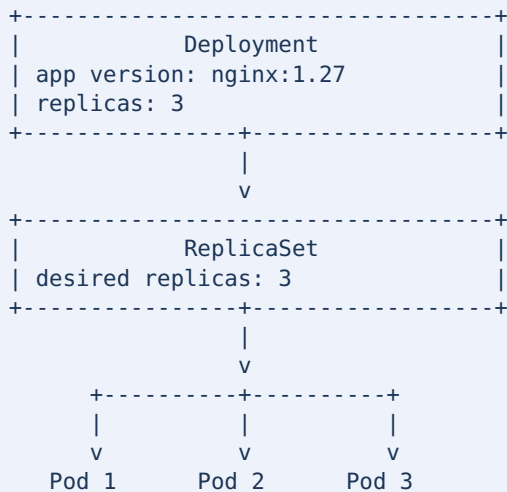
- Which app version should run
- How many copies should run
- How to replace old version with new version

- How to roll back if new version fails
- How to track revision history

ReplicaSet is like the team supervisor maintaining the number of workers.

Pod is the actual worker.

14. Deployment Architecture



When image changes, Deployment creates a new ReplicaSet.

15. Deployment YAML Example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  strategy:
    type: RollingUpdate
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

YAML

Apply:

```
kubectl apply -f deployment.yaml
```

SHELL

Check:

```
kubectl get deployments
kubectl get rs
kubectl get pods
```

SHELL

16. Deployment YAML Explanation

```
apiVersion: apps/v1
```

YAML

Uses Kubernetes apps API.

```
kind: Deployment
```

YAML

Creates a Deployment.

```
metadata:  
  name: nginx-deployment
```

YAML

Names the Deployment.

```
replicas: 3
```

YAML

Requests 3 Pod replicas.

```
selector:  
  matchLabels:  
    app: nginx
```

YAML

Deployment uses this selector to find managed Pods.

```
strategy:  
  type: RollingUpdate
```

YAML

Uses rolling update strategy.

```
template:
```

YAML

Defines Pod template.

```
image: nginx:1.27
```

YAML

Container image version.

17. Deployment, ReplicaSet, Pod Relationship

After applying Deployment:

```
kubectl get deployment  
kubectl get rs  
kubectl get pods
```

SHELL

You may see:

```
deployment.apps/nginx-deployment
replicaset.apps/nginx-deployment-abc123
pod/nginx-deployment-abc123-x1
pod/nginx-deployment-abc123-x2
pod/nginx-deployment-abc123-x3
```

Kubernetes automatically creates the ReplicaSet and Pods.

18. Scaling a Deployment

Scale using command:

```
kubectl scale deployment nginx-deployment --replicas=5
```

SHELL

Check:

```
kubectl get pods
```

SHELL

Or update YAML:

```
spec:
  replicas: 5
```

YAML

Apply:

```
kubectl apply -f deployment.yaml
```

SHELL

PRODUCTION BEST PRACTICE

Prefer updating YAML/GitOps source instead of manual scaling, unless it is temporary.

19. Self-Healing

If a Pod crashes or is deleted, Deployment indirectly restores it through ReplicaSet.

Flow:

```
Pod deleted
|
v
ReplicaSet sees actual replicas decreased
|
v
ReplicaSet creates replacement Pod
|
v
Deployment desired state restored
```

This is one of Kubernetes' most important features.

20. Rolling Updates

A rolling update replaces old Pods with new Pods gradually.

Example:

Old version:

nginx:1.26

New version:

nginx:1.27

Instead of deleting all old Pods at once, Kubernetes gradually creates new Pods and removes old ones.

This helps avoid downtime.

21. Rolling Update Diagram

```
Initial:
[old] [old] [old]

Step 1:
[old] [old] [old] [new]

Step 2:
[old] [old] [new]

Step 3:
[old] [new] [new]

Step 4:
[new] [new] [new]
```

The exact behavior depends on rollout settings.

22. Updating an Image

Command:

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.28
```

SHELL

Check rollout:

```
kubectl rollout status deployment/nginx-deployment
```

SHELL

View history:

```
kubectl rollout history deployment/nginx-deployment
```

SHELL

23. Updating Through YAML

Better production approach:

Change:

```
image: nginx:1.27
```

YAML

to:

```
image: nginx:1.28
```

YAML

Apply:

```
kubectl apply -f deployment.yaml
```

SHELL

This keeps desired state in code.

24. maxSurge and maxUnavailable

Rolling update behavior is controlled by:

YAML

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 1
```

maxSurge

How many extra Pods can be created above desired replicas.

maxUnavailable

How many Pods can be unavailable during update.

Example with replicas 3:

YAML

```
maxSurge: 1
maxUnavailable: 1
```

Kubernetes may allow up to 4 Pods temporarily and at least 2 available Pods.

25. Safer Production Rolling Update

For high availability:

YAML

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 25%
    maxUnavailable: 0
```

This ensures no existing available Pod is intentionally taken down before a new one is ready.

Useful for critical services.

26. Rollout Status

Check rollout:

SHELL

```
kubectl rollout status deployment/nginx-deployment
```

Example:

```
deployment "nginx-deployment" successfully rolled out
```

If rollout hangs, check:

```
kubectl describe deployment nginx-deployment
kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod>
```

SHELL

27. Rollout History

Command:

```
kubectl rollout history deployment/nginx-deployment
```

SHELL

Shows previous Deployment revisions.

To add useful history, use annotations:

```
kubectl annotate deployment nginx-deployment kubernetes.io/change-cause="Upgrade nginx
to 1.28"
```

SHELL

28. Rollback

If a rollout fails, rollback:

```
kubectl rollout undo deployment/nginx-deployment
```

SHELL

Rollback to specific revision:

```
kubectl rollout undo deployment/nginx-deployment --to-revision=2
```

SHELL

Check status:

```
kubectl rollout status deployment/nginx-deployment
```

SHELL

29. Revision History Limit

Deployment keeps old ReplicaSets for rollback.

Control with:

```
revisionHistoryLimit: 5
```

YAML

This keeps the last 5 revisions.

Without limit, too many old ReplicaSets may remain.

30. Deployment Strategies

Common strategies:

Strategy	Meaning
Recreate	Delete old Pods, then create new Pods
RollingUpdate	Gradually replace Pods
Blue-Green	Run two environments, switch traffic
Canary	Send small traffic to new version first

Kubernetes Deployment natively supports:

- Recreate
- RollingUpdate

Blue-green and canary usually require Services, Ingress, service mesh, or progressive delivery tools.

31. Recreate Strategy

Example:

```
strategy:  
  type: Recreate
```

YAML

Behavior:

Delete old Pods

Create new Pods

Use cases:

- Apps that cannot run two versions at once
- Single-writer workloads
- Development/testing

Downside:

- Downtime likely

32. RollingUpdate Strategy

Example:

```
strategy:  
  type: RollingUpdate
```

YAML

Behavior:

Create new Pods gradually

Delete old Pods gradually

Maintain availability

This is default for Deployments.

33. Blue-Green Deployment

Blue-green means:

- Blue = current production version
- Green = new version
- Test green
- Switch traffic from blue to green

Concept:

Service → Blue Pods

After switch:

Service → Green Pods

Kubernetes can do this using labels and Service selectors.

34. Canary Deployment

Canary means releasing a new version to a small percentage of traffic first.

Example:

90% traffic → stable version

10% traffic → new version

Useful for risk reduction.

Often implemented with:

- Ingress controller
- Service mesh
- Argo Rollouts
- Flagger
- Progressive delivery tools

35. Deployment Conditions

Check Deployment:

```
kubectl describe deployment nginx-deployment
```

SHELL

You may see conditions:

Condition	Meaning
Available	Minimum availability met
Progressing	Rollout progressing
ReplicaFailure	Replica creation failed

Conditions help troubleshoot rollout problems.

36. Common Deployment Problems

Problem	Possible Cause
Rollout stuck	New Pods not becoming ready
ImagePullBackOff	Bad image/tag/registry
CrashLoopBackOff	App crashes
Pending Pods	Scheduling issue
Service has no endpoints	Label mismatch/readiness failure
Too many old ReplicaSets	revisionHistoryLimit missing
No traffic to new Pods	Selector mismatch

37. Troubleshooting Rollout

Commands:

```
kubectl rollout status deployment/<name>
kubectl describe deployment <name>
kubectl get rs
kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod>
kubectl get events
```

SHELL

If new Pods fail readiness probe, rollout may pause or hang.

38. Troubleshooting Label Selector Issues

Deployment selector:

```
selector:
  matchLabels:
    app: nginx
```

YAML

Pod template labels:

YAML

```
template:
  metadata:
    labels:
      app: nginx
```

These must match.

Service selector must also match Pod labels:

YAML

```
selector:
  app: nginx
```

If labels do not match, Service may have no endpoints.

Check endpoints:

SHELL

```
kubectl get endpoints
kubectl get endpointslices
```

39. Production Deployment YAML Example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  labels:
    app: api
spec:
  replicas: 3
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 0
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
      containers:
        - name: api
          image: example.com/api:1.4.2
          ports:
            - containerPort: 8080
          env:
            - name: APP_ENV
              value: production
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
          readinessProbe:
            httpGet:
              path: /ready
              port: 8080
            initialDelaySeconds: 10
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 20
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
```

40. Production Best Practices

- Use Deployments for stateless applications.
- Do not create standalone Pods for production apps.
- Use specific image tags or digests.
- Avoid latest.
- Define resource requests and limits.
- Configure readiness and liveness probes.
- Use rolling updates.
- Set maxUnavailable: 0 for critical services if capacity allows.
- Set revisionHistoryLimit.
- Keep manifests in Git.
- Use GitOps for deployment.
- Use labels consistently.
- Monitor rollout status.
- Validate Service selectors.
- Use PodDisruptionBudgets for availability.
- Use HorizontalPodAutoscaler when appropriate.
- Use progressive delivery for high-risk releases.

41. Common Mistakes

Mistake	Why Bad
Creating standalone Pods	No controller self-healing
Selector mismatch	Pods not managed or traffic broken
Using latest image tag	Unpredictable rollouts
No readiness probe	Traffic may hit unready Pods
No resource requests	Scheduling problems
Manual kubectl changes only	Drift from Git
No rollback plan	Longer recovery
Too aggressive rollout	Availability risk
Ignoring rollout status	Failed deployments unnoticed

42. Interview Questions and Answers

Beginner

1. What is a ReplicaSet?

A ReplicaSet ensures a specified number of Pod replicas are running.

2. What is a Deployment?

A Deployment manages ReplicaSets and Pods for stateless applications and supports rolling updates and rollbacks.

3. Why should we use Deployment instead of standalone Pod?

Deployment provides self-healing, scaling, rolling updates, and rollback support.

4. What is rolling update?

Rolling update gradually replaces old Pods with new Pods.

5. What is rollback?

Rollback returns a Deployment to a previous revision.

Intermediate

1. Relationship between Deployment, ReplicaSet, and Pod?

Deployment manages ReplicaSets. ReplicaSets manage Pods. Pods run containers.

2. What is maxSurge?

maxSurge controls how many extra Pods can be created during rolling update.

3. What is maxUnavailable?

maxUnavailable controls how many Pods can be unavailable during rolling update.

4. How do you scale a Deployment?

Use `kubectl scale deployment <name> --replicas=N` or update `spec.replicas` in YAML.

5. How do you check rollout status?

Use `kubectl rollout status deployment/<name>`.

Advanced

1. Why can a rollout get stuck?

New Pods may fail readiness probes, image pull, scheduling, or crash repeatedly.

2. How does Kubernetes know which Pods belong to a Deployment?

Through labels and selectors.

3. What happens during rollback?

Deployment scales up the previous ReplicaSet and scales down the current failed ReplicaSet.

4. Why is readiness probe important during rolling update?

It prevents traffic from being sent to Pods that are not ready.

5. Why use progressive delivery tools?

For safer canary/blue-green releases with traffic control, metrics analysis, and automated rollback.

43. Scenario-Based Interview Questions

Scenario 1: Pod Deleted

QUESTION A Pod managed by Deployment is deleted manually. What happens?

EXPECTED ANSWER ReplicaSet detects fewer actual replicas than desired and creates a replacement Pod.

Scenario 2: Rollout Stuck

QUESTION Deployment rollout is stuck. What do you check?

EXPECTED ANSWER Check rollout status, describe Deployment, ReplicaSet, Pods, events, logs, image pull errors, readiness probe failures, and scheduling problems.

Scenario 3: Service Has No Endpoints

QUESTION Deployment Pods are running, but Service has no endpoints.

EXPECTED ANSWER Check Service selector and Pod labels. Also check readiness probes because unready Pods may not appear as ready endpoints.

Scenario 4: Bad Version Deployed

QUESTION New version causes errors. How do you rollback?

EXPECTED ANSWER Use `kubectl rollout undo deployment/<name>` or apply previous manifest from Git. Then monitor rollout status.

44. Hands-On Labs

Lab 1: Create Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

YAML

Apply:

```
kubectl apply -f deployment.yaml
kubectl get deployments
kubectl get rs
kubectl get pods
```

SHELL

Lab 2: Delete a Pod

```
kubectl get pods
kubectl delete pod <pod-name>
kubectl get pods
```

SHELL

Observe replacement Pod.

Lab 3: Scale Deployment

```
kubectl scale deployment nginx-deployment --replicas=5
kubectl get pods
```

SHELL

Lab 4: Rolling Update

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.28
kubectl rollout status deployment/nginx-deployment
```

SHELL

Lab 5: Rollback

```
kubectl rollout history deployment/nginx-deployment
kubectl rollout undo deployment/nginx-deployment
kubectl rollout status deployment/nginx-deployment
```

SHELL

45. Chapter Revision Notes

- ReplicaSet maintains a desired number of Pod replicas.
- Deployment manages ReplicaSets.
- ReplicaSet manages Pods.
- Pods run containers.
- Deployments are preferred for stateless production applications.
- Deployments support scaling, rolling updates, and rollbacks.
- Labels and selectors are critical.
- Rolling updates gradually replace old Pods with new Pods.
- Rollbacks restore previous revisions.
- Readiness probes are critical during rollouts.
- Avoid standalone Pods in production.

46. Key Interview Statement

Memorize this:

KEY STATEMENT

A Deployment is the standard Kubernetes controller for stateless applications. It manages ReplicaSets, and ReplicaSets maintain the desired number of Pods. Deployments provide self-healing, scaling, rolling updates, rollback support, and revision history, making them suitable for production workloads.

End of Volume 02 — Chapter 04

Next chapter:

Chapter 05 — Services, Service Discovery, ClusterIP, NodePort, LoadBalancer, and Headless Services

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

Services & Service Discovery

Chapter 05 — ClusterIP, NodePort,
LoadBalancer, and Headless Services for stable
networking.

👑 Chapter 05

ClusterIP · NodePort · LoadBalancer · Headless

Chapter Purpose

Pods are temporary.

A Pod can be created, deleted, restarted, or moved to another node. When that happens, its IP address can change.

This creates a problem:

If Pod IPs change, how can one application reliably connect to another application?

Kubernetes solves this problem using Services.

A Service provides a stable network identity for a group of Pods.

This chapter explains Kubernetes Services from beginner level to production troubleshooting level.

Learning Objectives

By the end of this chapter, you should be able to explain:

- Why Services exist.
- What Service discovery means.
- How Services select Pods.
- ClusterIP Service.
- NodePort Service.
- LoadBalancer Service.
- ExternalName Service.
- Headless Service.
- Endpoints and EndpointSlices.
- kube-proxy role.
- DNS-based discovery.
- Service troubleshooting.
- Production best practices.
- Interview answers.

1. Why Services Exist

Pods are not permanent.

A Pod may disappear because:

- Deployment rollout happened.
- Node failed.

- Pod crashed.
- ReplicaSet replaced it.
- Autoscaler changed replica count.
- Manual deletion happened.
- Scheduler moved workload.

When a Pod is recreated, it usually gets a new IP address.

If another application connects directly to Pod IP, it can break.

Bad:

```
frontend → 10.244.1.25
```

If that Pod dies, the IP may disappear.

Better:

```
frontend → backend-service
```

The Service stays stable even when backend Pods change.

2. What Is a Kubernetes Service?

A Service is a Kubernetes object that provides stable networking for a group of Pods.

SIMPLE DEFINITION A Service gives a stable IP address and DNS name to reach changing Pods.

A Service selects Pods using labels.

Example:

```
Service selector: app=backend
                  |
                  v
Pods with label app=backend
```

3. Real-World Analogy

Think of a company call center.

Employees may change seats, shifts, or phone extensions.

Customers do not call each employee directly.

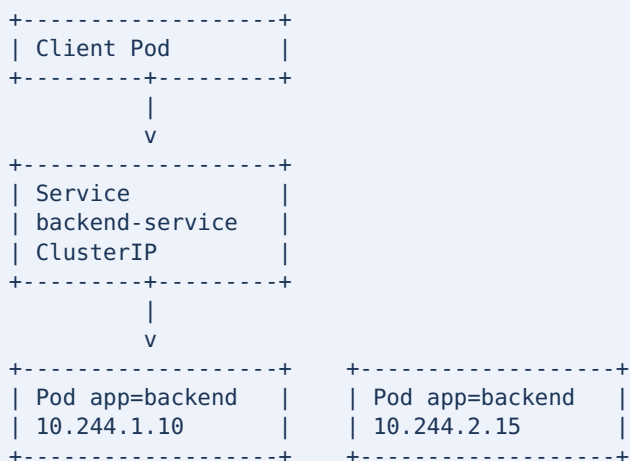
Customers call one stable company number.

The call center routes the call to an available employee.

Kubernetes Concept	Call Center Analogy
Pod	Employee
Pod IP	Employee extension
Service	Company phone number
Selector	Department name
Endpoint	Available employee
kube-proxy	Call routing system

A Service is the stable phone number for changing Pods.

4. Service Architecture



The client talks to the Service.

The Service routes traffic to matching Pods.

5. Labels and Selectors

Services use selectors to find Pods.

Pod labels:

YAML

```
metadata:
  labels:
    app: backend
```

Service selector:

YAML

```
selector:
  app: backend
```

The Service sends traffic to Pods matching the selector.

If labels do not match, the Service has no endpoints.

6. Basic Service YAML

Deployment Pods:

YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: nginx:1.27
          ports:
            - containerPort: 80
```

Service:

```
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  selector:
    app: backend
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

YAML

7. Service YAML Explanation

```
apiVersion: v1
```

YAML

Service belongs to core Kubernetes API.

```
kind: Service
```

YAML

Creates a Service object.

```
metadata:
  name: backend-service
```

YAML

This name becomes part of DNS.

```
selector:
  app: backend
```

YAML

Selects Pods with label app=backend.

```
port: 80
```

YAML

Service port.

```
targetPort: 80
```

YAML

Pod/container port where traffic is sent.

```
type: ClusterIP
```

YAML

Creates an internal cluster Service.

8. port vs targetPort vs nodePort

This is a common interview topic.

Field	Meaning
port	Port exposed by the Service
targetPort	Port on the Pod/container
nodePort	Port exposed on each node for NodePort Service

Example:

```
ports:
- port: 80
  targetPort: 8080
  nodePort: 30080
```

YAML

Meaning:

Client → Service port 80 → Pod port 8080
External client → NodeIP:30080 → Service → Pod port 8080

9. Service Discovery

Service discovery means applications can find each other without hardcoding IP addresses.

Kubernetes provides Service discovery using:

- DNS
- Environment variables
- Service names
- ClusterIP

Most common method:

DNS

Example:

```
backend-service
backend-service.default
backend-service.default.svc
backend-service.default.svc.cluster.local
```

10. CoreDNS

CoreDNS is the DNS server used in most Kubernetes clusters.

It resolves Kubernetes Service names to Service IPs.

Example:

```
nslookup backend-service
```

SHELL

Inside a Pod, this resolves the Service name.

CoreDNS is critical for cluster service discovery.

If CoreDNS fails, many applications may not communicate correctly.

11. Service Types

Kubernetes Service types:

Type	Purpose
ClusterIP	Internal cluster access
NodePort	Expose Service on each node port
LoadBalancer	Create external load balancer
ExternalName	DNS alias to external name
Headless	No ClusterIP, direct Pod DNS

12. ClusterIP Service

ClusterIP is the default Service type.

It exposes the Service only inside the cluster.

Example:

```

apiVersion: v1
kind: Service
metadata:
  name: backend
spec:
  type: ClusterIP
  selector:
    app: backend
  ports:
    - port: 80
      targetPort: 8080

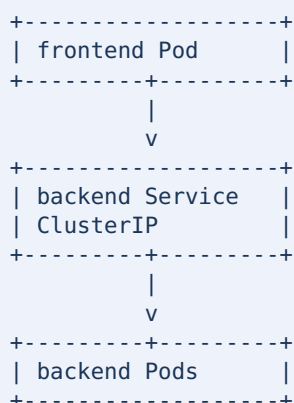
```

Use ClusterIP for internal communication between services.

Example:

```
frontend Pod → backend ClusterIP Service → backend Pods
```

13. ClusterIP Diagram



ClusterIP is not directly accessible from outside the cluster.

14. NodePort Service

NodePort exposes a Service on a port on every worker node.

Example:

```
apiVersion: v1
kind: Service
metadata:
  name: web-nodeport
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

Access:

```
http://NodeIP:30080
```

15. NodePort Diagram

```
External Client
  |
  v
NodeIP:30080
  |
  v
NodePort Service
  |
  v
Pod
```

NodePort is useful for:

- Testing
- Simple external access
- Bare-metal environments
- Load balancer integration

In cloud production, LoadBalancer or Ingress is usually preferred.

16. NodePort Range

Default NodePort range is usually:

```
30000-32767
```

If you do not specify nodePort, Kubernetes assigns one automatically.

Example:

```
ports:
  - port: 80
    targetPort: 80
```

YAML

Kubernetes chooses a NodePort.

17. LoadBalancer Service

LoadBalancer Service requests an external load balancer from the cloud provider.

Example:

```
apiVersion: v1
kind: Service
metadata:
  name: web-lb
spec:
  type: LoadBalancer
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
```

YAML

In AWS, this may create an Elastic Load Balancer depending on cloud integration.

18. LoadBalancer Diagram

```
graph TD
  Internet --> CLB[Cloud Load Balancer]
  CLB --> KS[Kubernetes Service]
  KS --> Pods
```

Use LoadBalancer for external TCP/UDP access.

For HTTP/HTTPS routing, Ingress or Gateway API is often better.

19. LoadBalancer in Cloud

In managed Kubernetes:

Platform	LoadBalancer Creates
AWS EKS	AWS ELB/NLB depending config/controllers
GKE	Google Cloud Load Balancer
AKS	Azure Load Balancer

Cloud provider integration is required.

In local clusters like kind/minikube, LoadBalancer may stay pending unless special support exists.

20. ExternalName Service

ExternalName maps a Service name to an external DNS name.

Example:

```
apiVersion: v1
kind: Service
metadata:
  name: external-db
spec:
  type: ExternalName
  externalName: database.example.com
```

YAML

Pods can connect to:

external-db

which resolves to:

database.example.com

Use for DNS aliasing external services.

21. Headless Service

A Headless Service has no ClusterIP.

Example:


```

apiVersion: v1
kind: Service
metadata:
  name: database
spec:
  clusterIP: None
  selector:
    app: database
  ports:
    - port: 5432
      targetPort: 5432

```

Instead of returning one Service IP, DNS returns Pod IPs directly.

Useful for:

- StatefulSets
- Databases
- Direct Pod discovery
- Stable network identities

22. Headless Service Diagram

```

DNS query: database
  |
  v
Returns Pod IPs:
10.244.1.10
10.244.2.15
10.244.3.20

```

Stateful applications often need direct identity for each Pod.

Headless Services are covered again in StatefulSet chapters.

23. Endpoints and EndpointSlices

A Service does not directly send traffic to Pods by magic.

Kubernetes creates endpoint records.

Older object:

Endpoints

Modern scalable object:

EndpointSlice

EndpointSlices store the list of backend Pod IPs and ports for a Service.

24. Check Service Endpoints

Commands:

```
kubectl get service
kubectl get endpoints
kubectl get endpointslices
```

SHELL

If Service has no endpoints, common causes:

- Selector does not match Pod labels.
- Pods are not Ready.
- Pods do not exist.
- Wrong namespace.
- EndpointSlice controller issue.

25. kube-proxy and Services

kube-proxy runs on nodes and helps implement Service routing.

It watches Services and EndpointSlices.

Then it programs networking rules using:

- iptables
- IPVS
- nftables depending cluster/platform

Concept:

```
Service IP:Port → backend Pod IP:Port
```

26. Service Traffic Flow

```
Client Pod
  |
  v
DNS lookup backend-service
  |
  v
Service ClusterIP
  |
  v
kube-proxy rules
  |
  v
Backend Pod IP
```

The client does not need to know Pod IPs directly.

27. Service Without Selector

A Service can exist without selector.

Example use cases:

- Manually managed endpoints
- External database
- Migration scenarios
- Custom endpoint routing

Most normal Services use selectors.

28. Named Ports

Pods can name container ports.

Example:

```
ports:
- name: http
  containerPort: 8080
```

YAML

Service can use targetPort by name:

```
ports:
- port: 80
  targetPort: http
```

YAML

This improves readability and allows container port changes without changing Service targetPort name.

29. Session Affinity

Service can route traffic from same client IP to same backend Pod.

Example:

```
spec:
  sessionAffinity: ClientIP
```

YAML

Default:

None

Use carefully.

Most stateless services should not require session affinity.

30. Internal Traffic Policy

Service can control internal traffic behavior.

Example:

```
spec:
  internalTrafficPolicy: Local
```

YAML

This tries to route traffic only to node-local endpoints.

Useful for some performance or locality-sensitive workloads.

Default is usually cluster-wide routing.

31. External Traffic Policy

For NodePort/LoadBalancer Services:

```
spec:
  externalTrafficPolicy: Local
```

YAML

This preserves client source IP and routes only to node-local endpoints.

Tradeoff:

- Preserves source IP
- May reduce load spreading if endpoints are not on every node

Common with ingress controllers and load balancers.

32. Service Troubleshooting

Problem: Service Has No Endpoints

Check Service:

```
kubectl describe service <service>
```

SHELL

Check Pods labels:

```
kubectl get pods --show-labels
```

SHELL

Check endpoints:

```
kubectl get endpoints <service>  
kubectl get endpointslices
```

SHELL

Fix selector/labels.

Problem: DNS Not Resolving Service

Check CoreDNS:

```
kubectl get pods -n kube-system  
kubectl logs -n kube-system deployment/coredns
```

SHELL

Test from Pod:

```
kubectl run dns-test --image=busybox:1.36 -it --rm --restart=Never -- nslookup  
kubernetes.default
```

SHELL

Common causes:

- CoreDNS down
- NetworkPolicy blocking DNS
- Pod DNS config issue

- kube-system problem

Problem: Service Resolves but Connection Fails

Check:

```
kubectl get endpoints <service>
kubectl describe pod <backend-pod>
kubectl logs <backend-pod>
```

SHELL

Possible causes:

- Wrong targetPort
- App not listening
- Pod not ready
- NetworkPolicy blocking
- Container port mismatch
- Application error

Problem: LoadBalancer Pending

Common causes:

- Local cluster without load balancer support
- Cloud provider integration missing
- IAM/permission issue
- Quota issue
- Unsupported annotations
- Subnet/security group tagging issue in cloud

Check:

```
kubectl describe service <service>
kubectl get events
```

SHELL

33. Production Best Practices

- Use ClusterIP for internal service-to-service communication.
- Use LoadBalancer or Ingress/Gateway for external access.
- Avoid using NodePort directly in production unless needed.
- Use meaningful Service names.
- Keep labels and selectors consistent.
- Use named ports for clarity.
- Do not hardcode Pod IPs.

- Monitor CoreDNS.
- Check EndpointSlices when troubleshooting.
- Use readiness probes so Services only send traffic to ready Pods.
- Avoid exposing databases externally.
- Use NetworkPolicies for traffic control.
- Use externalTrafficPolicy carefully.
- Prefer Ingress/Gateway for HTTP routing.

34. Common Mistakes

Mistake	Why Bad
Using Pod IP directly	Pod IP changes
Selector mismatch	Service has no endpoints
Wrong targetPort	Traffic reaches wrong port
No readiness probe	Traffic may go to unready Pods
Exposing DB with LoadBalancer	Security risk
Expecting ClusterIP externally	ClusterIP is internal only
NodePort in production by default	Harder to manage securely
Ignoring CoreDNS	Service discovery fails

35. Interview Questions and Answers

Beginner

1. What is a Kubernetes Service?

A Service provides stable networking and DNS for a group of Pods.

2. Why do we need Services?

Pods are temporary and their IPs can change. Services provide stable access.

3. What is ClusterIP?

ClusterIP is the default Service type for internal cluster access.

4. What is NodePort?

NodePort exposes a Service on a port on every node.

5. What is LoadBalancer?

LoadBalancer creates or integrates with an external load balancer in cloud environments.

Intermediate

1. How does a Service find Pods?

Using selectors that match Pod labels.

2. What happens if Service selector does not match any Pod?

The Service will have no endpoints and traffic will not reach Pods.

3. Difference between port and targetPort?

port is the Service port. targetPort is the backend Pod/container port.

4. What is a headless Service?

A Service with clusterIP: None that returns Pod IPs directly instead of one ClusterIP.

5. What is CoreDNS?

CoreDNS provides DNS-based service discovery inside Kubernetes.

Advanced

1. What are EndpointSlices?

EndpointSlices store backend endpoints for Services in a scalable way.

2. What does kube-proxy do?

kube-proxy watches Services and EndpointSlices and configures node networking rules to route traffic to backend Pods.

3. Why use externalTrafficPolicy Local?

To preserve client source IP and route only to local node endpoints.

4. Why might LoadBalancer stay pending?

Cloud integration may be missing, IAM permissions may be wrong, quotas may be exceeded, or local cluster may not support load balancers.

5. How does readiness affect Services?

Only ready Pods should receive Service traffic. If readiness fails, Pod may be removed from ready endpoints.

36. Scenario-Based Interview Questions

Scenario 1: Service Has No Endpoints

QUESTION Pods are running but Service has no endpoints. What do you check?

EXPECTED ANSWER Check Service selector, Pod labels, namespace, readiness status, and EndpointSlices.

Scenario 2: ClusterIP Not Accessible Externally

QUESTION User tries to access ClusterIP from outside cluster and fails.

EXPECTED ANSWER ClusterIP is internal only. Use LoadBalancer, NodePort, Ingress, Gateway, or port-forward for external access.

Scenario 3: DNS Fails Inside Pod

QUESTION Pod cannot resolve service name.

EXPECTED ANSWER Check CoreDNS Pods, DNS logs, NetworkPolicy, Pod DNS config, and test with nslookup from another Pod.

Scenario 4: Service Resolves but App Connection Fails

QUESTION Service DNS works, but connection times out.

EXPECTED ANSWER Check endpoints, targetPort, app listening address, readiness, NetworkPolicy, and backend Pod logs.

37. Hands-On Labs

Lab 1: Create Deployment and ClusterIP Service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
```

YAML

Apply:

```
kubectl apply -f web.yaml
kubectl get svc
kubectl get endpoints
```

SHELL

Lab 2: Test DNS

```
kubectl run test --image=busybox:1.36 -it --rm --restart=Never -- nslookup web
```

SHELL

Lab 3: Test Service Access

```
kubectl run curl --image=curlimages/curl -it --rm --restart=Never -- curl http://web
```

SHELL

Lab 4: Break Selector

Edit Service selector to wrong label and apply.

Check:

```
kubectl get endpoints web
```

SHELL

Observe no endpoints.

Fix selector.

38. Chapter Revision Notes

- Pods are temporary; Services provide stable access.
- Services use selectors to find Pods.
- ClusterIP is internal-only.
- NodePort exposes a port on each node.
- LoadBalancer creates external cloud load balancer.
- ExternalName creates DNS alias.
- Headless Service returns Pod IPs directly.
- CoreDNS provides Service DNS.
- EndpointSlices store backend Pod IPs/ports.
- kube-proxy helps route Service traffic.
- port is Service port.
- targetPort is Pod port.
- Readiness probes affect Service traffic.
- Selector mismatch is a common cause of no endpoints.

39. Key Interview Statement

Memorize this:

KEY STATEMENT

A Kubernetes Service provides stable networking for dynamic Pods. It selects Pods using labels, exposes a stable DNS name and virtual IP, and routes traffic to ready backend endpoints. ClusterIP is for internal access, NodePort exposes a node port, LoadBalancer integrates with external load balancers, and Headless Services provide direct Pod discovery.

End of Volume 02 — Chapter 05

Next chapter:

Chapter 06 — Ingress, Ingress Controllers, Gateway API, and External Traffic Routing

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

Ingress, Gateway API & External Traffic Routing

Chapter 06 — routing external HTTP/HTTPS
traffic with Ingress, Controllers, and the
Gateway API.

👑 Chapter 06

Ingress · Controllers · TLS · Gateway API

Chapter Purpose

In the previous chapter, you learned about Kubernetes Services.

Services provide stable networking for Pods.

But Services alone are not always enough for real web applications.

Most production applications need external HTTP or HTTPS traffic routing.

Example:

```
https://app.example.com    → frontend service
https://api.example.com    → api service
https://example.com/admin  → admin service
```

Kubernetes solves HTTP/HTTPS routing using Ingress and, more recently, Gateway API.

This chapter explains how external traffic reaches applications inside Kubernetes.

Learning Objectives

By the end of this chapter, you should be able to explain:

- Why Ingress exists.
- What an Ingress object is.
- What an Ingress Controller is.
- Difference between Service and Ingress.
- How HTTP routing works.
- Host-based routing.
- Path-based routing.
- TLS termination.
- Common Ingress Controllers.
- Gateway API basics.
- Ingress vs Gateway API.
- External traffic flow.
- Troubleshooting Ingress.
- Production best practices.
- Interview answers.

1. Why Ingress Exists

A Service exposes Pods inside or outside the cluster.

For external access, one option is Service type LoadBalancer.

Example:

```
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  type: LoadBalancer
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
```

YAML

This works, but there is a problem.

If you have many applications, creating one LoadBalancer per Service can become expensive and hard to manage.

Example:

```
frontend service → Load Balancer 1
api service      → Load Balancer 2
admin service    → Load Balancer 3
reports service  → Load Balancer 4
```

Ingress solves this by allowing one entry point to route traffic to many Services.

2. What Is Ingress?

Ingress is a Kubernetes API object that defines HTTP and HTTPS routing rules.

SIMPLE DEFINITION Ingress defines rules for routing external HTTP/HTTPS traffic to Services inside the cluster.

Ingress can route based on:

- Hostname
- URL path
- TLS configuration

Example:

```
app.example.com → frontend service
api.example.com → api service
example.com/admin → admin service
```

3. What Ingress Is Not

Ingress itself does not route traffic.

Ingress is only a set of rules.

To make Ingress work, you need an Ingress Controller.

IMPORTANT INTERVIEW ANSWER Ingress is the routing rule. Ingress Controller is the actual component that implements the routing.

4. Real-World Analogy

Think of a large office building.

The building has one main reception desk.

Visitors do not directly know every employee's room.

They come to reception and say:

I want to meet the HR team.

I want to meet the Finance team.

I want to meet the Engineering team.

Reception routes them to the right department.

Kubernetes Concept	Office Analogy
External user	Visitor
Ingress Controller	Reception desk
Ingress rules	Routing instructions
Service	Department phone line
Pods	Employees
TLS certificate	Security ID verification

5. Service vs Ingress

Feature	Service	Ingress
Layer	L4 TCP/UDP mostly	L7 HTTP/HTTPS
Purpose	Stable access to Pods	HTTP routing to Services
Routing by hostname	No	Yes
Routing by path	No	Yes
TLS termination	Not typical	Yes
External exposure	NodePort/LoadBalancer	Through Ingress Controller
Object alone enough?	Usually yes	Needs controller

SHORT ANSWER Service provides stable networking to Pods. Ingress provides HTTP/HTTPS routing into Services.

6. Ingress Controller

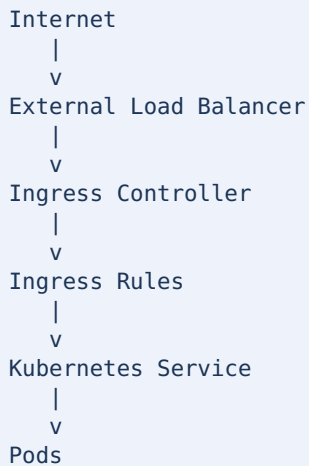
An Ingress Controller is a running component that watches Ingress objects and configures a proxy/load balancer.

Common Ingress Controllers:

- NGINX Ingress Controller
- HAProxy Ingress
- Traefik
- Envoy-based controllers
- AWS Load Balancer Controller
- Contour
- Kong Ingress Controller

The controller does the actual work.

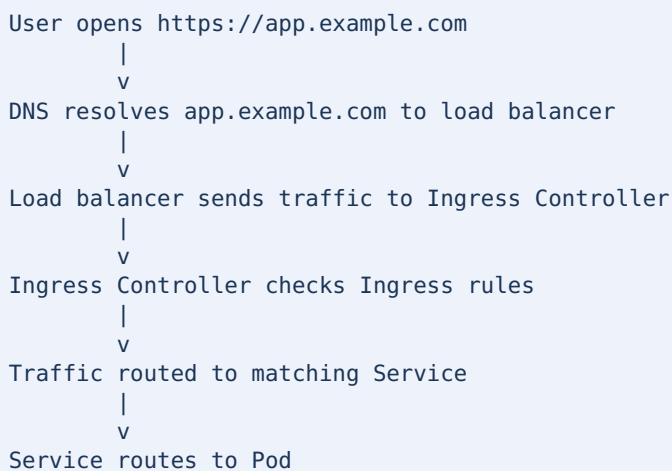
7. Ingress Architecture



Ingress Controller usually runs as Pods inside the cluster.

It is often exposed externally using a LoadBalancer Service.

8. Basic Ingress Flow



9. Basic Ingress Example

Deployment and Service:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: nginx:1.27
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80

```

Ingress:

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend
spec:
  ingressClassName: nginx
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 80

```

10. Ingress YAML Explanation

```
apiVersion: networking.k8s.io/v1
```

YAML

Uses Kubernetes networking API.

```
kind: Ingress
```

YAML

Creates an Ingress object.

```
metadata:  
  name: frontend
```

YAML

Names the Ingress.

```
ingressClassName: nginx
```

YAML

Specifies which Ingress Controller should handle this Ingress.

```
host: app.example.com
```

YAML

Matches requests for this hostname.

```
path: /
```

YAML

Matches URL path.

```
pathType: Prefix
```

YAML

Means all paths starting with / match.

```
backend:  
  service:  
    name: frontend
```

YAML

Routes traffic to the frontend Service.

```
port:  
  number: 80
```

YAML

Uses Service port 80.

11. Host-Based Routing

Host-based routing routes traffic based on domain name.

Example:

```
app.example.com → frontend service
api.example.com → api service
```

Ingress YAML:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: host-routing
spec:
  ingressClassName: nginx
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 80
    - host: api.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 80
```

YAML

12. Path-Based Routing

Path-based routing routes traffic based on URL path.

Example:

```
example.com/      → frontend service
example.com/api    → api service
example.com/admin  → admin service
```

Ingress YAML:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: path-routing
spec:
  ingressClassName: nginx
  rules:
    - host: example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 80
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 80
          - path: /admin
            pathType: Prefix
            backend:
              service:
                name: admin
                port:
                  number: 80
```

YAML

13. pathType

Ingress supports path types.

pathType	Meaning
Prefix	Matches URL prefix
Exact	Matches exact path
ImplementationSpecific	Controller-specific behavior

Example:

```
path: /api
pathType: Prefix
```

YAML

Matches:

```
/api
/api/users
/api/v1/orders
```

Example:

```
path: /api
pathType: Exact
```

YAML

Matches only:

```
/api
```

14. TLS Termination

TLS termination means HTTPS is decrypted at the Ingress Controller.

Flow:

```
User HTTPS request
    |
    v
Ingress Controller terminates TLS
    |
    v
Plain HTTP or HTTPS to backend Service
```

Ingress TLS example:

YAML

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: frontend-tls
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - app.example.com
      secretName: app-example-com-tls
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 80
```

15. TLS Secret

TLS certificate is stored in a Kubernetes Secret.

Create TLS Secret:

SHELL

```
kubectl create secret tls app-example-com-tls \
  --cert=tls.crt \
  --key=tls.key
```

The Secret contains:

- Certificate
- Private key

Ingress references it:

YAML

```
secretName: app-example-com-tls
```

16. cert-manager

In production, TLS certificates are often managed automatically using cert-manager.

cert-manager can request and renew certificates from providers such as Let's Encrypt or internal certificate authorities.

Typical flow:

```
graph TD
    A[Ingress requests TLS] --> B[cert-manager creates Certificate]
    B --> C[ACME/CA validates domain]
    C --> D[Certificate stored as Kubernetes Secret]
    D --> E[Ingress Controller uses Secret]
```

17. IngressClass

IngressClass tells Kubernetes which controller handles an Ingress.

Example:

```
spec:
  ingressClassName: nginx
```

YAML

This is important when a cluster has multiple Ingress Controllers.

Example:

```
nginx controller
aws alb controller
internal controller
```

Without correct class, the wrong controller may ignore or process the Ingress.

18. Ingress Annotations

Many Ingress Controllers use annotations for controller-specific behavior.

Example NGINX-style annotations:

```
metadata:
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/proxy-body-size: "20m"
```

YAML

Example AWS ALB-style annotations:

```
metadata:
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
```

YAML

IMPORTANT

Annotations are controller-specific. Do not assume one controller's annotations work with another controller.

19. Rewrite Rules

Sometimes backend apps expect different paths.

Example external path:

```
/api/users
```

Backend expects:

```
/users
```

Ingress Controller may support rewrite annotations.

Example:

```
metadata:
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
```

YAML

Rewrite behavior depends heavily on controller.

Always test carefully.

20. External DNS

ExternalDNS is a tool that can automatically create DNS records from Kubernetes resources.

Example:

Ingress host:

```
app.example.com
```

ExternalDNS can create DNS record pointing to the load balancer.

Typical flow:

```
graph TD
    A[Ingress created] --> B[ExternalDNS watches Ingress]
    B --> C[DNS record created in Route53/Cloud DNS/etc.]
```

This is common in cloud-native production environments.

21. Ingress Controller Deployment Pattern

Common pattern:

```
graph TD
    A[Ingress Controller Deployment/DaemonSet] --> B[Service type LoadBalancer]
    B --> C[Cloud Load Balancer]
    C --> D[Ingress Controller Pods]
    D --> E[Application Services]
    E --> F[Application Pods]
```

The external load balancer sends traffic to the controller.

The controller routes traffic based on Ingress rules.

22. Ingress vs LoadBalancer Service

Feature	LoadBalancer Service	Ingress
Layer	L4 mostly	L7 HTTP/HTTPS
Routing by path	No	Yes
Routing by host	No	Yes
TLS routing	Limited	Yes
One LB per app	Often yes	One LB can route many apps
Best for	TCP/UDP or simple exposure	Web apps

23. What Is Gateway API?

Gateway API is a newer Kubernetes API family for traffic routing.

It was designed to improve and extend limitations of Ingress.

Gateway API introduces more expressive and role-oriented resources.

Important resources:

- GatewayClass
- Gateway
- HTTPRoute
- TCPRoute
- TLSRoute
- GRPCRoute

SIMPLE DEFINITION

Gateway API is a more advanced and extensible way to manage Kubernetes traffic routing.

24. Why Gateway API Was Created

Ingress is useful but limited.

Common Ingress limitations:

- Too many controller-specific annotations.
- Limited traffic routing model.
- Harder multi-team ownership.
- Limited support for advanced routing.

- Controller behavior varies widely.

Gateway API improves this by defining richer standard resources.

25. Gateway API Roles

Gateway API separates responsibilities.

Role	Resource
Infrastructure/platform team	GatewayClass, Gateway
Application team	HTTPRoute
Controller implementation	Gateway controller

This separation is useful in platform engineering.

Platform team manages shared gateways.

App teams attach routes.

26. Gateway API Architecture

```
graph TD
    Internet --> Gateway
    Gateway --> HTTPRoute
    HTTPRoute --> Service
    Service --> Pods
```

27. GatewayClass

GatewayClass defines a class of gateways controlled by a specific controller.

Example:

YAML

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: example-gateway-class
spec:
  controllerName: example.com/gateway-controller
```

This is similar in concept to IngressClass but more powerful.

28. Gateway

Gateway represents an actual traffic entry point.

Example:

YAML

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: public-gateway
spec:
  gatewayClassName: example-gateway-class
  listeners:
    - name: http
      protocol: HTTP
      port: 80
```

This defines a gateway listening on port 80.

29. HTTPRoute

HTTPRoute defines HTTP routing rules.

Example:

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: frontend-route
spec:
  parentRefs:
    - name: public-gateway
  hostnames:
    - app.example.com
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
      backendRefs:
        - name: frontend
          port: 80
```

This routes traffic from the Gateway to the frontend Service.

30. Ingress vs Gateway API

Feature	Ingress	Gateway API
Maturity	Older, widely used	Newer, growing adoption
Routing model	Basic HTTP routing	More expressive
Extensibility	Annotation-heavy	API-native extensions
Role separation	Limited	Stronger
Multi-team support	Harder	Better
Advanced traffic	Controller-specific	More standardized
Learning curve	Lower	Higher

PRACTICAL ANSWER

Ingress is still common and widely used. Gateway API is designed as a more powerful, extensible, and role-oriented successor for advanced traffic management.

31. When to Use Ingress

Use Ingress when:

- You need simple HTTP/HTTPS routing.

- Your cluster already has a stable Ingress Controller.
- Your routing needs are simple.
- Your platform standard uses Ingress.
- You want a mature and widely supported pattern.

32. When to Use Gateway API

Use Gateway API when:

- You need advanced routing.
- You want better separation between platform and app teams.
- You need more standardized traffic APIs.
- Your organization is building a platform.
- Your controller supports Gateway API well.
- You want future-ready traffic management.

33. External Traffic Flow Summary

Typical web traffic flow:

```
graph TD
    User --> DNS
    DNS --> CloudLoadBalancer[Cloud Load Balancer]
    CloudLoadBalancer --> IngressController[Ingress Controller / Gateway]
    IngressController --> KubernetesService[Kubernetes Service]
    KubernetesService --> Pod
    Pod --> Container
```

This flow is important for interviews and troubleshooting.

34. Troubleshooting Ingress

Problem: Ingress Not Getting Address

Check:

```
kubectl get ingress
kubectl describe ingress <name>
kubectl get svc -n <ingress-namespace>
```

SHELL

Common causes:

- Ingress Controller not installed.
- Wrong IngressClass.
- LoadBalancer Service pending.
- Cloud permissions missing.
- Controller error.

Problem: 404 Not Found

Possible causes:

- Host mismatch.
- Path mismatch.
- Wrong Ingress rule.
- Wrong IngressClass.
- Request not reaching correct controller.
- Default backend response.

Check:

```
kubectl describe ingress <name>
kubectl logs -n <ingress-namespace> <controller-pod>
```

SHELL

Problem: 502 or 503 Error

Possible causes:

- Service has no endpoints.
- Backend Pods not ready.
- Wrong targetPort.
- App not listening.
- NetworkPolicy blocking.
- Backend protocol mismatch.

Check:

```
kubectl get svc
kubectl get endpoints
kubectl get endpointslices
kubectl describe pod <pod>
kubectl logs <pod>
```

Problem: TLS Certificate Error

Possible causes:

- Secret missing.
- Secret in wrong namespace.
- Wrong certificate CN/SAN.
- Expired certificate.
- Ingress references wrong secret.
- cert-manager failure.

Check:

```
kubectl get secret
kubectl describe ingress <name>
kubectl describe certificate
kubectl logs -n cert-manager deployment/cert-manager
```

Problem: DNS Not Pointing Correctly

Check:

```
nslookup app.example.com
dig app.example.com
kubectl get ingress
```

Verify DNS points to the load balancer address.

35. Production Best Practices

- Use Ingress or Gateway API for HTTP/HTTPS traffic.
- Use Services for internal routing.
- Use TLS for external traffic.
- Automate certificates using cert-manager where appropriate.
- Use ExternalDNS for DNS automation where appropriate.
- Use separate internal and external ingress classes.
- Avoid exposing every Service as LoadBalancer.

- Use readiness probes so traffic only reaches ready Pods.
- Set request body size/timeouts carefully.
- Use rate limiting/WAF where needed.
- Monitor ingress controller metrics and logs.
- Use consistent annotations and document controller-specific behavior.
- Use Gateway API for advanced platform-level traffic management when supported.
- Keep Ingress/Gateway manifests in Git.

36. Common Mistakes

Mistake	Why Bad
Creating Ingress without controller	Nothing routes traffic
Wrong ingressClassName	Controller ignores Ingress
Service selector mismatch	502/503 due to no endpoints
TLS Secret in wrong namespace	TLS fails
DNS not pointing to load balancer	Users cannot reach app
Using controller-specific annotations blindly	Not portable
Exposing all Services as LoadBalancer	Cost/security issue
No readiness probes	Traffic may hit unready Pods

37. Interview Questions and Answers

Beginner

1. What is Ingress?

Ingress is a Kubernetes object that defines HTTP/HTTPS routing rules to Services.

2. What is an Ingress Controller?

An Ingress Controller is the component that implements Ingress routing rules.

3. Difference between Service and Ingress?

Service provides stable networking to Pods. Ingress provides HTTP/HTTPS routing from outside the cluster to Services.

4. Can Ingress work without an Ingress Controller?

No. Ingress rules need a controller to implement them.

5. What is TLS termination?

TLS termination means HTTPS is decrypted at the load balancer or Ingress Controller.

Intermediate

1. What is host-based routing?

Routing traffic based on domain name, such as api.example.com.

2. What is path-based routing?

Routing traffic based on URL path, such as /api.

3. What is ingressClassName?

It specifies which Ingress Controller should handle the Ingress.

4. Why use cert-manager?

To automate certificate issuance and renewal.

5. What causes 503 from Ingress?

Usually no healthy backend endpoints, wrong Service targetPort, unready Pods, or backend connection failure.

Advanced

1. Why was Gateway API introduced?

To provide a more expressive, extensible, and role-oriented traffic routing model than Ingress.

2. What is HTTPRoute?

HTTPRoute is a Gateway API resource that defines HTTP routing rules from a Gateway to Services.

3. How does Gateway API help platform teams?

It separates infrastructure gateway ownership from application route ownership.

4. Why are annotations a limitation of Ingress?

Many advanced features are controller-specific annotations, reducing portability and consistency.

5. When would you use LoadBalancer Service instead of Ingress?

For non-HTTP TCP/UDP services, simple direct exposure, or workloads needing L4 load balancing.

38. Scenario-Based Interview Questions

Scenario 1: Ingress Created but No External Address

QUESTION Ingress is created, but no address appears. What do you check?

EXPECTED ANSWER Check whether Ingress Controller is installed, ingressClassName is correct, controller Service has external IP, LoadBalancer is provisioned, and controller logs/events.

Scenario 2: Ingress Returns 404

QUESTION User gets 404 from Ingress. What may be wrong?

EXPECTED ANSWER Host or path rule may not match, wrong IngressClass, request may be going to default backend, or DNS may point to wrong controller.

Scenario 3: Ingress Returns 503

QUESTION User gets 503 from Ingress. What do you check?

EXPECTED ANSWER Check Service endpoints, Pod readiness, Service targetPort, backend logs, NetworkPolicy, and Ingress Controller logs.

Scenario 4: TLS Not Working

QUESTION HTTPS shows certificate error. What do you check?

EXPECTED ANSWER Check TLS Secret, certificate hostnames, expiration, Ingress tls section, namespace, cert-manager Certificate/Order/Challenge resources, and DNS validation.

39. Hands-On Labs

Lab 1: Create Basic Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
spec:
  ingressClassName: nginx
  rules:
    - host: web.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80
```

YAML

Apply:

```
kubectl apply -f ingress.yaml
kubectl get ingress
kubectl describe ingress web
```

SHELL

Lab 2: Test With Host Header

```
curl -H "Host: web.local" http://<ingress-address>
```

SHELL

Lab 3: Check Service Endpoints

```
kubectl get svc web
kubectl get endpoints web
kubectl get endpointslices
```

SHELL

Lab 4: Debug 503

Break the Service selector intentionally.

Check:

```
kubectl get endpoints web  
kubectl describe ingress web
```

Fix selector.

40. Chapter Revision Notes

- Ingress defines HTTP/HTTPS routing rules.
- Ingress requires an Ingress Controller.
- Service routes traffic to Pods; Ingress routes external HTTP traffic to Services.
- Ingress supports host-based and path-based routing.
- TLS certificates are stored in Secrets.
- cert-manager can automate certificate management.
- ExternalDNS can automate DNS records.
- Gateway API is a newer, more expressive traffic routing API.
- Gateway API separates platform and application responsibilities.
- 404 usually means route mismatch.
- 503 usually means backend Service/Pod endpoint issue.
- Readiness probes are important for safe routing.

41. Key Interview Statement

Memorize this:

KEY STATEMENT

Ingress is a Kubernetes API object that defines HTTP and HTTPS routing rules, but it requires an Ingress Controller to actually process traffic. External traffic usually flows from DNS to a cloud load balancer, then to the Ingress Controller, then to a Kubernetes Service, and finally to ready backend Pods. Gateway API provides a newer and more expressive model for advanced traffic routing.

End of Volume 02 — Chapter 06

Next chapter:

Chapter 07 — ConfigMaps, Secrets, Environment Variables, and Application Configuration

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

ConfigMaps, Secrets & Application Configuration

Chapter 07 — externalizing configuration safely
with ConfigMaps, Secrets, and env vars.

👑 Chapter 07

Chapter Purpose

Applications need configuration.

Examples:

- Database host
- Log level
- Feature flags
- API URLs
- Environment name
- Timeout values
- Credentials
- Tokens
- Certificates

In Kubernetes, configuration should not usually be hardcoded inside container images. Instead, Kubernetes provides ConfigMaps and Secrets.

This chapter explains how to manage application configuration safely and cleanly.

Learning Objectives

By the end of this chapter, you should be able to explain:

- What ConfigMaps are.
- What Secrets are.
- Difference between ConfigMaps and Secrets.
- Why configuration should be externalized.
- How to inject config as environment variables.
- How to mount config as files.
- How Secrets are stored.
- Secret security risks.
- Common configuration patterns.
- Troubleshooting configuration issues.
- Production best practices.
- Interview answers.

1. Why Application Configuration Matters

A container image should be reusable.

The same image should run in:

```
dev
qa
stage
prod
```

But each environment may need different configuration.

Example:

```
dev database host    → dev-db.internal
prod database host   → prod-db.internal
dev log level        → debug
prod log level       → info
```

If configuration is baked into the image, you need separate images for each environment.

That is not ideal.

Better:

Same image + different runtime configuration

2. Bad Configuration Pattern

Bad Dockerfile:

```
ENV DATABASE_HOST=prod-db.internal
ENV LOG_LEVEL=info
ENV API_KEY=secret-value
```

SHELL

Problems:

- Image is environment-specific.
- Secrets may leak.
- Changing config requires rebuilding image.
- Same image cannot be reused safely.
- Harder rollback.
- Poor security.

3. Good Configuration Pattern

Good pattern:

Application image contains code.

Kubernetes provides configuration at runtime.

Example:

```
Image: myapp:1.4.2
ConfigMap: APP_ENV=prod, LOG_LEVEL=info
Secret: DATABASE_PASSWORD=...
```

This keeps image and configuration separate.

4. Real-World Analogy

Think of a mobile phone.

The phone hardware is the same.

But settings can change:

- Language
- Wi-Fi password
- Brightness
- Wallpaper
- Notifications

You do not rebuild the phone to change brightness.

Similarly, you should not rebuild an image just to change log level.

ConfigMaps and Secrets are runtime settings.

5. What Is a ConfigMap?

A ConfigMap is a Kubernetes object used to store non-sensitive configuration data.

SIMPLE DEFINITION A ConfigMap stores non-secret configuration for applications.

Examples:

- APP_ENV=production
- LOG_LEVEL=info
- API_URL=https://api.example.com
- FEATURE_X_ENABLED=true
- Config files
- nginx config
- application properties

6. ConfigMap Example

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: production
  LOG_LEVEL: info
  API_URL: https://api.example.com
```

YAML

Apply:

```
kubectl apply -f configmap.yaml
```

SHELL

Check:

```
kubectl get configmap
kubectl describe configmap app-config
```

SHELL

7. ConfigMap YAML Explanation

```
apiVersion: v1
```

YAML

ConfigMap belongs to the core API.

```
kind: ConfigMap
```

YAML

Creates a ConfigMap object.

```
metadata:
  name: app-config
```

YAML

Names the ConfigMap.

```
data:
```

YAML

Stores key-value configuration.

```
APP_ENV: production
```

YAML

Defines configuration key APP_ENV.

8. Using ConfigMap as Environment Variables

Deployment example:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: app
  template:
    metadata:
      labels:
        app: app
    spec:
      containers:
        - name: app
          image: myapp:1.0
          envFrom:
            - configMapRef:
                name: app-config
```

YAML

This loads all ConfigMap keys as environment variables.

Inside the container:

```
APP_ENV=production
LOG_LEVEL=info
API_URL=https://api.example.com
```

9. Using Specific ConfigMap Keys

Instead of loading all keys, load selected keys:

```
env:
  - name: APP_ENV
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: APP_ENV
  - name: LOG_LEVEL
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: LOG_LEVEL
```

This is more explicit.

10. Mounting ConfigMap as Files

ConfigMaps can be mounted as files.

ConfigMap:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  nginx.conf: |
    server {
      listen 80;
      location / {
        return 200 "hello from configmap\n";
      }
    }
  }
```

Pod volume mount:

```
volumeMounts:
  - name: nginx-config
    mountPath: /etc/nginx/conf.d
volumes:
  - name: nginx-config
    configMap:
      name: nginx-config
```

The key nginx.conf becomes a file.

11. ConfigMap as File Diagram

```
ConfigMap
  key: nginx.conf
    |
    v
Mounted Volume
    |
    v
Container File
/etc/nginx/conf.d/nginx.conf
```

This pattern is common for config files.

12. ConfigMap Limitations

ConfigMaps are not designed for:

- Sensitive data
- Large binary files
- Huge configuration payloads
- High-frequency dynamic config updates
- Strong encryption needs

For sensitive data, use Secrets.

13. What Is a Secret?

A Secret is a Kubernetes object used to store sensitive information.

SIMPLE DEFINITION A Secret stores sensitive data such as passwords, tokens, keys, and certificates.

Examples:

- Database password
- API token
- TLS private key
- Docker registry credential
- OAuth client secret
- SSH private key

14. Secret Example

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
stringData:
  DATABASE_PASSWORD: supersecret
  API_TOKEN: abc123
```

YAML

Apply:

```
kubectl apply -f secret.yaml
```

SHELL

Check:

```
kubectl get secret
kubectl describe secret app-secret
```

SHELL

15. stringData vs data

Secrets can use stringData or data.

stringData

Human-readable plain text in YAML:

```
stringData:
  PASSWORD: mypassword
```

YAML

Kubernetes converts it to base64 internally.

data

Base64-encoded values:

```
data:
  PASSWORD: bXlwYXNzd29yZA==
```

YAML

IMPORTANT Base64 is encoding, not encryption.

16. Base64 Is Not Encryption

This is a very common interview question.

Base64 only converts data into text format.

Anyone can decode it.

Example:

```
echo "bXlwYXNzd29yZA==" | base64 -d
```

SHELL

Output:

```
mypassword
```

Therefore:

Kubernetes Secrets are not automatically secure just because values are base64 encoded.

17. Using Secret as Environment Variable

```
env:
  - name: DATABASE_PASSWORD
    valueFrom:
      secretKeyRef:
        name: app-secret
        key: DATABASE_PASSWORD
```

YAML

Inside container:

```
DATABASE_PASSWORD=supersecret
```

18. Using Secret as Volume

```
volumeMounts:
  - name: app-secret
    mountPath: /etc/secrets
    readOnly: true
volumes:
  - name: app-secret
    secret:
      secretName: app-secret
```

YAML

The Secret keys become files:

```
/etc/secrets/DATABASE_PASSWORD  
/etc/secrets/API_TOKEN
```

This is often better than environment variables for sensitive data.

19. Environment Variable vs Mounted Secret File

Method	Pros	Cons
Environment variable	Simple, common	May appear in process/env dumps
Mounted file	Better for certificates/tokens	App must read file
External secret manager	Stronger production pattern	More components

For highly sensitive production secrets, use external secret management.

20. Secret Types

Common Kubernetes Secret types:

Type	Purpose
Opaque	Generic key-value secrets
kubernetes.io/tls	TLS certificate and key
kubernetes.io/dockerconfigjson	Registry credentials
kubernetes.io/service-account-token	Service account token

21. TLS Secret

Create TLS Secret:

```
kubectl create secret tls app-tls \  
  --cert=tls.crt \  
  --key=tls.key
```

SHELL

Used by Ingress:

YAML

```
tls:
  - hosts:
    - app.example.com
    secretName: app-tls
```

22. Image Pull Secret

Private registries require credentials.

Create:

SHELL

```
kubectl create secret docker-registry regcred \
  --docker-server=registry.example.com \
  --docker-username=myuser \
  --docker-password=myspassword \
  --docker-email=user@example.com
```

Use in Pod:

YAML

```
imagePullSecrets:
  - name: regcred
```

This allows kubelet to pull private images.

23. ConfigMap Updates

If a ConfigMap is mounted as a volume, updates may eventually appear in the mounted files.

But if ConfigMap is injected as environment variables, running containers do not automatically update.

IMPORTANT

Environment variables are fixed when the container starts.

To apply env var changes, restart Pods.

24. Secret Updates

Similar to ConfigMaps:

- Secret mounted as volume may update eventually.
- Secret used as environment variable requires Pod restart.
- Some applications must reload config explicitly.

Production systems often use rollout restart after config changes.

```
kubectl rollout restart deployment/app
```

SHELL

25. Config Change Rollout Pattern

Common pattern:

```
ConfigMap changes
  |
  v
Deployment rollout restart
  |
  v
Pods recreated
  |
  v
New config loaded
```

Helm charts often include checksum annotations to trigger rollouts when config changes.

Example:

```
metadata:
  annotations:
    checksum/config: "<hash-of-config>"
```

YAML

26. External Secret Management

Kubernetes Secrets alone may not be enough for production.

External systems include:

- AWS Secrets Manager
- AWS SSM Parameter Store
- HashiCorp Vault
- Azure Key Vault
- Google Secret Manager
- External Secrets Operator
- Secrets Store CSI Driver

These tools improve secret lifecycle, rotation, auditing, and access control.

27. External Secrets Operator

External Secrets Operator syncs secrets from external secret stores into Kubernetes Secrets.

Flow:

```
graph TD
    A[AWS Secrets Manager / Vault / SSM] --> B[External Secrets Operator]
    B --> C[Kubernetes Secret]
    C --> D[Pod consumes Secret]
```

This is common in production cloud-native platforms.

28. Secrets Store CSI Driver

Secrets Store CSI Driver mounts secrets from external secret stores directly into Pods as volumes.

Flow:

```
graph TD
    A[External Secret Store] --> B[CSI Driver]
    B --> C[Mounted file inside Pod]
```

This can reduce the need to persist secrets as Kubernetes Secret objects, depending on configuration.

29. Configuration Precedence

Applications may read configuration from multiple places:

- Default app config
- Environment variables
- Config files
- Command-line arguments
- Secret files
- External config service

Define a clear precedence order.

Example:

```
command-line arg > environment variable > config file > default
```

Without clear rules, troubleshooting becomes hard.

30. Troubleshooting ConfigMaps

Problem: Environment variable missing

Check ConfigMap:

```
kubectl get configmap app-config -o yaml
```

SHELL

Check Pod spec:

```
kubectl describe pod <pod>
```

SHELL

Exec into Pod:

```
kubectl exec -it <pod> -- env
```

SHELL

Common causes:

- Wrong ConfigMap name
- Wrong key
- Wrong namespace
- Pod not restarted
- YAML indentation error

Problem: Config file not mounted

Check:

```
kubectl describe pod <pod>
```

SHELL

Look for:

- Volumes
- Mounts
- Events

Exec:

```
kubectl exec -it <pod> -- ls -l /path
```

SHELL

Common causes:

- Wrong mountPath
- Wrong ConfigMap name
- Key missing
- Mount overwrote directory unexpectedly

31. Troubleshooting Secrets

Problem: Secret not found

Error:

```
secret "app-secret" not found
```

Check:

```
kubectl get secret  
kubectl get secret -n <namespace>
```

SHELL

Common causes:

- Secret in wrong namespace
- Typo in name
- Secret not created
- GitOps sync issue

Problem: Private image pull fails

Symptoms:

```
ImagePullBackOff  
ErrImagePull  
unauthorized
```

Check:

```
kubectl describe pod <pod>  
kubectl get secret regcred -o yaml
```

SHELL

Fix:

- Create correct imagePullSecret
- Reference it in Pod/ServiceAccount
- Verify registry credentials
- Verify image name/tag

32. Security Best Practices for Secrets

- Do not commit raw secrets to Git.
- Use external secret managers.
- Enable encryption at rest for Kubernetes Secrets.
- Restrict Secret access with RBAC.
- Avoid exposing Secrets as environment variables when possible.
- Rotate secrets regularly.
- Audit secret access.
- Use namespace isolation.
- Avoid broad permissions like get secrets for all users.
- Do not print secrets in logs.
- Avoid storing secrets in container images.

33. ConfigMap Best Practices

- Store non-sensitive configuration only.
- Keep ConfigMaps small and focused.
- Use clear naming.
- Version configuration through GitOps.
- Restart workloads when required.
- Use checksum annotations in Helm.
- Avoid putting secrets in ConfigMaps.
- Use separate ConfigMaps per app or purpose.
- Document required config keys.
- Validate config during startup.

34. Common Mistakes

Mistake	Why Bad
Putting passwords in ConfigMaps	ConfigMaps are not for secrets
Thinking base64 is encryption	It is only encoding
Forgetting Pod restart after env config change	Old env values remain
Secret in wrong namespace	Pod cannot access it
Wrong key name	Env var/file missing
Committing secret YAML to Git	Secret leakage
Too much config in one object	Hard to manage
App cannot reload config	Requires rollout restart

35. Production Configuration Pattern

Good production pattern:

```
Container image:
  code only

ConfigMap:
  non-sensitive config

Secret / External Secret:
  sensitive config

Deployment:
  references ConfigMap and Secret

GitOps:
  stores manifests, not raw secrets

Secret manager:
  stores actual sensitive values
```

36. Interview Questions and Answers

Beginner

1. What is a ConfigMap?

A ConfigMap stores non-sensitive configuration data for Kubernetes workloads.

2. What is a Secret?

A Secret stores sensitive data such as passwords, tokens, keys, and certificates.

3. Difference between ConfigMap and Secret?

ConfigMap stores non-sensitive data. Secret stores sensitive data.

4. Is base64 encryption?

No. Base64 is encoding, not encryption.

5. How can Pods consume ConfigMaps?

As environment variables or mounted files.

Intermediate

1. What happens if a ConfigMap used as env var changes?

Running containers do not automatically get updated environment variables. Pods must be restarted.

2. How do you mount a Secret as a file?

Define a Secret volume and mount it into the container with volumeMounts.

3. What is imagePullSecret?

A Secret used by kubelet to authenticate to private container registries.

4. Why should secrets not be stored in Git?

Git history can expose secrets permanently, even if later removed.

5. Why use external secret managers?

They provide better rotation, auditing, access control, and integration with cloud IAM.

Advanced

1. How do Helm charts trigger rollout on ConfigMap changes?

They often add checksum annotations to Pod templates. When config changes, checksum changes, triggering a new ReplicaSet rollout.

2. Why restrict RBAC access to Secrets?

Anyone with permission to read Secrets can potentially access sensitive credentials.

3. What is Secrets Store CSI Driver?

It mounts secrets from external secret stores into Pods using CSI volumes.

4. What is External Secrets Operator?

It syncs secrets from external secret stores into Kubernetes Secrets.

5. Why might mounted Secret files update but env vars not?

Mounted Secret volumes can be refreshed by kubelet eventually, but environment variables are fixed at container start.

37. Scenario-Based Interview Questions

Scenario 1: Config Change Not Reflected

QUESTION A ConfigMap value was changed, but the application still uses the old value.

EXPECTED ANSWER If the ConfigMap was injected as environment variables, the Pod must be restarted. Use `kubectl rollout restart deployment/<name>`.

Scenario 2: Secret Not Found

QUESTION Pod fails with Secret not found.

EXPECTED ANSWER Check Secret name, namespace, Pod reference, and whether GitOps or deployment process created the Secret before the Pod.

Scenario 3: Private Image Pull Fails

QUESTION Pod fails with ImagePullBackOff due to unauthorized registry access.

EXPECTED ANSWER Create or fix imagePullSecret and reference it in Pod spec or ServiceAccount.

Scenario 4: Password Stored in ConfigMap

QUESTION A database password is stored in a ConfigMap. What is wrong?

EXPECTED ANSWER ConfigMaps are for non-sensitive data. Move the password to a Secret or external secret manager and rotate the exposed password.

38. Hands-On Labs

Lab 1: Create ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: dev
  LOG_LEVEL: debug
```

YAML

Apply:

```
kubectl apply -f configmap.yaml
kubectl get configmap app-config -o yaml
```

SHELL

Lab 2: Use ConfigMap as Env

```
apiVersion: v1
kind: Pod
metadata:
  name: config-env-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "env && sleep 3600"]
      envFrom:
        - configMapRef:
            name: app-config
```

YAML

Check:

```
kubectl logs config-env-demo
```

SHELL

Lab 3: Create Secret

```
kubectl create secret generic app-secret \
  --from-literal=DATABASE_PASSWORD=supersecret
```

SHELL

Check:

```
kubectl get secret app-secret -o yaml
```

SHELL

Decode:

```
kubectl get secret app-secret -o jsonpath='{.data.DATABASE_PASSWORD}' | base64 -d
```

SHELL

Lab 4: Mount Secret as File

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-file-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "ls -l /etc/secrets && sleep 3600"]
      volumeMounts:
        - name: secret-volume
          mountPath: /etc/secrets
          readOnly: true
  volumes:
    - name: secret-volume
      secret:
        secretName: app-secret
```

YAML

Apply:

```
kubectl apply -f secret-file-demo.yaml
kubectl exec -it secret-file-demo -- ls -l /etc/secrets
```

SHELL

39. Chapter Revision Notes

- ConfigMaps store non-sensitive configuration.
- Secrets store sensitive configuration.
- Base64 is not encryption.

- ConfigMaps and Secrets can be used as env vars or mounted files.
- Env vars do not update inside running containers automatically.
- Mounted config/secret files may update eventually.
- Secret access must be restricted with RBAC.
- Do not store secrets in Git.
- Use external secret managers for production.
- Use imagePullSecrets for private registries.
- Restart workloads after config changes when needed.
- Keep container images environment-neutral.

40. Key Interview Statement

Memorize this:

KEY STATEMENT ConfigMaps store non-sensitive configuration, while Secrets store sensitive data such as passwords and tokens. Both can be injected into Pods as environment variables or mounted files. Base64 encoding in Secrets is not encryption, so production systems should restrict Secret access with RBAC, enable encryption at rest, avoid committing secrets to Git, and often integrate external secret managers.

End of Volume 02 — Chapter 07

Next chapter:

Chapter 08 — Kubernetes Storage, Volumes, PersistentVolumes, PVCs, StorageClasses, and CSI

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

Kubernetes Storage, Volumes, PVs, PVCs & CSI

Chapter 08 — persistent storage with Volumes,
PVs, PVCs, StorageClasses, and CSI.

👑 Chapter 08

Volumes · PV · PVC · StorageClass · CSI

Chapter Purpose

Containers and Pods are temporary by design.

A Pod can be deleted, rescheduled, restarted, or moved to another node. If an application stores important data only inside the container filesystem, that data can be lost.

Kubernetes storage solves this problem by allowing Pods to use persistent storage.

This chapter explains Kubernetes storage from beginner level to production level.

Learning Objectives

By the end of this chapter, you should be able to explain:

- Why Kubernetes storage is needed.
- Difference between ephemeral and persistent storage.
- Kubernetes volumes.
- `emptyDir`.
- `hostPath`.
- `PersistentVolumes`.
- `PersistentVolumeClaims`.
- `StorageClasses`.
- Dynamic provisioning.
- CSI drivers.
- Access modes.
- Reclaim policies.
- Volume binding modes.
- Stateful storage patterns.
- Storage troubleshooting.
- Production best practices.

1. Why Kubernetes Storage Matters

Pods are replaceable.

This is good for reliability and scaling.

But applications often need data that survives Pod deletion.

Examples:

- Database files
- Uploaded files

- Message queue data
- Cache snapshots
- Application state
- Reports
- Persistent logs
- Model files
- Shared data

If a Pod is deleted, its local container filesystem disappears.

Therefore, important data must live outside the Pod lifecycle.

2. Ephemeral vs Persistent Storage

Storage Type	Lifetime
Ephemeral storage	Lives only with Pod/container
Persistent storage	Survives Pod restart/replacement

Ephemeral storage is useful for:

- Temporary files
- Cache
- Scratch space
- Intermediate processing

Persistent storage is needed for:

- Databases
- User uploads
- Stateful applications
- Durable files
- Shared application data

3. Real-World Analogy

Think of a hotel room.

A Pod is like a hotel room.

Anything you leave in the room may be cleaned when you check out.

Persistent storage is like a bank locker.

Even if you leave the hotel, your locker data remains.

Kubernetes Concept	Analogy
Pod	Hotel room
Container filesystem	Temporary items in room
emptyDir	Temporary drawer
PersistentVolume	Bank locker
PVC	Request form for locker
StorageClass	Type of locker
CSI Driver	Locker management company

4. Kubernetes Volume

A Kubernetes volume is storage that can be mounted into containers in a Pod.

Example:

```
volumes:
- name: data
  emptyDir: {}
```

YAML

Container uses it:

```
volumeMounts:
- name: data
  mountPath: /data
```

YAML

A volume belongs to the Pod, and containers inside the Pod can mount it.

5. Volume Mount

A volumeMount defines where a volume appears inside a container.

Example:

```
volumeMounts:
- name: data
  mountPath: /app/data
```

YAML

This means the volume named data is mounted at:

```
/app/data
```

inside the container.

6. emptyDir

emptyDir is temporary storage created when a Pod starts on a node.

Example:

```
apiVersion: v1
kind: Pod
metadata:
  name: emptydir-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "echo hello > /data/message.txt && sleep 3600"]
      volumeMounts:
        - name: temp-data
          mountPath: /data
  volumes:
    - name: temp-data
      emptyDir: {}
```

YAML

emptyDir exists as long as the Pod exists.

If the Pod is deleted, the data is deleted.

7. emptyDir Use Cases

Use emptyDir for:

- Temporary files
- Scratch space
- Sharing files between containers in same Pod
- Cache that does not need persistence
- Init container output used by app container

Do not use emptyDir for durable database data.

8. emptyDir Multi-Container Example

YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: shared-emptydir
spec:
  initContainers:
    - name: init
      image: busybox:1.36
      command: ["sh", "-c", "echo generated > /shared/file.txt"]
      volumeMounts:
        - name: shared
          mountPath: /shared
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "cat /shared/file.txt && sleep 3600"]
      volumeMounts:
        - name: shared
          mountPath: /shared
  volumes:
    - name: shared
      emptyDir: {}
```

Flow:

```
Init container writes file
      |
      v
App container reads file
```

9. hostPath

hostPath mounts a file or directory from the worker node into the Pod.

Example:

YAML

```
volumes:
  - name: host-logs
    hostPath:
      path: /var/log
      type: Directory
```

Use carefully.

hostPath ties the Pod to node-specific storage.

10. hostPath Risks

Risks:

- Security risk if mounting sensitive host paths
- Not portable across nodes
- Data depends on one node
- Pod rescheduled to another node may lose access
- Can expose host filesystem
- May break cluster isolation

Avoid hostPath for normal application storage.

Common valid uses:

- Node agents
- Log collectors
- Monitoring agents
- CNI/CSI system components
- Special infrastructure workloads

11. PersistentVolume

A PersistentVolume, or PV, represents a piece of storage in the cluster.

SIMPLE DEFINITION

A PersistentVolume is storage provisioned for Kubernetes.

A PV may represent:

- AWS EBS volume
- AWS EFS filesystem
- NFS share
- Azure Disk
- GCE Persistent Disk
- Ceph volume
- Local disk

PV is a cluster resource.

12. PersistentVolumeClaim

A PersistentVolumeClaim, or PVC, is a request for storage by a user or application.

SIMPLE DEFINITION A PVC is a request for persistent storage.

The application does not usually directly choose the physical disk.

It creates a PVC saying:

I need 10Gi storage with ReadWriteOnce access.

Kubernetes binds the PVC to a suitable PV or dynamically provisions one.

13. PV and PVC Analogy

Think of office storage lockers.

PV:

Actual locker available in building.

PVC:

Employee request for a locker.

StorageClass:

Type of locker: small, large, premium, shared.

CSI Driver:

Company that creates and manages lockers.

14. PV/PVC Flow

```
Pod
|
v
PVC
|
v
PV
|
v
Actual Storage
```

The Pod mounts the PVC.

The PVC binds to a PV.

The PV maps to real storage.

15. PVC Example

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

YAML

This requests 10Gi storage.

16. Pod Using PVC

```
apiVersion: v1
kind: Pod
metadata:
  name: pvc-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "echo hello > /data/message.txt && sleep 3600"]
      volumeMounts:
        - name: app-storage
          mountPath: /data
  volumes:
    - name: app-storage
      persistentVolumeClaim:
        claimName: app-data
```

YAML

The Pod mounts PVC app-data at /data.

17. Access Modes

Access modes define how storage can be mounted.

Access Mode	Meaning
ReadWriteOnce	Mounted read-write by one node
ReadOnlyMany	Mounted read-only by many nodes
ReadWriteMany	Mounted read-write by many nodes
ReadWriteOncePod	Mounted read-write by one Pod

Common:

ReadWriteOnce

for block storage like AWS EBS.

ReadWriteMany

for shared filesystems like NFS or AWS EFS.

18. ReadWriteOnce Important Note

ReadWriteOnce means one node, not always one Pod.

Multiple Pods on the same node may sometimes use the same RWO volume depending on storage and scheduling.

For strict one-Pod access, use ReadWriteOncePod if supported.

19. StorageClass

A StorageClass defines a type of storage and how it should be dynamically provisioned.

Example:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

YAML

This defines AWS EBS gp3 storage using EBS CSI driver.

20. Why StorageClass Exists

Different applications need different storage.

Examples:

App	Storage Need
Database	Fast block storage
Shared uploads	Shared filesystem
Logs	Large cheaper storage
Cache	Temporary storage
ML model files	Read-heavy shared storage

StorageClass lets platform teams define storage options.

Developers request them by name.

21. PVC With StorageClass

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data
spec:
  storageClassName: fast
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
```

YAML

This asks Kubernetes for 20Gi from StorageClass fast.

22. Dynamic Provisioning

Dynamic provisioning means Kubernetes automatically creates storage when a PVC is created.

Flow:

```
PVC created
|
v
StorageClass selected
|
v
CSI driver provisions storage
|
v
PV created
|
v
PVC bound to PV
|
v
Pod mounts storage
```

This avoids manually creating PVs.

23. Static Provisioning

Static provisioning means an administrator manually creates PVs.

Then users create PVCs that bind to available PVs.

Use cases:

- Existing storage
- Manually managed NFS
- Special storage requirements
- Migration scenarios

Dynamic provisioning is more common in cloud environments.

24. CSI

CSI stands for Container Storage Interface.

It is the standard interface for storage providers in Kubernetes.

CSI drivers allow Kubernetes to work with many storage systems.

Examples:

- AWS EBS CSI Driver
- AWS EFS CSI Driver
- Azure Disk CSI Driver
- GCE PD CSI Driver
- Ceph CSI
- NFS CSI

25. CSI Responsibilities

CSI drivers can handle:

- Create volume
- Delete volume
- Attach volume
- Detach volume
- Mount volume
- Unmount volume
- Expand volume
- Snapshot volume

Kubernetes delegates storage-specific work to CSI drivers.

26. Reclaim Policy

Reclaim policy controls what happens to the PV when PVC is deleted.

Common values:

Policy	Meaning
Delete	Delete underlying storage
Retain	Keep underlying storage

Example:

```
reclaimPolicy: Delete
```

YAML

For production databases, be careful with Delete.

Retain may be safer for critical data.

27. Volume Binding Mode

StorageClass supports volume binding mode.

Common values:

Mode	Meaning
Immediate	Provision/bind volume immediately
WaitForFirstConsumer	Wait until Pod is scheduled

WaitForFirstConsumer is important for zone-aware storage.

Example:

AWS EBS volumes are zonal.

If volume is created in one AZ but Pod schedules to another AZ, mount fails.

WaitForFirstConsumer helps Kubernetes choose storage zone based on Pod scheduling.

28. Volume Expansion

Some StorageClasses allow volume expansion.

StorageClass:

```
allowVolumeExpansion: true
```

YAML

PVC can be edited:

```
resources:
  requests:
    storage: 50Gi
```

YAML

Kubernetes and CSI driver expand the volume if supported.

29. StatefulSet Storage Preview

StatefulSets commonly use PVC templates.

Each Pod gets its own PVC.

Example:

```
postgres-0 → data-postgres-0
postgres-1 → data-postgres-1
postgres-2 → data-postgres-2
```

This is useful for stateful workloads.

StatefulSets are covered later.

30. Storage Troubleshooting

Problem: PVC Pending

Check:

```
kubectl get pvc
kubectl describe pvc <pvc-name>
kubectl get storageclass
```

SHELL

Common causes:

- No default StorageClass
- Wrong StorageClass name
- CSI driver missing
- Provisioner error
- Access mode unsupported
- Capacity unavailable
- Waiting for first consumer

Problem: Pod Stuck ContainerCreating

Check:

```
kubectl describe pod <pod-name>
kubectl get events
```

SHELL

Common storage causes:

- Volume mount failed
- Attach failed
- Permission issue
- CSI driver error
- Volume in wrong zone
- PVC not bound

Problem: Multi-Attach Error

Common with ReadWriteOnce block storage.

Example:

```
Multi-Attach error for volume
```

Cause:

Volume is already attached to one node and another Pod on another node wants it.

Fix:

- Ensure single writer
- Use StatefulSet properly
- Use ReadWriteMany storage if shared access is required
- Wait for detach
- Check stuck Pods/nodes

Problem: Permission Denied

Check container user:

```
kubectl exec -it <pod> -- id
```

SHELL

Check mount permissions:

```
kubectl exec -it <pod> -- ls -ld /data
```

SHELL

Fix options:

- Set fsGroup
- Use init container to chown
- Configure storage permissions
- Run app with correct UID/GID

31. fsGroup

fsGroup can set group ownership for mounted volumes.

Example:

```
spec:
  securityContext:
    fsGroup: 2000
```

YAML

This can help non-root containers write to mounted volumes.

Use carefully because changing ownership recursively can slow startup on large volumes.

32. Storage Production Best Practices

- Use dynamic provisioning where possible.

- Use appropriate StorageClass for workload.
- Use WaitForFirstConsumer for zonal block storage.
- Use ReadWriteMany storage only when truly needed.
- Avoid hostPath for application data.
- Back up persistent data.
- Test restore procedures.
- Monitor disk usage.
- Monitor PVC capacity.
- Use Retain policy for critical data when appropriate.
- Understand storage performance limits.
- Use StatefulSets for stateful applications.
- Avoid multiple writers on block volumes.
- Use external managed databases when suitable.
- Encrypt volumes where required.
- Use snapshots where supported.

33. Common Mistakes

Mistake	Why Bad
Storing data only in container filesystem	Data lost when Pod dies
Using emptyDir for database	Data lost with Pod
Using hostPath for app data	Node-coupled and risky
Wrong access mode	Pod scheduling/mount failures
No backup	No recovery
Ignoring reclaim policy	Accidental data deletion
Using RWO volume across nodes	Multi-attach errors
No storage monitoring	Disk full outages
Assuming PVC delete always keeps data	Depends on reclaim policy

34. Interview Questions and Answers

Beginner

1. Why does Kubernetes need persistent storage?

Pods are temporary, so important data must survive Pod deletion or recreation.

2. What is a Kubernetes volume?

A volume is storage mounted into containers in a Pod.

3. What is emptyDir?

Temporary Pod storage that exists only while the Pod exists.

4. What is a PersistentVolume?

A PersistentVolume is cluster storage available to Kubernetes.

5. What is a PersistentVolumeClaim?

A PVC is a request for storage by a Pod/application.

Intermediate

1. What is StorageClass?

StorageClass defines a type of storage and how it is dynamically provisioned.

2. What is dynamic provisioning?

Automatic creation of storage when a PVC is created.

3. What is CSI?

CSI is the standard interface Kubernetes uses to integrate storage providers.

4. What is ReadWriteOnce?

A volume access mode where storage can be mounted read-write by one node.

5. What is ReadWriteMany?

A volume access mode where storage can be mounted read-write by many nodes.

Advanced

1. Why is WaitForFirstConsumer useful?

It delays volume provisioning until Pod scheduling so zonal storage can be created in the correct zone.

2. What causes Multi-Attach errors?

A ReadWriteOnce block volume is already attached to one node and another node tries to attach it.

3. What does reclaimPolicy do?

It controls whether underlying storage is deleted or retained when the PVC is deleted.

4. How do you solve volume permission issues for non-root Pods?

Use fsGroup, init containers to set permissions, or configure storage permissions correctly.

5. Why use StatefulSets for stateful apps?

StatefulSets provide stable identities and persistent volume claims per Pod.

35. Scenario-Based Interview Questions

Scenario 1: PVC Pending

QUESTION PVC remains Pending. What do you check?

EXPECTED ANSWER Check StorageClass, CSI driver, events, access mode, requested size, default StorageClass, and whether volume binding mode waits for a Pod.

Scenario 2: Pod Cannot Mount Volume

QUESTION Pod stuck in ContainerCreating due to volume mount error.

EXPECTED ANSWER Check kubectl describe pod, PVC status, CSI driver logs, events, access mode, zone mismatch, and permissions.

Scenario 3: Database Lost Data

QUESTION Database Pod was deleted and data disappeared.

EXPECTED ANSWER It likely used ephemeral storage such as container filesystem or emptyDir. Use PVC-backed persistent storage.

Scenario 4: Multi-Attach Error

QUESTION Pod cannot start due to Multi-Attach error.

EXPECTED ANSWER The RWO volume is attached to another node. Ensure only one node uses it, wait for detach, or use RWX storage if shared access is needed.

36. Hands-On Labs

Lab 1: emptyDir

```
apiVersion: v1
kind: Pod
metadata:
  name: emptydir-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "echo hello > /data/msg && sleep 3600"]
      volumeMounts:
        - name: temp
          mountPath: /data
  volumes:
    - name: temp
      emptyDir: {}
```

YAML

Apply:

```
kubectl apply -f emptydir.yaml
kubectl exec -it emptydir-demo -- cat /data/msg
```

SHELL

Lab 2: PVC

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: demo-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

YAML

Apply:

```
kubectl apply -f pvc.yaml
kubectl get pvc
```

SHELL

Lab 3: Pod Using PVC

```
apiVersion: v1
kind: Pod
metadata:
  name: pvc-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "echo persistent > /data/msg && sleep 3600"]
      volumeMounts:
        - name: data
          mountPath: /data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: demo-pvc
```

YAML

Apply:

```
kubectl apply -f pvc-pod.yaml
kubectl exec -it pvc-demo -- cat /data/msg
```

SHELL

37. Chapter Revision Notes

- Pods are temporary, so persistent data needs external storage.
- emptyDir is temporary Pod storage.
- hostPath mounts node paths but is risky for applications.
- PV represents cluster storage.
- PVC requests storage.
- StorageClass defines dynamic provisioning.
- CSI drivers integrate Kubernetes with storage providers.
- Access modes control how volumes can be mounted.
- Reclaim policy controls what happens after PVC deletion.
- WaitForFirstConsumer helps with zonal storage.
- RWO block volumes can cause Multi-Attach errors.
- Stateful workloads need careful storage design and backups.

38. Key Interview Statement

Memorize this:

KEY STATEMENT

Kubernetes storage separates storage consumption from storage implementation. Applications request storage using PVCs, Kubernetes binds those claims to PersistentVolumes, and StorageClasses with CSI drivers dynamically provision real storage from providers such as EBS, EFS, NFS, or cloud disks. Since Pods are ephemeral, persistent data must be stored outside the container filesystem.

End of Volume 02 — Chapter 08

Next chapter:

Chapter 09 — StatefulSets, DaemonSets, Jobs, and CronJobs

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

StatefulSets, DaemonSets, Jobs & CronJobs

Chapter 09 — choosing the right workload
controller for stateful, node-level, and batch
workloads.

👑 Chapter 09

StatefulSet · DaemonSet · Job · CronJob

Chapter Purpose

Deployments are excellent for stateless applications.

But not every workload is stateless.

Some workloads need stable identity, persistent storage, one Pod per node, or run-to-completion behavior.

Kubernetes provides different workload controllers for different needs:

- StatefulSet
- DaemonSet
- Job
- CronJob

This chapter explains when and why to use each one.

Learning Objectives

By the end of this chapter, you should be able to explain:

- What StatefulSets are.
- Why StatefulSets exist.
- StatefulSet vs Deployment.
- Stable Pod identity.
- Stable storage with volumeClaimTemplates.
- Headless Services.
- What DaemonSets are.
- DaemonSet use cases.
- What Jobs are.
- What CronJobs are.
- Workload controller comparison.
- Troubleshooting workload controllers.
- Production best practices.
- Interview answers.

1. Why Different Workload Controllers Exist

Kubernetes has different workload controllers because applications behave differently.

A web API does not behave like a database.

A log collector does not behave like a batch job.

A scheduled backup does not behave like a frontend service.

Different workloads need different management patterns.

Workload Type	Best Kubernetes Object
Stateless web app	Deployment
Database / ordered identity app	StatefulSet
One Pod on every node	DaemonSet
One-time task	Job
Scheduled task	CronJob

2. Deployment Quick Revision

Deployment is used for stateless replicated apps.

Examples:

- Frontend
- Backend API
- Stateless workers
- Web services
- Microservices

Deployment Pods are replaceable.

Their names and identities are not stable.

Example Pods:

```
api-7c9f6d9b6d-x1abc  
api-7c9f6d9b6d-y2def  
api-7c9f6d9b6d-z3ghi
```

If deleted, new Pods get new names.

This is fine for stateless apps.

3. What Is a StatefulSet?

A StatefulSet is a Kubernetes controller used to manage stateful applications.

SIMPLE DEFINITION

A StatefulSet manages Pods that need stable identity, stable network names, ordered deployment, and stable storage.

Common use cases:

- Databases
- Kafka
- ZooKeeper
- Elasticsearch
- Redis clusters
- Cassandra
- Stateful distributed systems

4. Why StatefulSet Exists

Some applications need stable identity.

Example database cluster:

```
db-0  
db-1  
db-2
```

Each member may have a specific role or data volume.

If db-0 dies, the replacement should still be db-0.

It should use the same storage.

Deployment does not guarantee this.

StatefulSet does.

5. StatefulSet Analogy

Think of assigned office desks.

In a normal hot-desk office, anyone can sit anywhere.

That is like Deployment Pods.

In a bank, each officer may have a fixed desk, fixed drawer, and fixed responsibility.

That is like StatefulSet Pods.

Kubernetes Concept	Analogy
Deployment Pod	Any temporary desk
StatefulSet Pod	Assigned desk
PVC	Personal locked drawer
Pod name	Employee desk number
Headless Service	Directory listing all desks

6. StatefulSet Features

StatefulSet provides:

- Stable Pod names
- Stable network identity
- Ordered creation
- Ordered deletion
- Stable persistent storage
- Predictable scaling
- VolumeClaimTemplates
- Compatibility with Headless Services

Pod names are predictable:

```
mysql-0
mysql-1
mysql-2
```

7. StatefulSet vs Deployment

Feature	Deployment	StatefulSet
Main use	Stateless apps	Stateful apps
Pod names	Random	Stable ordinal names
Storage	Usually shared/none	Stable per-Pod PVC
Startup order	Not guaranteed	Ordered by ordinal
Scaling down	Any Pod	Highest ordinal first
Network identity	Not stable per Pod	Stable DNS
Example	API, frontend	DB, Kafka, ZooKeeper

8. StatefulSet Pod Identity

StatefulSet Pods get stable names.

Example StatefulSet name:

```
postgres
```

Replicas:

```
postgres-0  
postgres-1  
postgres-2
```

If postgres-1 is deleted, Kubernetes recreates:

```
postgres-1
```

not a random new name.

9. Headless Service for StatefulSet

StatefulSets commonly use Headless Services.

A Headless Service has:

```
clusterIP: None
```

```
YAML
```

It allows direct DNS records for individual Pods.

Example DNS:

```
postgres-0.postgres.default.svc.cluster.local  
postgres-1.postgres.default.svc.cluster.local  
postgres-2.postgres.default.svc.cluster.local
```

This is useful for clustered databases and distributed systems.

10. StatefulSet YAML Example

YAML

```
apiVersion: v1
kind: Service
metadata:
  name: postgres
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  replicas: 3
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_PASSWORD
              value: password
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes:
          - ReadWriteOnce
        resources:
          requests:
            storage: 10Gi
```

11. StatefulSet YAML Explanation

```
kind: StatefulSet
```

YAML

Creates a StatefulSet object.

```
metadata:  
  name: postgres
```

YAML

StatefulSet name. Pods will be named postgres-0, postgres-1, and so on.

```
serviceName: postgres
```

YAML

Connects StatefulSet to the Headless Service.

```
replicas: 3
```

YAML

Runs 3 replicas.

```
volumeClaimTemplates:
```

YAML

Creates one PVC per Pod.

For example:

```
data-postgres-0  
data-postgres-1  
data-postgres-2
```

Each Pod gets its own persistent storage.

12. StatefulSet Storage Behavior

Each StatefulSet Pod has its own PVC.

Example:

```
postgres-0 → data-postgres-0  
postgres-1 → data-postgres-1  
postgres-2 → data-postgres-2
```

If postgres-1 is deleted, the replacement postgres-1 uses the same PVC.

This preserves data.

13. StatefulSet Scaling

Scale up:

```
kubectl scale statefulset postgres --replicas=5
```

SHELL

Kubernetes creates:

```
postgres-3  
postgres-4
```

Scale down:

```
kubectl scale statefulset postgres --replicas=2
```

SHELL

Kubernetes removes highest ordinals first:

```
postgres-4  
postgres-3  
postgres-2
```

PVCs are usually retained unless manually deleted.

14. StatefulSet Update Strategy

StatefulSet supports rolling updates.

Default strategy:

```
updateStrategy:  
  type: RollingUpdate
```

YAML

StatefulSet updates Pods in reverse ordinal order.

Example:

```
postgres-2 updated first  
postgres-1 updated next  
postgres-0 updated last
```

This ordered behavior is useful for some stateful systems.

15. When Not to Use StatefulSet

Do not use StatefulSet just because an application writes logs or temporary files.

Use StatefulSet only when the application needs:

- Stable identity
- Stable network name
- Stable per-Pod storage
- Ordered startup/shutdown
- Stateful cluster membership

For normal stateless apps, use Deployment.

16. What Is a DaemonSet?

A DaemonSet ensures that a copy of a Pod runs on every selected node.

SIMPLE DEFINITION A DaemonSet runs one Pod per node.

Common use cases:

- Log collection agents
- Monitoring agents
- Security agents
- CNI agents
- Storage agents
- Node exporters
- Service mesh node components

17. DaemonSet Analogy

Think of a security guard in every building.

If your company has 10 buildings, each building needs one guard.

If a new building opens, it automatically gets a guard.

If a building closes, its guard leaves.

DemonSet works the same way for nodes.

Kubernetes Concept	Analogy
Node	Building
DaemonSet Pod	Security guard
New node	New building
Deleted node	Closed building

18. DaemonSet Behavior

If the cluster has 5 eligible nodes, DaemonSet runs 5 Pods.

If a new node joins, Kubernetes creates a DaemonSet Pod on it.

If a node is removed, that Pod disappears.

DaemonSets can use node selectors, affinity, and tolerations to control which nodes they run on.

19. DaemonSet YAML Example

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-logger
spec:
  selector:
    matchLabels:
      app: node-logger
  template:
    metadata:
      labels:
        app: node-logger
    spec:
      containers:
        - name: logger
          image: busybox:1.36
          command: ["sh", "-c", "while true; do echo collecting logs; sleep 30; done"]
```

Apply:

```
kubectl apply -f daemonset.yaml
```

Check:


```
kubectl get daemonset
kubectl get pods -o wide
```

You should see Pods distributed across nodes.

20. DaemonSet Use Cases

Common production DaemonSets:

Tool Type	Example
Logging	Fluent Bit, Fluentd
Monitoring	Node Exporter
Security	Falco agents
Networking	CNI plugin agents
Storage	CSI node plugins
Cloud	AWS VPC CNI node components

DaemonSets are heavily used by platform teams.

21. DaemonSet and Tolerations

Some nodes have taints.

Example:

```
node-role.kubernetes.io/control-plane:NoSchedule
```

DaemonSets may need tolerations to run on special nodes.

Example:

```
tolerations:
- key: node-role.kubernetes.io/control-plane
  operator: Exists
  effect: NoSchedule
```

Use carefully.

22. What Is a Job?

A Job runs Pods until a task completes successfully.

SIMPLE DEFINITION A Job is used for run-to-completion workloads.

Use cases:

- Database migration
- One-time batch processing
- Data import
- Report generation
- Backup task
- Cleanup task
- Script execution

Unlike Deployment, a Job is not meant to run forever.

23. Job Analogy

Think of a courier delivery task.

The courier has one job:

Deliver this package.

Once delivered, the job is complete.

The courier does not keep delivering forever.

A Kubernetes Job runs until completion.

24. Job YAML Example

```
apiVersion: batch/v1
kind: Job
metadata:
  name: hello-job
spec:
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: hello
          image: busybox:1.36
          command: ["sh", "-c", "echo job started; sleep 5; echo job completed"]
```

YAML

Apply:

```
kubectl apply -f job.yaml
```

SHELL

Check:

```
kubectl get jobs  
kubectl get pods  
kubectl logs job/hello-job
```

SHELL

25. Job Restart Policy

Jobs support:

```
restartPolicy: Never
```

YAML

or:

```
restartPolicy: OnFailure
```

YAML

They do not use Always.

For Jobs, the task should complete.

If it fails, Kubernetes may retry depending on Job settings.

26. Job completions and parallelism

Jobs can run multiple completions.

Example:

```
spec:  
  completions: 5  
  parallelism: 2
```

YAML

Meaning:

- Total successful completions needed: 5
- Run up to 2 Pods at a time

Use cases:

- Parallel data processing

- Batch workers
- Queue processing

27. Job backoffLimit

backoffLimit controls retry attempts.

Example:

```
spec:
  backoffLimit: 3
```

YAML

If the Job fails more than 3 times, Kubernetes marks it failed.

This prevents infinite retries.

28. What Is a CronJob?

A CronJob runs Jobs on a schedule.

SIMPLE DEFINITION

A CronJob is a scheduled Job.

Use cases:

- Nightly backups
- Hourly reports
- Daily cleanup
- Periodic sync
- Certificate checks
- Scheduled batch processing

29. CronJob Analogy

Think of an alarm clock.

A Job is one task.

A CronJob is a task that runs according to a schedule.

Example:

Run backup every day at 2 AM.

30. CronJob YAML Example

YAML

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello-cron
spec:
  schedule: "*/5 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: hello
              image: busybox:1.36
              command: ["sh", "-c", "date; echo cron job executed"]
```

This runs every 5 minutes.

Apply:

SHELL

```
kubectl apply -f cronjob.yaml
```

Check:

SHELL

```
kubectl get cronjobs
kubectl get jobs
```

31. Cron Schedule Format

Cron format:

The diagram shows the Cron schedule format: `* * * * *`. Brackets on the right side of the asterisks point to the following labels: minute (for the first asterisk), hour (for the second), day of month (for the third), month (for the fourth), and day of week (for the fifth).

Examples:

Schedule	Meaning
/ * * * *	Every 5 minutes
0 * * * *	Every hour
0 2 * * *	Every day at 2 AM
0 0 * * 0	Every Sunday midnight

32. CronJob concurrencyPolicy

concurrencyPolicy controls overlapping runs.

Values:

Policy	Meaning
Allow	Allow concurrent Jobs
Forbid	Skip new Job if previous still running
Replace	Replace running Job with new one

Example:

```
concurrencyPolicy: Forbid
```

YAML

Useful for backups or jobs that must not overlap.

33. CronJob history limits

Control how many old Jobs are kept:

```
successfulJobsHistoryLimit: 3
failedJobsHistoryLimit: 1
```

YAML

This prevents old Jobs from cluttering the cluster.

34. Workload Controller Comparison

Controller	Purpose	Runs Forever?	Stable Identity?
Deployment	Stateless apps	Yes	No
StatefulSet	Stateful apps	Yes	Yes
DaemonSet	One Pod per node	Yes	Node-based
Job	One-time task	No	No
CronJob	Scheduled task	No, scheduled	No

35. Troubleshooting StatefulSets

Problem: StatefulSet Pod stuck Pending

Check:

```
kubectl describe pod <pod>
kubectl get pvc
kubectl describe pvc <pvc>
```

SHELL

Common causes:

- PVC not bound
- StorageClass missing
- Zone mismatch
- Resource shortage
- Node selector issue

Problem: StatefulSet update stuck

Check:

```
kubectl rollout status statefulset/<name>
kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod>
```

SHELL

Common causes:

- New version not ready
- Readiness probe failing

- Storage mount issue
- Application cluster dependency issue

36. Troubleshooting DaemonSets

Check:

```
kubectl get daemonset
kubectl describe daemonset <name>
kubectl get pods -o wide
```

SHELL

Common causes:

- Node selector mismatch
- Taints not tolerated
- Image pull issue
- Resource shortage
- Pod security restrictions

37. Troubleshooting Jobs

Check:

```
kubectl get jobs
kubectl describe job <name>
kubectl get pods
kubectl logs job/<name>
```

SHELL

Common causes:

- Script error
- Image pull failure
- Restart policy issue
- Backoff limit reached
- Missing config/secret
- Permissions issue

38. Troubleshooting CronJobs

Check:


```
kubectl get cronjobs
kubectl describe cronjob <name>
kubectl get jobs
```

Common causes:

- Wrong schedule
- Suspended CronJob
- Concurrency policy blocking
- Timezone misunderstanding
- Failed Jobs
- Missing permissions

Check if suspended:

```
spec:
  suspend: true
```

39. Production Best Practices

StatefulSet

- Use StatefulSet only when stable identity/storage is required.
- Use Headless Service.
- Use volumeClaimTemplates.
- Back up persistent data.
- Understand update ordering.
- Test recovery.
- Avoid manual PVC deletion.

DaemonSet

- Use for node-level agents.
- Add tolerations carefully.
- Monitor node coverage.
- Set resources.
- Avoid unnecessary hostPath mounts.
- Harden security context.

Job

- Use for run-to-completion tasks.

- Set backoffLimit.
- Make jobs idempotent.
- Store outputs externally.
- Use TTL cleanup if appropriate.

CronJob

- Set concurrencyPolicy.
- Set history limits.
- Monitor failures.
- Be careful with timezone.
- Make scheduled tasks idempotent.

40. Common Mistakes

Mistake	Why Bad
Using Deployment for database cluster needing stable identity	Pod identity/storage not stable
Using StatefulSet for normal stateless API	Unnecessary complexity
Forgetting Headless Service	Stateful DNS may not work
Deleting StatefulSet PVCs accidentally	Data loss
Running node agents as Deployments	Not guaranteed one per node
Using CronJob without concurrency policy	Overlapping jobs
Jobs not idempotent	Retry causes duplicate work
No history limits	Too many old Jobs

41. Interview Questions and Answers

Beginner

1. What is StatefulSet?

A StatefulSet manages stateful applications that need stable identity and persistent storage.

2. What is DaemonSet?

A DaemonSet ensures one Pod runs on every selected node.

3. What is Job?

A Job runs a task until it completes successfully.

4. What is CronJob?

A CronJob runs Jobs on a schedule.

5. What is Headless Service?

A Service with clusterIP: None, commonly used for direct Pod DNS in StatefulSets.

Intermediate

1. StatefulSet vs Deployment?

Deployment is for stateless apps with replaceable Pods. StatefulSet is for stateful apps needing stable Pod names, storage, and ordered behavior.

2. Why do StatefulSets use volumeClaimTemplates?

To create stable per-Pod PVCs.

3. Give DaemonSet examples.

Log collectors, node exporters, CNI agents, storage node plugins, and security agents.

4. What does CronJob concurrencyPolicy do?

It controls whether scheduled Jobs can overlap.

5. What is backoffLimit in Job?

It controls how many times Kubernetes retries a failed Job before marking it failed.

Advanced

1. How does StatefulSet scale down?

It deletes highest ordinal Pods first.

2. Why are StatefulSet PVCs usually not deleted automatically?

To protect persistent data from accidental loss.

3. What happens when a new node joins and a DaemonSet exists?

Kubernetes schedules a DaemonSet Pod onto the new eligible node.

4. Why should Jobs be idempotent?

Because failed Jobs may retry, and retries should not corrupt data or duplicate unsafe actions.

5. Why can CronJobs miss schedules?

Controller downtime, suspended CronJob, concurrency rules, starting deadline, or cluster issues.

42. Scenario-Based Interview Questions

Scenario 1: Need One Log Agent Per Node

QUESTION You need one log collector on every worker node. Which object do you use?

EXPECTED ANSWER DaemonSet, because it ensures one Pod runs on every selected node.

Scenario 2: Database Needs Stable Pod Names

QUESTION A database cluster requires stable names like db-0, db-1, and persistent data per replica. Which object do you use?

EXPECTED ANSWER StatefulSet with Headless Service and volumeClaimTemplates.

Scenario 3: Nightly Backup

QUESTION You need to run backup every night at 2 AM. Which object do you use?

EXPECTED ANSWER CronJob with schedule 0 2 * * *.

Scenario 4: One-Time Migration

QUESTION You need to run a database migration once during deployment. Which object can you use?

EXPECTED ANSWER Job, because it runs to completion.

43. Hands-On Labs

Lab 1: Create Job

```
apiVersion: batch/v1
kind: Job
metadata:
  name: hello-job
spec:
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: hello
          image: busybox:1.36
          command: ["sh", "-c", "echo hello job"]
```

YAML

Apply:

```
kubectl apply -f job.yaml
kubectl logs job/hello-job
```

SHELL

Lab 2: Create CronJob

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello-cron
spec:
  schedule: "*/5 * * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: hello
              image: busybox:1.36
              command: ["sh", "-c", "date; echo hello cron"]
```

YAML

Apply:

```
kubectl apply -f cronjob.yaml
kubectl get cronjobs
kubectl get jobs
```

Lab 3: Create DaemonSet

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-printer
spec:
  selector:
    matchLabels:
      app: node-printer
  template:
    metadata:
      labels:
        app: node-printer
    spec:
      containers:
        - name: printer
          image: busybox:1.36
          command: ["sh", "-c", "while true; do hostname; sleep 30; done"]
```

Apply:

```
kubectl apply -f daemonset.yaml
kubectl get pods -o wide
```

44. Chapter Revision Notes

- Deployment is best for stateless applications.
- StatefulSet is best for stateful apps needing stable identity and storage.
- StatefulSet Pods have stable ordinal names.
- StatefulSets commonly use Headless Services.
- volumeClaimTemplates create per-Pod PVCs.
- DaemonSet runs one Pod per selected node.
- Jobs run tasks to completion.
- CronJobs run Jobs on schedules.
- Jobs should be idempotent.
- CronJobs should define concurrency and history policies.
- Choose workload controllers based on workload behavior.

45. Key Interview Statement

Memorize this:

KEY STATEMENT

Kubernetes provides different workload controllers for different workload types. Deployments manage stateless replicated apps, StatefulSets manage stateful apps with stable identity and storage, DaemonSets run one Pod per selected node, Jobs run tasks to completion, and CronJobs run Jobs on a schedule.

End of Volume 02 — Chapter 09

Next chapter:

Chapter 10 — Scheduling, Node Selectors, Affinity, Taints, Tolerations, Requests, Limits, and QoS

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

Scheduling, Affinity, Taints, Requests, Limits & QoS

Chapter 10 — controlling Pod placement and
resource behavior across the cluster.

👑 Chapter 10

[Scheduler](#) · [Affinity](#) · [Taints](#) · [Requests](#) · [QoS](#)

Chapter Purpose

Kubernetes does not randomly place Pods on nodes.

When a Pod is created, the Kubernetes Scheduler decides which node should run it.

This chapter explains how scheduling works and how engineers control Pod placement using node selectors, affinity, taints, tolerations, resource requests, limits, and QoS classes.

These concepts are very important in production because wrong scheduling decisions can cause:

- Pending Pods
- Poor performance
- Expensive nodes being wasted
- Critical workloads running on wrong nodes
- GPU workloads scheduled incorrectly
- Apps running without enough CPU or memory
- Node pressure and evictions

Learning Objectives

By the end of this chapter, you should be able to explain:

- What Kubernetes scheduling is.
- How the Scheduler chooses nodes.
- What node selectors are.
- What node affinity is.
- What pod affinity is.
- What pod anti-affinity is.
- What taints and tolerations are.
- What requests and limits are.
- How Kubernetes uses requests for scheduling.
- What QoS classes are.
- Why Pods become Pending.
- Why Pods are evicted.
- Production scheduling best practices.
- Interview answers.

1. What Is Kubernetes Scheduling?

Scheduling is the process of assigning a Pod to a worker node.

When you create a Pod, it first exists without a node.

The Scheduler watches for unscheduled Pods and chooses the best node.

SIMPLE DEFINITION

Kubernetes scheduling means deciding which node should run a Pod.

2. Why Scheduling Is Needed

In a cluster, there may be many nodes.

Example:

```
node-1: 4 CPU, 16Gi memory  
node-2: 8 CPU, 32Gi memory  
node-3: GPU node  
node-4: spot instance  
node-5: database node
```

Pods have different needs.

Example:

```
frontend Pod → small CPU/memory  
api Pod → medium CPU/memory  
ml Pod → GPU  
database Pod → storage optimized node  
critical Pod → on-demand node only
```

Scheduler matches Pods to nodes based on requirements and policies.

3. Real-World Analogy

Think of a hotel receptionist assigning rooms.

Guests have needs:

- Single room
- Family room
- Quiet floor
- Accessible room
- Sea view
- Non-smoking
- Premium suite

Rooms have properties.

The receptionist assigns each guest to a suitable room.

Kubernetes Scheduler does the same for Pods and nodes.

Kubernetes Concept	Hotel Analogy
Pod	Guest
Node	Hotel room
Scheduler	Receptionist
Node labels	Room features
Node selector	Required room feature
Affinity	Preference/requirement
Taint	Room restriction
Toleration	Guest permission
Request	Minimum space needed
Limit	Maximum allowed usage

4. Scheduler High-Level Flow

```
Pod created
|
v
Pod has no nodeName
|
v
Scheduler watches Pod
|
v
Filter unsuitable nodes
|
v
Score suitable nodes
|
v
Choose best node
|
v
Bind Pod to node
|
v
kubelet starts Pod
```

5. Filtering and Scoring

The Scheduler works in two important stages:

Filtering → Scoring

Filtering

Filtering removes nodes that cannot run the Pod.

Reasons:

- Not enough CPU
- Not enough memory
- Node selector mismatch
- Required affinity not met
- Taint not tolerated
- Volume zone mismatch
- Node not ready

Scoring

Scoring ranks the remaining nodes.

Examples:

- Prefer less loaded nodes.
- Prefer nodes with matching preferred affinity.
- Prefer topology spreading.
- Prefer balanced resource usage.

6. Node Labels

Labels are key-value pairs attached to nodes.

Example:

```
kubectl label node node-1 disktype=ssd
kubectl label node node-2 lifecycle=spot
kubectl label node node-3 accelerator=gpu
```

SHELL

Check labels:

```
kubectl get nodes --show-labels
```

SHELL

Labels describe node properties.

Examples:

```
instance-type=m6i.large
zone=us-east-1a
lifecycle=spot
workload=frontend
disktype=ssd
accelerator=gpu
```

7. Node Selector

nodeSelector is the simplest way to force a Pod onto nodes with specific labels.

Example:

```
spec:
  nodeSelector:
    disktype: ssd
```

YAML

This Pod can only run on nodes with:

```
disktype=ssd
```

8. Node Selector Example

Label node:

```
kubectl label node node-1 disktype=ssd
```

SHELL

Pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: ssd-pod
spec:
  nodeSelector:
    disktype: ssd
  containers:
    - name: app
      image: nginx:1.27
```

YAML

Apply:

```
kubectl apply -f pod.yaml
```

SHELL

Check:

```
kubectl get pod ssd-pod -o wide
```

SHELL

The Pod should run on a node labeled disktype=ssd.

9. Node Selector Limitation

nodeSelector is simple but limited.

It only supports exact key-value matching.

Example:

```
nodeSelector:  
  disktype: ssd
```

YAML

It cannot easily express:

Run on nodes in us-east-1a OR us-east-1b.

Prefer spot but allow on-demand.

Avoid GPU nodes.

Require instance type in a list.

For advanced rules, use affinity.

10. Node Affinity

Node affinity is a more powerful way to control Pod placement based on node labels.

It supports:

- Required rules
- Preferred rules
- Multiple expressions
- Operators like In, NotIn, Exists, DoesNotExist

SIMPLE DEFINITION

Node affinity controls which nodes a Pod can or should run on using flexible label rules.

11. Required Node Affinity

Required node affinity is mandatory.

If no node matches, Pod stays Pending.

Example:

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - key: disktype
              operator: In
              values:
                - ssd
```

YAML

Meaning:

Pod must run on a node where disktype is ssd.

12. Preferred Node Affinity

Preferred node affinity is a soft preference.

If matching node exists, Scheduler prefers it.

If not, Pod can still run elsewhere.

Example:

```
affinity:
  nodeAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 80
        preference:
          matchExpressions:
            - key: lifecycle
              operator: In
              values:
                - spot
```

YAML

Meaning:

Prefer spot nodes, but allow other nodes if needed.

13. Node Affinity Operators

Operator	Meaning
In	Label value is in list
NotIn	Label value is not in list
Exists	Label key exists
DoesNotExist	Label key does not exist
Gt	Greater than
Lt	Less than

14. Node Affinity vs Node Selector

Feature	nodeSelector	nodeAffinity
Simplicity	Very simple	More complex
Operators	Exact match only	Multiple operators
Required rules	Yes	Yes
Preferred rules	No	Yes
Use case	Simple placement	Advanced scheduling

15. Pod Affinity

Pod affinity schedules Pods near other Pods.

Example use case:

Run app Pod on same node/zone as cache Pod.

SIMPLE DEFINITION Pod affinity attracts Pods toward other Pods based on labels.

16. Pod Anti-Affinity

Pod anti-affinity schedules Pods away from other Pods.

Example use case:

Do not place all replicas of same app on the same node.

SIMPLE DEFINITION Pod anti-affinity spreads Pods away from matching Pods.

This improves high availability.

17. Pod Anti-Affinity Example

YAML

```
affinity:
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchLabels:
              app: api
          topologyKey: kubernetes.io/hostname
```

Meaning:

Prefer not to schedule this Pod on a node that already has a Pod labeled app=api.

This helps spread replicas across nodes.

18. topologyKey

topologyKey defines the domain for affinity/anti-affinity.

Common values:

topologyKey	Meaning
kubernetes.io/hostname	Node-level spread
topology.kubernetes.io/zone	Zone-level spread
topology.kubernetes.io/region	Region-level spread

Example:

YAML

```
topologyKey: kubernetes.io/hostname
```

means spread across nodes.

19. Taints and Tolerations

Taints and tolerations are used to repel Pods from nodes unless the Pods explicitly tolerate the taint.

SIMPLE DEFINITION

Taints are applied to nodes. Tolerations are applied to Pods. A Pod can run on a tainted node only if it tolerates the taint.

20. Real-World Analogy for Taints

Think of a restricted room.

A room has a sign:

Authorized staff only.

That is a taint.

Only people with authorization can enter.

That authorization is a toleration.

21. Taint Example

Taint a node:

```
kubectl taint nodes node-1 dedicated=database:NoSchedule
```

SHELL

Meaning:

Do not schedule Pods on node-1 unless they tolerate `dedicated=database`.

22. Toleration Example

Pod toleration:

```
tolerations:
- key: dedicated
  operator: Equal
  value: database
  effect: NoSchedule
```

YAML

This Pod can be scheduled on nodes tainted:

```
dedicated=database:NoSchedule
```

IMPORTANT

Toleration allows scheduling on tainted node, but does not force scheduling there.

To force placement, combine toleration with `nodeSelector` or `nodeAffinity`.

23. Taint Effects

Effect	Meaning
NoSchedule	Do not schedule new Pods unless tolerated
PreferNoSchedule	Try to avoid scheduling unless tolerated
NoExecute	Evict existing Pods unless tolerated

24. Taints vs Node Affinity

Feature	Taints/Tolerations	Node Affinity
Behavior	Repel Pods	Attract Pods
Applied to	Node taint, Pod toleration	Pod rules using node labels
Main use	Reserve nodes	Select/prefer nodes
Force placement?	No, only allows	Yes if required

Example:

To reserve database nodes:

Taint database nodes.

Add toleration to database Pods.

Add `nodeAffinity` to database Pods.

25. Resource Requests

Requests define the minimum resources a Pod needs.

Example:

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
```

YAML

Scheduler uses requests to decide whether a node has enough capacity.

IMPORTANT Requests affect scheduling.

26. CPU Units

CPU can be expressed as cores or millicores.

Examples:

CPU Value	Meaning
1	1 CPU core
500m	0.5 CPU
250m	0.25 CPU
100m	0.1 CPU

m means millicpu.

1000m = 1 CPU

27. Memory Units

Memory examples:

Value	Meaning
128Mi	128 mebibytes
512Mi	512 mebibytes
1Gi	1 gibibyte
2Gi	2 gibibytes

Use Mi and Gi commonly.

28. Resource Limits

Limits define the maximum resources a container can use.

Example:

```
resources:
  limits:
    cpu: "500m"
    memory: "512Mi"
```

YAML

Important:

- CPU over limit is throttled.
- Memory over limit can cause OOMKilled.

29. Requests vs Limits

Concept	Used For	Effect
Request	Scheduling	Reserves capacity for placement
Limit	Runtime maximum	Restricts usage

Example:

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

YAML

Meaning:

Needs at least 250m CPU and 256Mi memory for scheduling.

Can use up to 500m CPU and 512Mi memory at runtime.

30. Why Requests Are Critical

If Pods have no requests, Scheduler cannot make good placement decisions.

Problems:

- Too many Pods on one node

- Resource contention
- Poor performance
- Node pressure
- Evictions
- Unpredictable scheduling
- Cost inefficiency

Production workloads should define resource requests.

31. CPU Throttling

If a container exceeds CPU limit, it is throttled.

CPU throttling means the app is slowed down, not killed.

Symptoms:

- High latency
- Slow request handling
- Timeouts
- Low throughput
- CPU throttling metrics high

Sometimes CPU limits are harmful for latency-sensitive apps if set too low.

32. Memory OOMKilled

If a container exceeds memory limit, it may be killed.

Status:

OOMKilled

Check:

```
kubectl describe pod <pod>
kubectl top pod <pod>
```

SHELL

Fix:

- Increase memory limit
- Fix memory leak
- Tune app memory
- Reduce worker count
- Improve memory profiling

33. QoS Classes

Kubernetes assigns QoS classes based on resource requests and limits.

QoS classes:

QoS Class	Meaning
Guaranteed	Highest priority for resource stability
Burstable	Some requests/limits set
BestEffort	No requests/limits

QoS affects eviction priority under node pressure.

34. Guaranteed QoS

A Pod gets Guaranteed QoS when every container has CPU and memory request equal to limit.

Example:

```
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

YAML

Best for critical workloads that need predictable resources.

35. Burstable QoS

A Pod gets Burstable QoS when it has some requests/limits, but not equal for all resources.

Example:

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "1000m"
    memory: "1Gi"
```

YAML

Most production workloads are Burstable.

36. BestEffort QoS

A Pod gets BestEffort QoS when no requests or limits are set.

Example:

```
resources: {}
```

YAML

BestEffort Pods are first candidates for eviction during node pressure.

Avoid for important production workloads.

37. Eviction

Eviction means Kubernetes removes Pods from a node due to pressure.

Common pressure types:

- MemoryPressure
- DiskPressure
- PIDPressure

Eviction priority often follows:

```
BestEffort evicted first  
Burstable next  
Guaranteed last
```

38. LimitRange

LimitRange sets default/min/max resource values in a namespace.

Example:


```

apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
spec:
  limits:
    - type: Container
      default:
        cpu: "500m"
        memory: "512Mi"
      defaultRequest:
        cpu: "250m"
        memory: "256Mi"

```

This helps enforce resource hygiene.

39. ResourceQuota

ResourceQuota limits total resource usage in a namespace.

Example:

```

apiVersion: v1
kind: ResourceQuota
metadata:
  name: namespace-quota
spec:
  hard:
    requests.cpu: "10"
    requests.memory: "20Gi"
    limits.cpu: "20"
    limits.memory: "40Gi"
    pods: "50"

```

Use for multi-team clusters.

40. Pod Priority and Preemption

PriorityClass allows important Pods to have higher priority.

If resources are scarce, Kubernetes may preempt lower-priority Pods.

Example:

YAML

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: critical-apps
value: 100000
globalDefault: false
description: "Priority for critical application pods"
```

Use carefully.

41. Common Pending Pod Causes

Cause	Explanation
Insufficient CPU	Requests exceed available CPU
Insufficient memory	Requests exceed available memory
Taint not tolerated	Node taint blocks Pod
Node selector mismatch	No matching node labels
Required affinity mismatch	No eligible nodes
PVC unbound	Storage not ready
Zone mismatch	Volume/node zone conflict
Quota exceeded	Namespace resource quota hit

42. Troubleshooting Pending Pods

Commands:

SHELL

```
kubectl describe pod <pod>
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl get nodes
kubectl describe node <node>
kubectl get quota
kubectl get pvc
```

Most scheduling failures are shown in Pod events.

Look for messages like:

0/5 nodes are available: insufficient memory.
0/5 nodes are available: node(s) had intolerated taint.

43. Troubleshooting OOMKilled

Commands:

```
kubectl describe pod <pod>
kubectl top pod <pod>
kubectl logs <pod> --previous
```

SHELL

Check:

- Memory limit
- Application memory usage
- Recent config changes
- Traffic spike
- Memory leak
- JVM/Node/Python worker settings

44. Troubleshooting CPU Throttling

Symptoms:

- Slow app
- Timeouts
- High latency
- CPU usage near limit
- Throttling metrics

Fix:

- Increase CPU limit
- Remove CPU limit carefully for some workloads
- Tune app concurrency
- Scale replicas
- Profile CPU usage

45. Production Best Practices

- Always set resource requests for production Pods.

- Set memory limits carefully.
- Avoid overly low CPU limits for latency-sensitive apps.
- Use node affinity for workload placement.
- Use taints/tolerations to reserve special nodes.
- Combine tolerations with affinity when targeting dedicated nodes.
- Use pod anti-affinity or topology spread for high availability.
- Use LimitRanges to enforce defaults.
- Use ResourceQuotas for namespaces.
- Monitor CPU throttling and OOMKilled events.
- Use priority classes only when needed.
- Label nodes consistently.
- Avoid hardcoding node names unless absolutely necessary.

46. Common Mistakes

Mistake	Why Bad
No resource requests	Bad scheduling
No memory limits	Risk node memory pressure
Very low CPU limits	Throttling and latency
Toleration without affinity	Pod may run elsewhere
Hardcoding nodeName	Bypasses scheduler
Using nodeSelector for complex rules	Too limited
Ignoring Pod events	Misses scheduling reason
No anti-affinity	Replicas may land on same node

47. Interview Questions and Answers

Beginner

1. What is Kubernetes scheduling?

Scheduling is assigning a Pod to a suitable worker node.

2. What is nodeSelector?

A simple way to schedule Pods only on nodes with matching labels.

3. What are taints and tolerations?

Taints repel Pods from nodes. Tolerations allow Pods to run on tainted nodes.

4. What are resource requests?

Minimum CPU/memory required by a container and used for scheduling.

5. What are resource limits?

Maximum CPU/memory a container can use at runtime.

Intermediate

1. Difference between requests and limits?

Requests are used for scheduling. Limits restrict runtime maximum usage.

2. What is node affinity?

A flexible way to schedule Pods based on node labels using required or preferred rules.

3. What is pod anti-affinity?

A rule that spreads Pods away from other matching Pods.

4. Why does a Pod stay Pending?

No suitable node due to resources, taints, affinity, PVC, quota, or node conditions.

5. What happens when memory limit is exceeded?

The container may be killed with OOMKilled.

Advanced

1. What are QoS classes?

QoS classes are Guaranteed, Burstable, and BestEffort, based on requests and limits. They influence eviction priority.

2. Why can CPU limits cause latency?

CPU over the limit is throttled, which can slow request processing and cause timeouts.

3. Why use WaitForFirstConsumer with storage?

It helps provision zonal storage in the same zone where the Pod is scheduled.

4. Why combine taints/tolerations with node affinity?

Toleration only allows scheduling on tainted nodes; affinity attracts the Pod to those nodes.

5. What is preemption?

Kubernetes may evict lower-priority Pods to schedule a higher-priority Pod.

48. Scenario-Based Interview Questions

Scenario 1: Pod Pending Due to Taint

QUESTION Pod is Pending and event says node had untolerated taint.

EXPECTED ANSWER Add a matching toleration to the Pod or remove the taint if inappropriate.

Scenario 2: Critical Pods on Dedicated Nodes

QUESTION You need critical workloads to run only on dedicated nodes and prevent other workloads from running there.

EXPECTED ANSWER Label dedicated nodes, taint them, add nodeAffinity and matching tolerations to critical workloads.

Scenario 3: API Pods All on Same Node

QUESTION Three replicas of an API are all scheduled on one node. How do you improve availability?

EXPECTED ANSWER Use pod anti-affinity or topology spread constraints to spread replicas across nodes or zones.

Scenario 4: App OOMKilled

QUESTION Pod is restarting with OOMKilled.

EXPECTED ANSWER Check memory usage, memory limit, logs, traffic changes, memory leaks, and increase/tune memory settings if needed.

49. Hands-On Labs

Lab 1: Node Selector

```
kubectl label node <node-name> disktype=ssd
```

SHELL

Pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: node-selector-demo
spec:
  nodeSelector:
    disktype: ssd
  containers:
    - name: nginx
      image: nginx:1.27
```

YAML

Lab 2: Taint and Toleration

Taint node:

```
kubectl taint nodes <node-name> dedicated=test:NoSchedule
```

SHELL

Pod toleration:

```
tolerations:
- key: dedicated
  operator: Equal
  value: test
  effect: NoSchedule
```

YAML

Lab 3: Requests and Limits

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-demo
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      resources:
        requests:
          cpu: "100m"
          memory: "128Mi"
        limits:
          cpu: "250m"
          memory: "256Mi"
```

YAML

Check:

```
kubectl describe pod resource-demo
```

SHELL

50. Chapter Revision Notes

- Scheduler assigns Pods to nodes.
- Scheduler filters unsuitable nodes and scores suitable nodes.
- Node labels describe nodes.
- nodeSelector is simple exact-match placement.
- Node affinity is more flexible.
- Pod anti-affinity helps spread replicas.
- Taints repel Pods from nodes.
- Tolerations allow Pods onto tainted nodes.
- Requests are used for scheduling.
- Limits restrict runtime usage.
- CPU over limit is throttled.
- Memory over limit may cause OOMKilled.
- QoS classes influence eviction priority.
- Pending Pods are usually explained in events.

51. Key Interview Statement

Memorize this:

KEY STATEMENT

Kubernetes scheduling assigns Pods to suitable nodes by filtering and scoring nodes. Resource requests drive scheduling decisions, limits control runtime usage, node selectors and affinity attract Pods to specific nodes, taints repel Pods from nodes unless tolerated, and QoS classes influence eviction behavior under node pressure.

End of Volume 02 — Chapter 10

Next chapter:

Chapter 11 — Autoscaling: HPA, VPA, Cluster Autoscaler, Karpenter, and Production Scaling

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

Autoscaling: HPA, VPA, Cluster Autoscaler & Karpenter

Chapter 11 — automatically adjusting capacity with HPA, VPA, Cluster Autoscaler, and Karpenter.

👑 Chapter 11

HPA · VPA · Cluster Autoscaler · Karpenter

Chapter Purpose

Autoscaling allows Kubernetes platforms to automatically adjust capacity based on demand.

Scaling can happen at different levels:

- Application replicas
- Pod resource recommendations
- Cluster node capacity
- Cloud infrastructure capacity

This chapter explains Kubernetes autoscaling from practical production and interview perspective.

1. What Is Autoscaling?

Autoscaling means automatically increasing or decreasing resources based on workload demand.

SIMPLE DEFINITION Autoscaling helps Kubernetes run enough capacity when traffic increases and reduce capacity when demand decreases.

2. Why Autoscaling Is Needed

Without autoscaling:

- Too few Pods can cause downtime.
- Too many Pods waste money.
- Traffic spikes can overload applications.
- Manual scaling is slow.
- Teams may overprovision permanently.

Autoscaling helps balance:

Reliability + Performance + Cost

3. Types of Kubernetes Autoscaling

Autoscaler	What It Scales
HPA	Number of Pod replicas
VPA	Pod CPU/memory requests
Cluster Autoscaler	Number of nodes
Karpenter	Cloud nodes/provisioning capacity
Custom autoscalers	App-specific scaling

4. Real-World Analogy

Think of a restaurant.

Kubernetes Autoscaling	Restaurant Analogy
HPA	Add more waiters when customers increase
VPA	Give each waiter better tools/capacity
Cluster Autoscaler	Open more dining rooms
Karpenter	Rent the best room type quickly based on demand

5. Horizontal Pod Autoscaler

HPA scales the number of Pod replicas.

SIMPLE DEFINITION HPA increases or decreases replicas based on metrics such as CPU, memory, or custom metrics.

Example:

```
Current replicas: 2
Traffic increases
CPU goes high
HPA scales to 5 replicas
```

6. HPA Architecture

```
Metrics Server / Custom Metrics
  |
  v
Horizontal Pod Autoscaler
  |
  v
Deployment / ReplicaSet
  |
  v
More or fewer Pods
```

HPA does not create nodes directly.

It changes replica count.

7. HPA Basic YAML

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

YAML

8. HPA Explanation

```
scaleTargetRef:
```

YAML

Defines what object HPA scales.

```
minReplicas: 2
maxReplicas: 10
```

YAML

Sets lower and upper replica boundaries.

```
averageUtilization: 70
```

YAML

Target average CPU utilization is 70% of requested CPU.

IMPORTANT

HPA CPU percentage is based on CPU requests, not CPU limits.

9. HPA Requirements

HPA usually needs:

- Metrics Server installed.
- Resource requests configured.
- A scalable target like Deployment.
- Correct metrics API availability.

Check metrics:

```
kubectl top pods
kubectl top nodes
```

SHELL

If these fail, HPA may not work.

10. Metrics Server

Metrics Server collects basic CPU and memory metrics from kubelets.

It supports:

- `kubectl top`
- CPU/memory-based HPA

It is not a full monitoring system.

For long-term metrics, use tools like Prometheus, Datadog, CloudWatch, or Grafana.

11. HPA With Memory

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-memory-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  minReplicas: 2
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 75
```

YAML

Memory autoscaling can be useful, but memory does not always decrease quickly after load drops.

12. HPA With Multiple Metrics

```
metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80
```

YAML

When multiple metrics are used, HPA calculates desired replicas for each metric and chooses the highest replica count.

13. Custom Metrics

HPA can scale using custom metrics.

Examples:

- Requests per second
- Queue depth
- Kafka lag
- SQS queue length
- Active sessions
- HTTP latency

This requires custom metrics adapters such as Prometheus Adapter or KEDA.

14. KEDA

KEDA stands for Kubernetes Event-driven Autoscaling.

It can scale workloads based on event sources.

Examples:

- AWS SQS
- Kafka
- RabbitMQ
- Redis
- Prometheus
- Azure Queue
- Cron schedule

KEDA is useful for event-driven workloads and queue-based workers.

15. HPA Production Best Practices

- Always set CPU/memory requests.
- Set sensible minReplicas.
- Set safe maxReplicas.
- Use readiness probes.
- Monitor scaling behavior.
- Avoid scaling on noisy metrics.
- Use custom metrics for queue workers.
- Ensure cluster has node autoscaling.
- Load test before production.
- Avoid very aggressive scale-down.
- Do not use HPA as a replacement for app performance tuning.

16. Common HPA Problems

Problem	Cause
HPA unknown metrics	Metrics Server issue
No scaling	Missing requests
Too much scaling	Target too low
Slow scale-up	Stabilization/window behavior
Pods Pending after scale	No node capacity
CPU high but no scale	Requests missing or metrics unavailable

17. Vertical Pod Autoscaler

VPA recommends or adjusts CPU and memory requests for Pods.

SIMPLE DEFINITION VPA helps right-size Pod resource requests.

VPA can:

- Recommend resources.
- Automatically update requests.
- Evict Pods to apply new recommendations.

18. Why VPA Exists

Many teams guess resource requests.

Bad guesses cause problems:

Bad Request	Problem
Too low	Poor scheduling, throttling, OOM
Too high	Wasted capacity and cost
Missing	Unreliable scheduling

VPA analyzes usage and recommends better values.

19. VPA Modes

Mode	Meaning
Off	Only recommendations
Initial	Set requests only when Pod starts
Auto	Update requests, may evict Pods
Recreate	Similar to Auto behavior depending implementation

Production teams often start with recommendation mode.

20. VPA Example

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: api-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api
  updatePolicy:
    updateMode: "Off"
```

YAML

This gives recommendations without modifying Pods.

21. HPA vs VPA

Feature	HPA	VPA
Scales	Replica count	CPU/memory requests
Best for	Traffic changes	Right-sizing
Disruption	Usually less	May evict Pods
Common use	Web/API/worker scale	Resource tuning
Works together?	Carefully	Carefully

IMPORTANT HPA and VPA can conflict if both control CPU/memory-based scaling on the same workload.

22. Cluster Autoscaler

Cluster Autoscaler adjusts the number of worker nodes.

SIMPLE DEFINITION Cluster Autoscaler adds nodes when Pods cannot schedule and removes underused nodes.

It watches for Pending Pods caused by insufficient capacity.

23. Cluster Autoscaler Flow

```
Pod Pending due to insufficient resources
  |
  v
Cluster Autoscaler detects unschedulable Pod
  |
  v
Increases node group size
  |
  v
New node joins cluster
  |
  v
Scheduler places Pod
```

24. Scale Down Flow

```
Node underutilized
  |
  v
Cluster Autoscaler checks if Pods can move
  |
  v
Evicts/moves Pods safely
  |
  v
Removes node
```

Scale-down must respect:

- PodDisruptionBudgets
- Local storage

- System Pods
- Safe eviction rules
- Node group min size

25. HPA and Cluster Autoscaler Together

Common production pattern:

```
graph TD; A[Traffic increases] --> B[HPA creates more Pods]; B --> C[Some Pods become Pending]; C --> D[Cluster Autoscaler adds nodes]; D --> E[Pending Pods get scheduled];
```

HPA scales applications.

Cluster Autoscaler scales infrastructure.

26. Karpenter

Karpenter is a Kubernetes node provisioning system commonly used on AWS EKS.

SIMPLE DEFINITION Karpenter provisions right-sized nodes based on unscheduled Pods.

It can choose instance types dynamically instead of only scaling fixed node groups.

27. Karpenter vs Cluster Autoscaler

Feature	Cluster Autoscaler	Karpenter
Scaling model	Scales existing node groups	Provisions nodes directly
Flexibility	Lower	Higher
Instance selection	Node group based	Dynamic
Speed	Good	Often faster
Cost optimization	Moderate	Stronger options
Common use	Managed node groups	EKS flexible provisioning

28. Karpenter Flow

```
Pod cannot schedule
  |
  v
Karpenter evaluates Pod requirements
  |
  v
Chooses suitable instance type/capacity
  |
  v
Launches node
  |
  v
Pod schedules on new node
```

Karpenter can consider:

- CPU/memory needs
- Architecture
- zones
- instance families
- capacity type
- taints
- labels
- cost

29. Autoscaling and Requests

Autoscaling depends heavily on correct resource requests.

If requests are too low:

- Too many Pods may schedule on a node.
- CPU utilization percentage may look high.
- HPA may scale unexpectedly.
- Performance may suffer.

If requests are too high:

- Pods may stay Pending.
- Cluster may over-scale.
- Costs increase.

Right-sizing requests is critical.

30. Scaling Stateless vs Stateful Apps

Stateless apps are easier to scale.

Examples:

- APIs
- frontends
- workers
- web apps

Stateful apps are harder because they may need:

- Stable storage
- Data replication
- Leader election
- Ordered scaling
- Quorum rules
- Manual safety checks

Do not blindly autoscale databases without understanding the application.

31. Scale-Up vs Scale-Down

Scale-up should usually be faster than scale-down.

Why?

- Scale-up protects availability.
- Scale-down saves cost but can be risky.
- Too-fast scale-down may cause flapping.

Good systems scale up quickly and scale down gradually.

32. PodDisruptionBudget

A PodDisruptionBudget, or PDB, protects availability during voluntary disruptions.

Example:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: api
```

YAML

This tells Kubernetes:

At least 2 API Pods should remain available during voluntary disruption.

Autoscaler scale-down respects PDBs.

33. Common Autoscaling Failure Modes

Failure	Explanation
HPA scales but Pods Pending	No node capacity
Cluster Autoscaler not scaling	Pod not considered schedulable by new nodes
Metrics unknown	Metrics Server/custom metrics issue
Too many replicas	Bad target metric
Cost spike	Max replicas too high
OOM after scale	Per-Pod memory too low
Slow traffic recovery	Scale-up too slow
Scale flapping	Metrics noisy or windows too aggressive

34. Troubleshooting HPA

Commands:

```
kubectl get hpa
kubectl describe hpa <name>
kubectl top pods
kubectl top nodes
kubectl describe deployment <name>
```

Check:

- Metrics available?
- Requests set?
- Target reference correct?
- min/max replicas correct?
- Events?
- Pods healthy?
- Custom metrics adapter working?

35. Troubleshooting Pending Pods After HPA

Commands:

```
kubectl get pods
kubectl describe pod <pending-pod>
kubectl get nodes
kubectl describe nodes
```

Common causes:

- Insufficient CPU/memory
- Node selector mismatch
- Taints not tolerated
- PVC issue
- Quota exceeded
- Autoscaler not configured
- Max node group size reached

36. Troubleshooting Cluster Autoscaler

Check autoscaler logs:

```
kubectl logs -n kube-system deployment/cluster-autoscaler
```


Check node groups:

```
kubectl get nodes
kubectl describe pod <pending-pod>
```

SHELL

Common causes:

- Max node group size reached.
- IAM/cloud permission issue.
- Pod requests too large.
- Taints/affinity impossible.
- PDB blocks scale-down.
- Pods use local storage.
- Autoscaler cannot find matching node group.

37. Troubleshooting Karpenter

Common checks:

```
kubectl get nodes
kubectl get pods
kubectl get events
kubectl logs -n karpenter deployment/karpenter
kubectl get nodepools
kubectl get nodeclaims
```

SHELL

Common causes:

- IAM permissions issue.
- Subnet/security group discovery issue.
- NodePool constraints too strict.
- Instance type unavailable.
- Capacity unavailable.
- Pod requirements impossible.
- AMI/bootstrap issue.

38. Production Best Practices

- Use HPA for stateless services.
- Use custom metrics for queue workers.
- Set requests correctly.
- Use VPA initially in recommendation mode.

- Use Cluster Autoscaler or Karpenter with HPA.
- Set min/max replicas carefully.
- Use PDBs for important workloads.
- Avoid autoscaling critical databases blindly.
- Monitor scaling events.
- Load test scaling behavior.
- Protect against cost explosions.
- Use separate node pools for special workloads.
- Prefer gradual scale-down.
- Document scaling assumptions.

39. Common Mistakes

Mistake	Why Bad
HPA without requests	CPU utilization cannot work correctly
Max replicas too low	Cannot handle traffic spike
Max replicas too high	Cost or dependency overload
No cluster autoscaler	HPA Pods remain Pending
Scaling on bad metric	Flapping or wrong behavior
VPA auto mode on critical app without testing	Unexpected evictions
Autoscaling stateful databases blindly	Data/quorum risk
No PDB	Scale-down/maintenance may reduce availability

40. Interview Questions and Answers

Beginner

1. What is HPA?

HPA scales the number of Pod replicas based on metrics.

2. What is VPA?

VPA recommends or updates Pod CPU/memory requests.

3. What is Cluster Autoscaler?

It adds or removes worker nodes based on scheduling needs and utilization.

4. What is Karpenter?

Karpenter dynamically provisions right-sized nodes for unscheduled Pods, commonly in EKS.

5. What is Metrics Server?

Metrics Server provides CPU and memory metrics used by kubectl top and basic HPA.

Intermediate

1. Difference between HPA and VPA?

HPA changes replica count. VPA changes resource requests.

2. Why does HPA need resource requests?

CPU utilization percentage is calculated relative to CPU requests.

3. Why can HPA-created Pods stay Pending?

The cluster may not have enough node capacity or scheduling constraints cannot be satisfied.

4. How do HPA and Cluster Autoscaler work together?

HPA creates more Pods; Cluster Autoscaler adds nodes if those Pods cannot schedule.

5. Why use custom metrics?

CPU/memory may not reflect actual business demand. Queue depth or request rate may scale better.

Advanced

1. Why can VPA conflict with HPA?

If HPA scales on CPU utilization and VPA changes CPU requests, the HPA calculation changes.

2. Why is Karpenter more flexible than Cluster Autoscaler?

Karpenter can provision nodes directly based on Pod requirements instead of only scaling existing node groups.

3. Why should scale-down be conservative?

Fast scale-down can cause instability, cold starts, and repeated scale-up/scale-down flapping.

4. How do PDBs affect autoscaler scale-down?

Autoscaler respects PDBs and may avoid removing nodes if doing so would violate availability requirements.

5. What causes autoscaling cost explosions?

Bad metrics, high max replicas, dependency failures, retry storms, or missing limits can cause excessive scaling.

41. Scenario-Based Interview Questions

Scenario 1: HPA Not Scaling

QUESTION CPU is high, but HPA is not scaling. What do you check?

EXPECTED ANSWER Check Metrics Server, `kubectl top`, HPA target, CPU requests, HPA events, min/max replicas, and whether metrics are available.

Scenario 2: Pods Pending After Scale-Up

QUESTION HPA increased replicas, but new Pods are Pending.

EXPECTED ANSWER Check resource availability, Pod events, node constraints, taints/tolerations, PVCs, quotas, and whether Cluster Autoscaler/Karpenter is working.

Scenario 3: Cluster Autoscaler Not Adding Nodes

QUESTION Pods are Pending but Cluster Autoscaler does not add nodes.

EXPECTED ANSWER Check autoscaler logs, max node group size, cloud permissions, pod requirements, node selectors, affinity, taints, and whether any node group can satisfy the Pod.

Scenario 4: Queue Worker Scaling

QUESTION You have workers processing SQS/Kafka messages. CPU is low but queue is growing. What scaling metric should you use?

EXPECTED ANSWER Use custom/event-driven metrics such as queue length, Kafka lag, or KEDA scaler instead of CPU.

42. Hands-On Labs

Lab 1: Create HPA

```
kubectl autoscale deployment api --cpu-percent=70 --min=2 --max=10
kubectl get hpa
```

SHELL

Lab 2: Inspect HPA

```
kubectl describe hpa api
kubectl top pods
```

SHELL

Lab 3: Add Requests to Deployment

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

YAML

Lab 4: Watch Scaling

```
kubectl get hpa -w
kubectl get pods -w
```

SHELL

43. Chapter Revision Notes

- HPA scales Pod replicas.
- VPA recommends or updates CPU/memory requests.
- Cluster Autoscaler scales node count.
- Karpenter provisions right-sized nodes dynamically.
- HPA needs metrics and resource requests.
- CPU utilization is based on CPU requests.
- HPA and Cluster Autoscaler often work together.
- Custom metrics are best for queue/event-driven workloads.
- VPA should be introduced carefully.
- PDBs protect availability during scale-down.

- Correct resource requests are the foundation of good autoscaling.

44. Key Interview Statement

Memorize this:

KEY STATEMENT

Kubernetes autoscaling happens at multiple layers. HPA scales replicas based on metrics, VPA right-sizes Pod resource requests, and Cluster Autoscaler or Karpenter scales node capacity when Pods cannot schedule. Reliable autoscaling depends on accurate resource requests, healthy metrics pipelines, safe min/max limits, readiness probes, and careful production testing.

End of Volume 02 — Chapter 11

Next chapter:

Chapter 12 — RBAC, Service Accounts, Authentication, Authorization, Admission Controllers, and Pod Security

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

RBAC, Service Accounts, Admission Control & Pod Security

Chapter 12 — the Kubernetes security model:
who can access, what they can do, and what
runs.

👑 Chapter 12

RBAC · ServiceAccounts · Admission · Pod Security

Chapter Purpose

Kubernetes security decides:

- Who can access the cluster
- What they can do
- Which workloads can run
- What permissions Pods have
- Which policies are enforced before objects are created

This chapter covers the core Kubernetes security model.

1. Kubernetes Security Flow

When a request reaches Kubernetes API Server:

```
Request
  |
  v
Authentication
  |
  v
Authorization
  |
  v
Admission Control
  |
  v
Object stored in etcd
```

Simple meaning:

Authentication = Who are you?

Authorization = Are you allowed?

Admission = Is this request acceptable?

2. Authentication

Authentication verifies identity.

Examples of identities:

- Human user
- CI/CD system
- Service account
- Cloud IAM identity

- Certificate-based user
- OIDC user

Kubernetes itself does not usually manage normal human users directly.

It integrates with external identity systems.

Examples:

- OIDC
- AWS IAM for EKS
- Azure AD
- Google IAM
- Client certificates

3. Authorization

Authorization decides what an authenticated identity can do.

Example:

Can Saichand list Pods in dev namespace?

Can CI/CD create Deployments in prod?

Can this ServiceAccount read Secrets?

Kubernetes supports multiple authorization modes, but RBAC is the most common.

4. What Is RBAC?

RBAC means Role-Based Access Control.

SIMPLE DEFINITION

RBAC controls what users, groups, and ServiceAccounts can do in Kubernetes.

RBAC answers:

Who can do what, on which resources, in which namespace?

5. RBAC Objects

Main RBAC objects:

Object	Scope	Purpose
Role	Namespace	Permissions inside one namespace
RoleBinding	Namespace	Assigns Role to user/group/ServiceAccount
ClusterRole	Cluster-wide	Cluster-level or reusable permissions
ClusterRoleBinding	Cluster-wide	Assigns ClusterRole cluster-wide

6. Role

A Role defines permissions inside a namespace.

Example:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

YAML

This Role allows reading Pods in the dev namespace.

7. RoleBinding

A RoleBinding assigns a Role to a subject.

Example:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: sa-pod-reader
  namespace: dev
subjects:
- kind: ServiceAccount
  name: app-sa
  namespace: dev
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

YAML

This gives app-sa permission to read Pods in dev.

8. ClusterRole

A ClusterRole can define cluster-wide permissions.

Example:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list", "watch"]
```

YAML

Nodes are cluster-scoped resources, so ClusterRole is needed.

9. ClusterRoleBinding

A ClusterRoleBinding grants permissions across the whole cluster.

Example:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: node-reader-binding
subjects:
- kind: ServiceAccount
  name: monitoring-sa
  namespace: monitoring
roleRef:
  kind: ClusterRole
  name: node-reader
  apiGroup: rbac.authorization.k8s.io
```

YAML

Use ClusterRoleBinding carefully because it can grant broad access.

10. Service Accounts

A ServiceAccount is an identity used by Pods.

SIMPLE DEFINITION

A ServiceAccount gives a Pod an identity when talking to the Kubernetes API.

Example:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: dev
```

YAML

Pod using it:

```
spec:
  serviceAccountName: app-sa
```

YAML

11. Why Service Accounts Exist

Applications sometimes need to call Kubernetes API.

Examples:

- Controller watches resources.
- CI/CD runner creates Pods.
- Monitoring agent lists nodes.
- External Secrets Operator reads SecretStore objects.
- Ingress controller watches Ingress resources.

Instead of using human credentials, Pods use ServiceAccounts.

12. Default Service Account

Every namespace has a default ServiceAccount.

If you do not specify one, Pods use:

```
default
```

BEST PRACTICE

Create dedicated ServiceAccounts for applications instead of relying on default.

13. Least Privilege

Least privilege means giving only the permissions required.

Bad:

Give app cluster-admin because it needs to read one ConfigMap.

Good:

Give app only get/list permission for that ConfigMap in its namespace.

Least privilege reduces blast radius.

14. Dangerous Permissions

Be careful with permissions such as:

```
cluster-admin
secrets get/list/watch
pods/exec
pods/attach
pods/create
roles create/update
rolebindings create/update
clusterrolebindings create/update
nodes/proxy
```

Why?

Some permissions can lead to privilege escalation.

For example, if a user can create Pods using a privileged ServiceAccount, they may inherit that ServiceAccount's permissions.

15. Check Permissions With `kubectl auth can-i`

Command:

```
kubectl auth can-i get pods
```

SHELL

Check for another namespace:

```
kubectl auth can-i get pods -n prod
```

SHELL

Check as ServiceAccount:

```
kubectl auth can-i list secrets \
  --as=system:serviceaccount:dev:app-sa \
  -n dev
```

This is very useful for RBAC troubleshooting.

16. Admission Controllers

Admission controllers run after authentication and authorization but before object persistence.

They can:

- Validate requests
- Mutate requests
- Reject requests
- Enforce policies
- Inject defaults
- Add sidecars
- Enforce security standards

Examples:

- NamespaceLifecycle
- ResourceQuota
- LimitRanger
- MutatingAdmissionWebhook
- ValidatingAdmissionWebhook
- Pod Security Admission

17. Mutating Admission

Mutating admission can modify objects before they are stored.

Examples:

- Add sidecar container
- Add labels
- Add default resource requests
- Inject environment variables
- Add security settings

Service mesh sidecar injection is a common example.

18. Validating Admission

Validating admission checks whether an object is allowed.

Examples:

- Reject privileged Pods
- Require resource requests
- Require approved image registry
- Require labels
- Block latest tag
- Enforce runAsNonRoot

Policy tools often use validating admission webhooks.

19. Pod Security Admission

Pod Security Admission enforces Pod Security Standards.

Common policy levels:

Level	Meaning
Privileged	Unrestricted
Baseline	Prevents known privilege escalations
Restricted	Stronger security hardening

Example namespace labels:

```
apiVersion: v1
kind: Namespace
metadata:
  name: prod
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
```

YAML

20. Pod Security Context

Pod/container security settings can include:

YAML

```
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop:
      - ALL
```

These settings reduce runtime risk.

21. Good Secure Pod Example

YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: secure-app
spec:
  serviceAccountName: app-sa
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    fsGroup: 10001
  containers:
    - name: app
      image: example.com/app:1.0.0
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop:
            - ALL
      resources:
        requests:
          cpu: "250m"
          memory: "256Mi"
        limits:
          memory: "512Mi"
```

22. Image Security Policies

Production clusters often enforce image rules:

- Only approved registries
- No latest tag
- Signed images only

- Vulnerability scan passed
- Non-root images
- Required SBOM/provenance

Tools:

- Kyverno
- OPA Gatekeeper
- Validating Admission Policy
- Sigstore policy-controller
- Cloud provider policy tools

23. Network Security

RBAC controls Kubernetes API permissions.

It does not control Pod-to-Pod traffic.

For network traffic, use:

NetworkPolicy

NetworkPolicies are covered in a later chapter.

IMPORTANT RBAC protects API access. NetworkPolicy protects network communication.

24. Secrets and RBAC

Secrets are sensitive.

Anyone who can read Secrets may access passwords, tokens, certificates, or credentials.

Avoid broad permissions:

```
resources: ["secrets"]
verbs: ["get", "list", "watch"]
```

YAML

Give Secret access only when required.

Prefer external secret managers for production.

25. EKS IAM Note

In Amazon EKS, cluster access often combines:

- AWS IAM authentication
- Kubernetes RBAC authorization

For workloads accessing AWS services, EKS commonly uses:

IRSA / EKS Pod Identity

This gives Pods AWS permissions without storing AWS keys in Kubernetes Secrets.

EKS is covered deeply in the EKS volume.

26. RBAC Troubleshooting

Common error:

Error from server (Forbidden): pods is forbidden

Check current user:

```
kubectl auth whoami
```

SHELL

Check permission:

```
kubectl auth can-i list pods -n dev
```

SHELL

Check bindings:

```
kubectl get rolebinding -n dev  
kubectl get clusterrolebinding
```

SHELL

Describe:

```
kubectl describe rolebinding <name> -n dev  
kubectl describe clusterrolebinding <name>
```

SHELL

27. ServiceAccount Troubleshooting

Check Pod ServiceAccount:

```
kubectl get pod <pod> -o jsonpath='{.spec.serviceAccountName}'
```

SHELL

Check token/mount behavior:

```
kubectl describe pod <pod>
```

SHELL

Check permissions:

```
kubectl auth can-i get pods \
  --as=system:serviceaccount:<namespace>:<serviceaccount> \
  -n <namespace>
```

SHELL

28. Admission Troubleshooting

If object creation is rejected:

```
kubectl apply -f pod.yaml
```

SHELL

Possible errors:

```
violates PodSecurity restricted policy
admission webhook denied the request
resource quota exceeded
must set resources
image registry not allowed
```

Check:

```
kubectl get events
kubectl describe namespace <namespace>
kubectl get validatingwebhookconfiguration
kubectl get mutatingwebhookconfiguration
```

SHELL

29. Production Best Practices

- Use RBAC least privilege.

- Avoid cluster-admin for applications.
- Use dedicated ServiceAccounts.
- Avoid default ServiceAccount for apps.
- Restrict Secret access.
- Use namespace boundaries.
- Enforce Pod Security Standards.
- Run containers as non-root.
- Drop Linux capabilities.
- Avoid privileged Pods.
- Disable privilege escalation.
- Use read-only root filesystem where possible.
- Use admission policies.
- Audit permissions regularly.
- Use external identity provider.
- Use external secret manager.
- Use image policy enforcement.

30. Common Mistakes

Mistake	Why Bad
Giving cluster-admin to apps	Huge blast radius
Using default ServiceAccount everywhere	Poor isolation
Allowing broad Secret access	Secret leakage
Ignoring admission errors	Policy violations remain unclear
Running privileged Pods	Node compromise risk
No Pod security standards	Weak runtime baseline
No RBAC review	Permission creep
Confusing RBAC with network security	Different controls

31. Interview Questions and Answers

Beginner

1. What is RBAC?

RBAC is Role-Based Access Control. It controls what users, groups, and ServiceAccounts can do in Kubernetes.

2. What is a ServiceAccount?

A ServiceAccount is an identity used by Pods to access the Kubernetes API.

3. What is a Role?

A Role defines permissions inside a namespace.

4. What is a ClusterRole?

A ClusterRole defines cluster-wide or reusable permissions.

5. What is admission control?

Admission control validates or mutates requests before they are stored in Kubernetes.

Intermediate

1. Role vs ClusterRole?

Role is namespaced. ClusterRole can grant cluster-scoped permissions or be reused across namespaces.

2. RoleBinding vs ClusterRoleBinding?

RoleBinding grants permissions within a namespace. ClusterRoleBinding grants permissions cluster-wide.

3. What is least privilege?

Giving only the minimum permissions required to perform a task.

4. How do you check if a user can perform an action?

Use `kubectl auth can-i`.

5. Why should apps avoid default ServiceAccount?

It reduces clarity and may accidentally inherit unwanted permissions.

Advanced

1. Why is Secret access dangerous?

Secrets may contain credentials. Reading Secrets can expose passwords, tokens, certificates, and cloud credentials.

2. What is privilege escalation through RBAC?

A user with permissions to create certain objects or bind roles may gain permissions beyond what was intended.

3. What is Pod Security Admission?

A built-in admission mechanism that enforces Pod Security Standards at namespace level.

4. Mutating vs validating webhook?

Mutating webhook modifies requests. Validating webhook allows or rejects requests.

5. How do admission policies improve security?

They enforce standards such as non-root containers, approved registries, required labels, and blocked privileged Pods.

32. Scenario-Based Interview Questions

Scenario 1: Forbidden Error

QUESTION A user gets pods is forbidden. What do you check?

EXPECTED ANSWER Check identity, namespace, Role/RoleBinding, ClusterRole/ClusterRoleBinding, and verify using `kubectl auth can-i`.

Scenario 2: App Needs to Read ConfigMaps

QUESTION A Pod needs to read ConfigMaps in its namespace. What do you create?

EXPECTED ANSWER Create a dedicated ServiceAccount, Role with get/list/watch on ConfigMaps, and RoleBinding to that ServiceAccount.

Scenario 3: Secret Access Too Broad

QUESTION A team has list access to all Secrets in all namespaces. What is the risk?

EXPECTED ANSWER They can read sensitive credentials cluster-wide. Restrict access using least privilege and namespace-specific RoleBindings.

Scenario 4: Pod Rejected by Admission

QUESTION A Pod is rejected because it violates restricted policy. What do you check?

EXPECTED ANSWER Check Pod securityContext, namespace Pod Security labels, privileged settings, runAsNonRoot, capabilities, hostPath, hostNetwork, and admission error message.

33. Hands-On Labs

Lab 1: Create ServiceAccount

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: dev
```

YAML

Lab 2: Create Role

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev
  name: config-reader
rules:
  - apiGroups: [""]
    resources: ["configmaps"]
    verbs: ["get", "list", "watch"]
```

YAML

Lab 3: Create RoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-config-reader
  namespace: dev
subjects:
  - kind: ServiceAccount
    name: app-sa
    namespace: dev
roleRef:
  kind: Role
  name: config-reader
  apiGroup: rbac.authorization.k8s.io
```

YAML

Lab 4: Test Permissions

```
kubectl auth can-i list configmaps \
  --as=system:serviceaccount:dev:app-sa \
  -n dev
```

SHELL

34. Chapter Revision Notes

- Authentication verifies identity.
- Authorization checks permissions.
- Admission control validates or mutates requests.
- RBAC is the common Kubernetes authorization model.
- Role and RoleBinding are namespace-scoped.
- ClusterRole and ClusterRoleBinding are cluster-scoped.
- ServiceAccounts are identities for Pods.
- Use least privilege.
- Avoid broad Secret access.
- Admission policies enforce standards before objects are created.
- Pod Security Admission enforces Pod Security Standards.
- Runtime hardening uses securityContext.

35. Key Interview Statement

Memorize this:

KEY STATEMENT

Kubernetes API security follows authentication, authorization, and admission control. RBAC defines who can perform which actions on which resources, ServiceAccounts provide identities for Pods, and admission controllers enforce policies before objects are stored. Production clusters should use least privilege, dedicated ServiceAccounts, restricted Secret access, Pod Security Standards, and policy enforcement.

End of Volume 02 — Chapter 12

Next chapter:

Chapter 13 — NetworkPolicies, Kubernetes Networking Security, and Zero Trust Traffic Control

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

NetworkPolicies & Zero Trust Traffic Control

Chapter 13 — controlling Pod-to-Pod and external traffic with default-deny and allow-lists.

👑 Chapter 13

NetworkPolicy · Ingress · Egress · Zero Trust

Chapter Purpose

Kubernetes networking allows Pods to communicate with each other.

By default, many Kubernetes clusters allow all Pods to talk to all other Pods.

That may be acceptable for a small lab, but it is risky in production.

NetworkPolicies allow you to control Pod-to-Pod and Pod-to-external traffic.

This chapter explains Kubernetes network security from beginner level to production level.

Learning Objectives

By the end of this chapter, you should be able to explain:

- What NetworkPolicy is.
- Why NetworkPolicy exists.
- Default Kubernetes network behavior.
- Ingress and egress traffic.
- Pod selectors.
- Namespace selectors.
- IP blocks.
- Default deny policies.
- Allow-list traffic model.
- NetworkPolicy limitations.
- CNI requirements.
- Zero trust networking.
- Troubleshooting NetworkPolicies.
- Production best practices.
- Interview answers.

1. Why Network Security Is Needed

In Kubernetes, many applications run in the same cluster.

Example:

```
frontend
backend-api
payment-service
database
redis
monitoring
logging
admin-service
```

Without network restrictions, a compromised Pod may try to connect to many internal services.

Example risk:

```
compromised frontend Pod → database Pod
compromised test Pod → production API
compromised app Pod → metadata/internal service
```

NetworkPolicy reduces this risk.

2. Default Kubernetes Network Behavior

In many clusters, default behavior is:

All Pods can talk to all Pods.

All Pods can send outbound traffic.

This is easy for development but weak for security.

Production clusters should move toward:

Deny by default.

Allow only required traffic.

This is called an allow-list model.

3. What Is NetworkPolicy?

NetworkPolicy is a Kubernetes object used to control network traffic to and from Pods.

SIMPLE DEFINITION A NetworkPolicy defines which Pods are allowed to send or receive network traffic.

NetworkPolicy can control:

- Ingress traffic to Pods
- Egress traffic from Pods
- Traffic from selected Pods
- Traffic from selected namespaces

- Traffic to selected IP ranges
- Allowed ports and protocols

4. Important Requirement

NetworkPolicies only work if the cluster CNI plugin supports them.

Examples of CNIs that support NetworkPolicy:

- Calico
- Cilium
- Weave Net
- Antrea

Some CNI plugins may not enforce NetworkPolicy by default.

IMPORTANT INTERVIEW ANSWER

Creating a NetworkPolicy object does nothing unless the CNI plugin enforces NetworkPolicy.

5. Real-World Analogy

Think of an office building.

By default, everyone can enter every room.

NetworkPolicy is like access control doors.

Example:

- Employees can enter general office.
- Finance team can enter finance room.
- Visitors cannot enter server room.
- Only database admin can enter database room.

Kubernetes Concept	Office Analogy
Pod	Room/person
Namespace	Department floor
NetworkPolicy	Access control rule
Ingress	Who can enter room
Egress	Where room occupant can go
Default deny	Locked doors
Allow rule	Access badge permission

6. Ingress vs Egress

Ingress

Ingress means traffic entering a Pod.

Example:

```
frontend → backend
```

From backend perspective, this is ingress.

Egress

Egress means traffic leaving a Pod.

Example:

```
backend → database
```

From backend perspective, this is egress.

7. NetworkPolicy Structure

Basic structure:

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: example-policy
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Ingress
    - Egress
  ingress: []
  egress: []

```

Important parts:

Field	Purpose
podSelector	Selects Pods policy applies to
policyTypes	Ingress, Egress, or both
ingress	Allowed incoming traffic
egress	Allowed outgoing traffic

8. How NetworkPolicy Selects Pods

A NetworkPolicy applies to Pods selected by podSelector.

Example:

```

podSelector:
  matchLabels:
    app: backend

```

This policy applies to Pods with:

```
app=backend
```

It does not apply to other Pods.

9. Default Deny Ingress

A common first security step is deny all ingress to Pods in a namespace.

YAML

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}
  policyTypes:
    - Ingress
```

Explanation:

YAML

```
podSelector: {}
```

Selects all Pods in the namespace.

No ingress allow rules means no ingress traffic is allowed.

10. Default Deny Egress

Deny all outbound traffic:

YAML

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-egress
spec:
  podSelector: {}
  policyTypes:
    - Egress
```

This blocks all egress traffic from selected Pods unless another policy allows it.

Be careful.

If you deny all egress, DNS may stop working unless explicitly allowed.

11. Allow Frontend to Backend

Scenario:

frontend Pods should call backend Pods on port 8080.

Policy for backend:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 8080
```

This allows traffic into backend Pods only from frontend Pods on TCP 8080.

12. Allow Backend to Database

Scenario:

backend Pods should connect to database Pods on port 5432.

Policy for database:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-backend-to-db
spec:
  podSelector:
    matchLabels:
      app: database
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: backend
      ports:
        - protocol: TCP
          port: 5432
```


13. Namespace Selector

You can allow traffic from selected namespaces.

First label namespace:

```
kubectl label namespace monitoring purpose=monitoring
```

SHELL

Allow monitoring namespace to scrape metrics:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-monitoring
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              purpose: monitoring
      ports:
        - protocol: TCP
          port: 9090
```

YAML

This allows Pods from namespaces labeled `purpose=monitoring`.

14. Pod Selector + Namespace Selector

You can combine namespace and Pod selectors.

Example:

```
from:
  - namespaceSelector:
      matchLabels:
        purpose: monitoring
    podSelector:
      matchLabels:
        app: prometheus
```

YAML

Meaning:

Allow only Pods labeled `app=prometheus` in namespaces labeled `purpose=monitoring`.

This is stricter than namespace-only.

15. Allow Egress to DNS

When using egress deny, allow DNS.

CoreDNS usually runs in kube-system.

Example:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns-egress
spec:
  podSelector: {}
  policyTypes:
    - Egress
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
                kubernetes.io/metadata.name: kube-system
  ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
```

YAML

Without DNS egress, service discovery may fail.

16. Allow Egress to External IP

Example:

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-external-api
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Egress
  egress:
    - to:
        - ipBlock:
            cidr: 203.0.113.0/24
      ports:
        - protocol: TCP
          port: 443

```

This allows backend Pods to call external IP range on HTTPS.

17. ipBlock

ipBlock selects IP CIDR ranges.

Example:

```

ipBlock:
  cidr: 10.0.0.0/8
  except:
    - 10.10.0.0/16

```

This allows 10.0.0.0/8 except 10.10.0.0/16.

Use carefully because Pod IP ranges and cluster networking may vary.

18. NetworkPolicy Rules Are Additive

NetworkPolicies are additive.

If multiple policies select the same Pod, allowed traffic is the union of all allow rules.

IMPORTANT

NetworkPolicy does not have explicit deny rules.

It works by selecting Pods and allowing specific traffic.

Default-deny happens when a Pod is selected by a policy but no allow rule permits traffic.

19. Ingress Isolation

A Pod becomes ingress-isolated when at least one NetworkPolicy selects it for Ingress.

Then only allowed ingress traffic is permitted.

If no ingress policy selects a Pod, ingress is unrestricted by NetworkPolicy.

20. Egress Isolation

A Pod becomes egress-isolated when at least one NetworkPolicy selects it for Egress.

Then only allowed egress traffic is permitted.

If no egress policy selects a Pod, egress is unrestricted by NetworkPolicy.

21. NetworkPolicy Limitations

Standard Kubernetes NetworkPolicy does not cover everything.

Limitations may include:

- No FQDN-based egress in standard API
- No L7 HTTP path rules in standard NetworkPolicy
- No explicit deny rules
- Depends on CNI support
- Behavior may vary by CNI
- Node traffic handling can vary
- HostNetwork Pods may behave differently

Advanced CNIs like Cilium and Calico provide extended policy features.

22. Zero Trust Networking

Zero trust means:

Do not trust traffic just because it is inside the cluster.

Principles:

- Deny by default.
- Allow only required communication.
- Authenticate services.
- Encrypt sensitive traffic.
- Limit lateral movement.
- Monitor traffic.

- Use least privilege.

NetworkPolicy is one part of zero trust.

23. NetworkPolicy Design Pattern

A good production pattern:

1. Label namespaces and Pods clearly.
2. Apply default deny ingress.
3. Apply default deny egress where practical.
4. Allow DNS egress.
5. Allow required service-to-service traffic.
6. Allow monitoring traffic.
7. Test application flows.
8. Monitor denied/failed connections.

24. Example Secure App Traffic Model

Application:

```
frontend → api → database
monitoring → api metrics
api → external payment API
```

Policies:

```
default deny ingress
default deny egress
allow frontend to api:8080
allow api to database:5432
allow monitoring to api:9090
allow api to DNS
allow api to payment API:443
```

This reduces unnecessary lateral movement.

25. NetworkPolicy Troubleshooting Commands

SHELL

```
kubectl get networkpolicy
kubectl describe networkpolicy <name>
kubectl get pods --show-labels
kubectl get namespaces --show-labels
kubectl exec -it <pod> -- nslookup service-name
kubectl exec -it <pod> -- wget -q0- http://service:port
kubectl exec -it <pod> -- nc -vz service port
```

Use a debug image if app image has no tools.

26. Debug Pod for Network Testing

SHELL

```
kubectl run net-debug \
  --image=nicolaka/netshoot \
  --rm -it \
  --restart=Never \
  -- bash
```

Useful tools:

- curl
- dig
- nslookup
- nc
- tcpdump
- ip
- ss

27. Common Problem: Policy Not Working

Possible causes:

- CNI does not support NetworkPolicy.
- Policy in wrong namespace.
- Pod labels do not match selector.
- Namespace labels do not match.
- Wrong port.
- Wrong protocol.
- Egress DNS blocked.
- Testing from wrong source Pod.

- Policy selects wrong destination Pods.

28. Common Problem: DNS Broken After Egress Deny

Symptom:

Temporary failure in name resolution

Cause:

DNS egress blocked.

Fix:

Allow egress to CoreDNS on TCP/UDP port 53.

29. Common Problem: Service Connection Fails

Check:

1. Does DNS resolve?
2. Does Service have endpoints?
3. Does NetworkPolicy allow source to destination?
4. Does destination port match?
5. Does app listen on that port?
6. Is readiness passing?
7. Is CNI enforcing correctly?

Commands:

```
kubectl get svc
kubectl get endpoints
kubectl get networkpolicy
kubectl describe pod <pod>
```

SHELL

30. Production Best Practices

- Use default deny ingress in production namespaces.
- Use egress control for sensitive workloads.
- Always allow required DNS egress.
- Label Pods and namespaces consistently.
- Write policies based on labels, not Pod IPs.

- Keep policies simple and readable.
- Test policies before enforcing broadly.
- Use CNI with strong policy support.
- Monitor dropped traffic if CNI supports it.
- Use service mesh/mTLS for identity-based encryption where required.
- Avoid broad allow-all policies.
- Document application communication flows.
- Include NetworkPolicies in GitOps.

31. Common Mistakes

Mistake	Why Bad
Assuming NetworkPolicy works on every CNI	Some CNIs do not enforce it
Forgetting DNS egress	Service discovery breaks
Wrong namespace	Policies are namespace-scoped
Label mismatch	Policy selects nothing
Using Pod IPs	Pod IPs change
Too broad allow rules	Weak security
Applying deny all without testing	Causes outage
Confusing Ingress object with ingress traffic	Different concepts

32. Interview Questions and Answers

Beginner

1. What is NetworkPolicy?

NetworkPolicy controls allowed network traffic to and from Pods.

2. What is ingress traffic?

Traffic entering a Pod.

3. What is egress traffic?

Traffic leaving a Pod.

4. Does NetworkPolicy work with every CNI?

No. The CNI plugin must support and enforce NetworkPolicy.

5. What is default deny?

A policy model where traffic is blocked unless explicitly allowed.

Intermediate

1. How does NetworkPolicy select Pods?

Using podSelector labels.

2. What is namespaceSelector?

It selects namespaces based on namespace labels.

3. Why allow DNS egress?

Pods need DNS to resolve Services and external domains.

4. Are NetworkPolicy rules additive?

Yes. Allowed traffic is the union of all policies selecting the Pod.

5. Does NetworkPolicy provide explicit deny?

Standard Kubernetes NetworkPolicy uses allow rules, not explicit deny rules.

Advanced

1. What are NetworkPolicy limitations?

Standard NetworkPolicy lacks FQDN egress, L7 HTTP rules, explicit deny, and depends on CNI enforcement.

2. How do you implement zero trust traffic control?

Start with default deny, allow only required flows, use identity-based controls, encrypt traffic where needed, and monitor flows.

3. Why might a policy not block traffic?

CNI may not enforce policies, policy selector may not match, or traffic path may not be covered.

4. How do NetworkPolicies affect existing connections?

Behavior can depend on CNI implementation. Generally policies affect allowed traffic rules enforced by the dataplane.

5. How do you troubleshoot NetworkPolicy?

Verify labels, namespace, policies, endpoints, DNS, ports, protocols, CNI support, and test from source Pods.

33. Scenario-Based Interview Questions

Scenario 1: DNS Stops Working

QUESTION After applying default deny egress, Pods cannot resolve services.

EXPECTED ANSWER DNS egress is blocked. Allow UDP/TCP 53 traffic to CoreDNS in kube-system.

Scenario 2: Policy Applied but Traffic Still Works

QUESTION You created NetworkPolicy but traffic is still allowed.

EXPECTED ANSWER Check if the CNI supports NetworkPolicy, whether policy is in correct namespace, whether podSelector matches Pods, and whether another policy allows traffic.

Scenario 3: Backend Should Only Accept Frontend

QUESTION How do you allow only frontend Pods to call backend Pods?

EXPECTED ANSWER Create NetworkPolicy selecting backend Pods and allow ingress from Pods labeled frontend on required port.

Scenario 4: API Needs External Payment Provider

QUESTION Default deny egress is enabled. API must call external payment API over HTTPS.

EXPECTED ANSWER Allow egress from API Pods to the payment provider IP/CIDR on TCP 443. If FQDN control is needed, use CNI extended policy or egress gateway.

34. Hands-On Labs

Lab 1: Default Deny Ingress

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}
  policyTypes:
    - Ingress
```

YAML

Apply:

```
kubectl apply -f default-deny-ingress.yaml
```

SHELL

Lab 2: Allow Frontend to Backend

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 8080
```

YAML

Lab 3: Test Connectivity

```
kubectl run test-client --image=nicolaka/netshoot --rm -it --restart=Never -- bash
```

SHELL

Inside:

```
curl http://backend:8080  
nc -vz backend 8080
```

35. Chapter Revision Notes

- NetworkPolicy controls Pod ingress and egress traffic.
- It requires CNI support.
- Default Kubernetes networking may allow broad Pod communication.
- Default deny is a common production pattern.
- Policies select Pods using labels.
- Namespace selectors use namespace labels.
- ipBlock allows CIDR-based rules.
- Rules are additive.
- Standard NetworkPolicy has allow rules, not explicit deny.
- Always allow DNS when restricting egress.
- NetworkPolicy is one part of zero trust networking.

36. Key Interview Statement

Memorize this:

KEY STATEMENT

NetworkPolicy is Kubernetes' built-in mechanism for controlling Pod network traffic. It uses label selectors to define allowed ingress and egress flows, but it only works when the CNI enforces it. Production clusters commonly use default-deny policies and then explicitly allow required service-to-service, DNS, monitoring, and external traffic.

End of Volume 02 — Chapter 13

Next chapter:

Chapter 14 — Observability: Logs, Metrics, Events, Traces, Probes, and Production Monitoring.

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

Observability: Logs, Metrics, Events, Traces & Probes

Chapter 14 — understanding and operating
production systems through observability
signals.

👑 Chapter 14

Logs · Metrics · Traces · Events · Probes

Chapter Purpose

Observability helps you understand what is happening inside a Kubernetes cluster and applications.

In production, you must answer questions like:

Is the application healthy?

Why is latency high?

Why did the Pod restart?

Is CPU throttling happening?

Are users getting errors?

Is traffic reaching the correct service?

Did a deployment break something?

Observability is not optional in Kubernetes. It is required for production reliability.

1. What Is Observability?

Observability means the ability to understand system behavior from external signals.

Main signals:

- Logs
- Metrics
- Traces
- Events
- Health probes
- Alerts
- Dashboards

SIMPLE DEFINITION

Observability helps engineers understand, debug, and operate systems in production.

2. Monitoring vs Observability

Monitoring	Observability
Watches known problems	Helps investigate unknown problems
Uses dashboards and alerts	Uses logs, metrics, traces, events
Answers “is it broken?”	Answers “why is it broken?”
Usually predefined	Exploratory

Monitoring tells you something is wrong.

Observability helps you find why.

3. Kubernetes Observability Layers

- Cluster level
- Node level
- Pod level
- Container level
- Application level
- Network level
- Storage level
- Control plane level

Each layer can fail differently.

Example:

```
App error → application logs
Pod restart → pod events/logs
Node pressure → node metrics/events
DNS failure → CoreDNS logs
Ingress 503 → ingress controller logs + service endpoints
```

4. Logs

Logs are text records emitted by applications or system components.

BEST PRACTICE

Containers should write logs to stdout and stderr.

Kubernetes collects container stdout/stderr.

View logs:

```
kubectl logs <pod-name>
```

SHELL

Follow logs:

```
kubectl logs -f <pod-name>
```

SHELL

Previous crashed container logs:

```
kubectl logs <pod-name> --previous
```

SHELL

5. Logs for Multi-Container Pods

If Pod has multiple containers:

```
kubectl logs <pod-name> -c <container-name>
```

SHELL

Follow:

```
kubectl logs -f <pod-name> -c <container-name>
```

SHELL

All containers:

```
kubectl logs <pod-name> --all-containers=true
```

SHELL

6. Why Centralized Logging Is Needed

Pod logs are not enough by themselves.

Pods are temporary.

Nodes can fail.

Containers can restart.

You need centralized logs.

Common tools:

- Splunk
- ELK / OpenSearch

- Loki
- CloudWatch Logs
- Datadog
- New Relic
- Fluent Bit
- Fluentd
- Vector

Typical flow:

```
Application stdout/stderr
  |
  v
Node log files
  |
  v
Log agent DaemonSet
  |
  v
Central log platform
```

7. Metrics

Metrics are numeric measurements over time.

Examples:

- CPU usage
- Memory usage
- Request count
- Error rate
- Latency
- Pod restarts
- Disk usage
- Network traffic
- Queue depth

View basic metrics:

```
kubectl top nodes
kubectl top pods
```

SHELL

This requires Metrics Server.

8. Metrics Server

Metrics Server provides basic CPU and memory metrics.

Used by:

- `kubectl top`
- Horizontal Pod Autoscaler

It is not a full monitoring system.

For production monitoring, use Prometheus, CloudWatch, Datadog, Grafana, or similar tools.

9. Prometheus

Prometheus is a common Kubernetes monitoring system.

It collects metrics by scraping HTTP endpoints.

Typical flow:

```
Application exposes /metrics
  |
  v
Prometheus scrapes metrics
  |
  v
Prometheus stores time-series data
  |
  v
Grafana shows dashboards
  |
  v
Alertmanager sends alerts
```

10. Important Kubernetes Metrics

Area	Important Metrics
Node	CPU, memory, disk, network
Pod	CPU, memory, restarts
Container	CPU throttling, memory usage
Deployment	desired vs available replicas
API Server	latency, error rate
etcd	disk latency, leader changes
Ingress	4xx, 5xx, latency
HPA	current vs desired replicas
Storage	PVC usage, IOPS, latency

11. Events

Kubernetes Events record important cluster actions.

Examples:

- Pod scheduled
- Image pull failed
- Container started
- Probe failed
- Volume mount failed
- Node not ready
- Failed scheduling

View events:

```
kubectl get events
```

SHELL

Sort by time:

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

SHELL

Events are extremely useful for troubleshooting.

12. Events vs Logs

Events	Logs
Kubernetes-generated	App/container-generated
Explain cluster actions	Explain app behavior
Scheduling, image, volume, probe issues	App errors and runtime output
Short-lived	Usually centralized

Use both together.

13. Distributed Tracing

Traces show request flow across services.

Example:

```
User request
  |
  v
frontend
  |
  v
api
  |
  v
payment-service
  |
  v
database
```

Tracing helps answer:

Where is latency happening?

Which service failed?

Which dependency is slow?

Common tools:

- Jaeger
- Tempo
- Zipkin
- OpenTelemetry
- Datadog APM
- New Relic

14. OpenTelemetry

OpenTelemetry is a vendor-neutral standard for collecting:

- Metrics
- Logs
- Traces

It helps avoid vendor lock-in.

Typical flow:

```
Application instrumentation
    |
    v
OpenTelemetry Collector
    |
    v
Backend: Jaeger / Tempo / Datadog / New Relic / Splunk
```

15. Health Probes

Kubernetes probes help determine container health.

Main probes:

Probe	Purpose
livenessProbe	Should container be restarted?
readinessProbe	Should Pod receive traffic?
startupProbe	Has slow-starting app finished startup?

Probes are critical for safe production operations.

16. Liveness Probe

Liveness probe checks whether the app is alive.

If liveness fails repeatedly, Kubernetes restarts the container.

Example:

YAML

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
```

Use liveness for deadlock or stuck-process detection.

17. Readiness Probe

Readiness probe checks whether the app is ready to receive traffic.

If readiness fails, Kubernetes removes the Pod from Service endpoints.

Example:

YAML

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

Readiness prevents traffic from reaching unready Pods.

18. Startup Probe

Startup probe is for slow-starting applications.

Until startup probe succeeds, liveness and readiness behavior is delayed.

Example:

YAML

```
startupProbe:
  httpGet:
    path: /startup
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
```

This gives the app up to 300 seconds to start.

19. Probe Mistakes

Mistake	Problem
Liveness too aggressive	Restarts healthy but slow app
Readiness missing	Traffic goes to unready Pod
Same endpoint for all probes	Poor signal quality
Probe depends on optional dependency	Unnecessary restarts
No startup probe for slow app	CrashLoop during startup
Timeout too low	False failures

20. Good Probe Design

Use separate endpoints:

```
/startup → app boot completed  
/ready → app can serve traffic  
/health → app process is alive
```

Good readiness should check critical dependencies.

Good liveness should not fail because of temporary dependency issues unless restart truly helps.

21. Alerts

Alerts notify engineers when action is needed.

Good alerts are:

- Actionable
- Urgent
- Clear
- Low-noise
- Business-impact focused

Bad alert:

CPU is 75%

Better alert:

API p95 latency above 1s for 10 minutes and error rate above threshold

22. Golden Signals

Common SRE golden signals:

- Latency
- Traffic
- Errors
- Saturation

For Kubernetes:

Signal	Example
Latency	p95 request duration
Traffic	requests per second
Errors	5xx rate
Saturation	CPU, memory, queue depth

23. Kubernetes Alert Examples

Useful alerts:

- Pod CrashLoopBackOff
- Deployment unavailable
- Node NotReady
- High container restarts
- PVC almost full
- HPA max replicas reached
- Ingress 5xx spike
- API latency high
- CoreDNS errors
- OOMKilled events
- CPU throttling high

24. Dashboards

Dashboards should show:

- Cluster health
- Node health
- Namespace usage

- Deployment health
- Pod restarts
- Ingress traffic
- Application latency
- Error rates
- Resource saturation
- Autoscaling behavior

Dashboards are for investigation.

Alerts are for action.

25. Troubleshooting With Observability

CrashLoopBackOff

Check:

```
kubectl logs <pod> --previous
kubectl describe pod <pod>
kubectl get events
```

SHELL

Look for:

- App error
- Missing env var
- Config issue
- Permission denied
- Failed probe
- OOMKilled

ImagePullBackOff

Check:

```
kubectl describe pod <pod>
```

SHELL

Look at Events.

Common causes:

- Wrong image
- Wrong tag
- Registry auth missing
- Network issue

- Registry unavailable

High Latency

Check:

- Ingress latency
- App latency
- Database latency
- CPU throttling
- Memory pressure
- Pod restarts
- Network errors
- Downstream services

Useful signals:

- p95/p99 latency
- request rate
- error rate
- CPU throttling
- queue depth
- trace spans

No Traffic to Pod

Check:

```
kubectl get endpoints <service>
kubectl describe service <service>
kubectl get pods --show-labels
kubectl describe pod <pod>
```

SHELL

Common causes:

- Readiness failing
- Service selector mismatch
- Wrong targetPort
- Pod not ready
- NetworkPolicy blocking

26. Production Observability Best Practices

- Log to stdout/stderr.
- Use structured JSON logs.

- Add request IDs/correlation IDs.
- Centralize logs.
- Collect metrics from apps and infrastructure.
- Use traces for distributed systems.
- Add liveness/readiness/startup probes.
- Alert on symptoms, not only causes.
- Monitor saturation and error rates.
- Track deployment versions in metrics/logs.
- Monitor CoreDNS and ingress controllers.
- Monitor HPA and node autoscaling.
- Use dashboards for investigation.
- Avoid noisy alerts.

27. Common Mistakes

Mistake	Why Bad
No readiness probe	Traffic hits unready Pods
Logs only inside files	Logs lost after Pod restart
No centralized logging	Hard incident debugging
Alerting on every warning	Alert fatigue
No correlation IDs	Hard request tracing
No metrics for business flows	Hard to detect real impact
Liveness checks dependency too aggressively	Restart storms
No PVC monitoring	Disk-full outages

28. Interview Questions and Answers

Beginner

1. What is observability?

Observability is the ability to understand system behavior using logs, metrics, traces, events, and health checks.

2. What are Kubernetes events?

Events are Kubernetes records describing cluster actions such as scheduling, image pulls, probe failures, and volume mount failures.

3. What is livenessProbe?

It checks whether a container should be restarted.

4. What is readinessProbe?

It checks whether a Pod should receive traffic.

5. What is Metrics Server?

Metrics Server provides basic CPU and memory metrics for kubectl top and HPA.

Intermediate

1. Difference between logs and metrics?

Logs are detailed event records. Metrics are numeric measurements over time.

2. Difference between monitoring and observability?

Monitoring detects known problems. Observability helps investigate unknown problems.

3. Why use distributed tracing?

To see request flow and latency across multiple services.

4. Why should apps log to stdout/stderr?

Kubernetes and log agents collect container logs from stdout/stderr.

5. What causes a Pod to be removed from Service endpoints?

Usually readiness probe failure or Pod not being Ready.

Advanced

1. Why can bad liveness probes cause outages?

Aggressive liveness probes can restart healthy but slow Pods, causing restart storms.

2. What are golden signals?

Latency, traffic, errors, and saturation.

3. Why is p95/p99 latency important?

Average latency hides tail latency experienced by real users.

4. How do traces help microservices?

They show where time is spent across service calls.

5. What should you alert on?

User-impacting symptoms and actionable conditions, such as high error rate, high latency, unavailable deployments, and saturation.

29. Scenario-Based Interview Questions

Scenario 1: Pod Restarting Frequently

QUESTION A Pod restarts every few minutes. What do you check?

EXPECTED ANSWER Check `kubectl describe pod`, restart reason, logs including `--previous`, probes, `OOMKilled` status, events, and application errors.

Scenario 2: Users Report Slowness

QUESTION Users report app slowness. What observability data do you check?

EXPECTED ANSWER Check ingress latency, app metrics, p95/p99 latency, traces, error rate, CPU throttling, memory, database latency, and downstream dependency health.

Scenario 3: No Logs Available

QUESTION Application logs are missing after Pod restart.

EXPECTED ANSWER The app may be logging to files only or logs were not centralized. Ensure `stdout/stderr` logging and centralized log collection.

Scenario 4: Service Returns 503

QUESTION Ingress returns 503. What do you check?

EXPECTED ANSWER Check Service endpoints, Pod readiness, ingress controller logs, Service `targetPort`, backend logs, and NetworkPolicy.

30. Hands-On Labs

Lab 1: View Logs

```
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
```

SHELL

Lab 2: View Events

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

SHELL

Lab 3: Check Metrics

```
kubectl top nodes
kubectl top pods
```

SHELL

Lab 4: Add Readiness Probe

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  periodSeconds: 5
```

YAML

Lab 5: Debug Endpoints

```
kubectl get svc
kubectl get endpoints
kubectl describe pod <pod>
```

SHELL

31. Chapter Revision Notes

- Observability uses logs, metrics, traces, events, and probes.
- Logs explain what happened.
- Metrics show numeric trends.
- Traces show request flow.
- Events explain Kubernetes actions.
- Probes help Kubernetes manage health and traffic.

- Liveness restarts bad containers.
- Readiness controls traffic.
- Startup helps slow applications.
- Centralized logging is required for production.
- Alerts should be actionable and user-impact focused.
- Golden signals are latency, traffic, errors, and saturation.

32. Key Interview Statement

Memorize this:

KEY STATEMENT

Kubernetes observability combines logs, metrics, traces, events, and probes. Logs explain application behavior, metrics show system trends, traces show request flow across services, events explain Kubernetes decisions, and probes help Kubernetes restart unhealthy containers or remove unready Pods from traffic.

End of Volume 02 — Chapter 14

Next chapter:

Chapter 15 — Kubernetes Troubleshooting Master Guide

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

Kubernetes Troubleshooting Master Guide

Chapter 15 — systematic debugging of Pending, CrashLoopBackOff, OOMKilled, Service, Ingress, DNS, PVC, and Node issues.

👑 Chapter 15

Chapter Purpose

This chapter is a practical troubleshooting guide.

In real DevOps, SRE, and platform engineering work, Kubernetes problems usually appear as:

- Pod Pending
- CrashLoopBackOff
- ImagePullBackOff
- OOMKilled
- Service not reachable
- Ingress 503
- DNS failure
- PVC Pending
- Node NotReady
- Deployment rollout stuck

This chapter teaches how to debug these issues step by step.

1. Troubleshooting Mindset

Do not guess randomly.

Follow a flow:

```
Application issue
  |
  v
Check Pod status
  |
  v
Check Pod events
  |
  v
Check logs
  |
  v
Check Service/endpoints
  |
  v
Check config/secrets
  |
  v
Check resources
  |
  v
Check node/network/storage
```

2. First Commands to Run

SHELL

```
kubectl get pods -A
kubectl get nodes
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl get deploy,rs,pods,svc,ingress
```

For one namespace:

SHELL

```
kubectl get all -n <namespace>
```

For one Pod:

SHELL

```
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace> --previous
```

3. Pod Pending

Meaning

Pod is accepted but not running.

Common Causes

- Insufficient CPU
- Insufficient memory
- Taints not tolerated
- Node selector mismatch
- Affinity mismatch
- PVC not bound
- Quota exceeded
- No available nodes

Debug

```
kubectl describe pod <pod-name>
kubectl get events
kubectl get nodes
kubectl describe node <node-name>
kubectl get pvc
kubectl get quota
```

SHELL

Look for:

```
0/5 nodes are available: insufficient memory
node(s) had untolerated taint
pod has unbound immediate PersistentVolumeClaims
```

4. ImagePullBackOff / ErrImagePull

Meaning

Kubernetes cannot pull the container image.

Common Causes

- Wrong image name
- Wrong tag
- Private registry authentication missing
- Registry unavailable
- Image does not exist
- Network issue
- Rate limit

Debug

```
kubectl describe pod <pod-name>
```

SHELL

Check Events.

Fix

- Correct image name/tag
- Create imagePullSecret
- Check registry credentials
- Check network access

- Verify image exists

5. CrashLoopBackOff

Meaning

Container starts, crashes, restarts, and repeats.

Debug

```
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
kubectl describe pod <pod-name>
kubectl get events
```

SHELL

Common Causes

- Application bug
- Missing environment variable
- Wrong command/args
- Config error
- Secret missing
- Permission denied
- Database unavailable
- Liveness probe killing app
- App exits immediately

Fix Pattern

- Read previous logs
- Check env/config/secret
- Check command and args
- Check permissions
- Check dependencies
- Check probes
- Fix app or manifest
- Redeploy

6. OOMKilled

Meaning

Container exceeded memory limit and was killed.

Debug

```
kubectl describe pod <pod-name>
kubectl top pod <pod-name>
kubectl logs <pod-name> --previous
```

SHELL

Look for:

```
Reason: OOMKilled
Exit Code: 137
```

Fix

- Increase memory limit
- Fix memory leak
- Reduce worker count
- Tune JVM/Node/Python memory
- Add memory monitoring
- Right-size requests/limits

7. Deployment Rollout Stuck

Debug

```
kubectl rollout status deployment/<name>
kubectl describe deployment <name>
kubectl get rs
kubectl get pods
kubectl describe pod <new-pod>
kubectl logs <new-pod>
```

SHELL

Common Causes

- New Pods not Ready
- Readiness probe failing
- ImagePullBackOff

- CrashLoopBackOff
- Pending Pods
- Resource shortage
- Bad config

Rollback

```
kubectl rollout undo deployment/<name>
```

SHELL

8. Service Not Working

Debug

```
kubectl get svc
kubectl describe svc <service-name>
kubectl get endpoints <service-name>
kubectl get endpointslices
kubectl get pods --show-labels
```

SHELL

Common Causes

- Service selector mismatch
- Pod labels wrong
- Pods not Ready
- Wrong targetPort
- Application not listening
- NetworkPolicy blocking

IMPORTANT

If a Service has no endpoints, traffic cannot reach Pods.

9. Ingress 404

Meaning

Ingress Controller received request, but no matching rule.

Debug

```
kubectl get ingress
kubectl describe ingress <name>
kubectl logs -n <ingress-namespace> <ingress-controller-pod>
```

SHELL

Common Causes

- Wrong host
- Wrong path
- Wrong ingressClassName
- DNS points to wrong load balancer
- Ingress rule missing

10. Ingress 502 / 503

Meaning

Ingress matched route but backend is unavailable or failing.

Debug

```
kubectl get svc
kubectl get endpoints <service>
kubectl describe ingress <ingress>
kubectl logs -n <ingress-namespace> <controller-pod>
kubectl describe pod <backend-pod>
```

SHELL

Common Causes

- Service has no endpoints
- Pods not Ready
- Wrong targetPort
- Backend app down
- NetworkPolicy blocking
- Protocol mismatch

11. DNS Failure

Symptoms

- Temporary failure in name resolution
- Cannot resolve service name
- nslookup fails

Debug

```
kubectl get pods -n kube-system
kubectl logs -n kube-system deployment/coredns
kubectl run dns-test --image=busybox:1.36 -it --rm --restart=Never -- nslookup
kubernetes.default
```

SHELL

Common Causes

- CoreDNS down
- NetworkPolicy blocks DNS
- CNI problem
- kube-dns service issue
- Node DNS problem

12. PVC Pending

Debug

```
kubectl get pvc
kubectl describe pvc <pvc-name>
kubectl get storageclass
kubectl get pv
```

SHELL

Common Causes

- No default StorageClass
- Wrong storageClassName
- CSI driver missing
- Access mode unsupported
- Requested size unavailable
- Waiting for first consumer
- Provisioner error

13. Volume Mount Failure

Debug

```
kubectl describe pod <pod-name>
kubectl get pvc
kubectl describe pvc <pvc-name>
kubectl get events
```

SHELL

Common Causes

- PVC not bound
- Volume attach failed
- CSI driver error
- Zone mismatch
- Permission issue
- Multi-Attach error

14. Node NotReady

Debug

```
kubectl get nodes
kubectl describe node <node-name>
```

SHELL

On the node:

```
systemctl status kubelet
systemctl status containerd
journalctl -u kubelet -xe
df -h
free -m
```

SHELL

Common Causes

- kubelet down
- container runtime down
- Node unreachable
- Disk pressure
- Memory pressure
- CNI issue

- Cloud instance issue

15. NetworkPolicy Blocking Traffic

Debug

```
kubectl get networkpolicy
kubectl describe networkpolicy <policy>
kubectl get pods --show-labels
kubectl get ns --show-labels
```

SHELL

Test:

```
kubectl run netshoot --image=nicolaka/netshoot --rm -it --restart=Never -- bash
```

SHELL

Inside:

```
curl http://service-name:port
nc -vz service-name port
nslookup service-name
```

SHELL

16. RBAC Forbidden Error

Error

Error from server (Forbidden)

Debug

```
kubectl auth can-i <verb> <resource> -n <namespace>
kubectl auth whoami
kubectl get rolebinding -n <namespace>
kubectl get clusterrolebinding
```

SHELL

Check as ServiceAccount:

```
kubectl auth can-i get pods \
  --as=system:serviceaccount:<namespace>:<serviceaccount> \
  -n <namespace>
```

SHELL

17. High CPU

Debug

```
kubectl top pods
kubectl top nodes
kubectl describe pod <pod-name>
```

SHELL

Check:

- CPU requests
- CPU limits
- CPU throttling
- Traffic spike
- Bad code loop
- Too many workers

Fix:

- Scale replicas
- Increase CPU request/limit
- Tune app concurrency
- Check HPA
- Profile application

18. High Memory

Debug

```
kubectl top pod <pod-name>
kubectl describe pod <pod-name>
kubectl logs <pod-name> --previous
```

SHELL

Fix:

- Increase memory limit
- Fix leak
- Tune runtime memory
- Reduce workers
- Use VPA recommendations

19. Useful Debug Commands

SHELL

```
kubectl get pod <pod> -o yaml
kubectl describe pod <pod>
kubectl logs <pod>
kubectl logs <pod> --previous
kubectl exec -it <pod> -- sh
kubectl get events
kubectl get endpoints
kubectl top pods
kubectl top nodes
kubectl rollout status deployment/<name>
kubectl rollout history deployment/<name>
```

20. Production Troubleshooting Checklist

```
[ ] Check namespace
[ ] Check Pod status
[ ] Check events
[ ] Check logs
[ ] Check previous logs
[ ] Check Deployment/ReplicaSet status
[ ] Check Service selector
[ ] Check endpoints
[ ] Check ingress rules
[ ] Check ConfigMaps/Secrets
[ ] Check resource requests/limits
[ ] Check node status
[ ] Check PVC/storage
[ ] Check DNS/CoreDNS
[ ] Check NetworkPolicy
[ ] Check RBAC
[ ] Check recent deployments
```

21. Interview Questions

Beginner

1. How do you troubleshoot a failing Pod?

Use `kubectl describe pod`, `kubectl logs`, `kubectl logs --previous`, and `kubectl get events`.

2. What is CrashLoopBackOff?

A container repeatedly starts, crashes, and restarts with backoff delay.

3. What is ImagePullBackOff?

Kubernetes cannot pull the image and is retrying with delay.

4. What is OOMKilled?

The container exceeded its memory limit and was killed.

5. What command shows cluster events?

kubectl get events.

Intermediate

1. How do you troubleshoot Service not routing traffic?

Check Service selector, Pod labels, endpoints, readiness probes, targetPort, and NetworkPolicy.

2. How do you troubleshoot Pending Pods?

Check events for scheduling failures, node resources, taints, affinity, PVCs, and quotas.

3. How do you troubleshoot DNS?

Check CoreDNS Pods/logs, test nslookup from a Pod, and verify NetworkPolicy allows DNS.

4. How do you troubleshoot Ingress 503?

Check Service endpoints, backend readiness, targetPort, Pod logs, and Ingress Controller logs.

5. How do you troubleshoot PVC Pending?

Check StorageClass, CSI driver, PVC events, access modes, and provisioner errors.

22. Scenario-Based Interview Questions

Scenario 1: Pod Pending

EXPECTED ANSWER I check kubectl describe pod, events, node resources, taints/tolerations, node selectors, affinity, PVC status, and namespace quota.

Scenario 2: CrashLoopBackOff

EXPECTED ANSWER I check current logs, previous logs, Pod events, command/args, environment variables, ConfigMaps, Secrets, probes, permissions, and dependencies.

Scenario 3: Service Has No Endpoints

EXPECTED ANSWER I check Service selector, Pod labels, namespace, readiness status, and EndpointSlices.

Scenario 4: Ingress 503

EXPECTED ANSWER I check backend Service endpoints, Pod readiness, Service targetPort, Ingress Controller logs, NetworkPolicy, and backend application logs.

23. Chapter Revision Notes

- Always start with Pod status, events, and logs.
- describe pod usually explains scheduling, image, probe, and volume problems.
- logs --previous is critical for CrashLoopBackOff.
- Pending usually means scheduling or storage issue.
- ImagePullBackOff usually means image or registry issue.
- OOMKilled means memory limit exceeded.
- Service issues often come from selectors, labels, endpoints, or targetPort.
- Ingress 404 means route mismatch.
- Ingress 503 means backend unavailable.
- DNS failures often involve CoreDNS or NetworkPolicy.
- PVC Pending means storage provisioning problem.
- Node NotReady means kubelet/runtime/node issue.

24. Key Interview Statement

Memorize this:

KEY STATEMENT

Kubernetes troubleshooting should be systematic. I first check resource status, events, logs, previous logs, and describe output. Then I follow the request path through Deployment, ReplicaSet, Pod, Service, Endpoints, Ingress, DNS, NetworkPolicy, storage, node health, and RBAC until I find where the actual state differs from the desired state.

End of Volume 02 — Chapter 15

Next chapter:

Chapter 16 — Kubernetes Production Best Practices, HA, Upgrades, Backup, Disaster Recovery, and Cost Optimization

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

Production Best Practices, HA, Upgrades, Backup, DR & Cost

Chapter 16 — running Kubernetes safely, reliably, and cost-effectively in production.

👑 Chapter 16

HA · Upgrades · Backup · DR · Cost

Chapter Purpose

This chapter summarizes the most important production Kubernetes practices.

The goal is to help you understand how real teams run Kubernetes safely, reliably, and cost-effectively.

1. Production Kubernetes Mindset

Production Kubernetes is not only about running Pods.

It is about operating a reliable platform.

Production concerns include:

- Availability
- Security
- Scalability
- Observability
- Backup
- Disaster recovery
- Cost control
- Upgrade safety
- Incident response
- Developer experience

2. High Availability

High availability means the system continues working even when some components fail.

In Kubernetes, HA applies to:

- Control plane
- Worker nodes
- Applications
- Networking
- Storage
- DNS
- Ingress
- Monitoring

3. Control Plane HA

A production control plane should be highly available.

Managed Kubernetes services like EKS, AKS, and GKE usually manage this for you.

Self-managed clusters must handle:

- Multiple API Server instances
- Multiple etcd members
- Load balancing to API Servers
- Secure certificates
- etcd backup
- Control plane upgrades

4. Worker Node HA

Applications should run across multiple nodes.

Bad:

All replicas on one node

Better:

Replicas spread across multiple nodes and zones

Use:

- Multiple worker nodes
- Multiple availability zones
- Pod anti-affinity
- Topology spread constraints
- PodDisruptionBudgets
- Readiness probes

5. Application HA

Production applications should have:

- Multiple replicas
- Readiness probes
- Liveness probes

- Rolling updates
- Resource requests
- Proper Service selectors
- Autoscaling
- PodDisruptionBudgets
- Graceful shutdown

Example:

```
replicas: 3
```

YAML

One replica is usually not enough for production web applications.

6. PodDisruptionBudget

A PodDisruptionBudget protects application availability during voluntary disruptions.

Example:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: api
```

YAML

This means at least 2 API Pods should remain available during voluntary disruption.

7. Graceful Shutdown

Applications should handle termination correctly.

Kubernetes sends SIGTERM before killing a container.

Good app behavior:

- Stop accepting new requests
- Finish in-flight requests
- Close database connections
- Flush logs
- Exit cleanly

Deployment setting:

```
terminationGracePeriodSeconds: 30
```

YAML

8. Production Deployment Safety

Use:

- Rolling updates
- Readiness probes
- Rollback strategy
- Versioned image tags
- GitOps
- Progressive delivery for risky apps
- Smoke tests
- Monitoring during rollout

Avoid:

```
latest image tag
manual kubectl edits
no readiness probe
no rollback plan
```

9. Cluster Upgrades

Kubernetes upgrades should be planned carefully.

Upgrade areas:

- Control plane
- Worker nodes
- CNI plugin
- CSI drivers
- CoreDNS
- kube-proxy
- Ingress controller
- Autoscaler/Karpenter
- Helm charts
- CRDs/operators

10. Upgrade Best Practices

Before upgrade:

- Read release notes
- Check deprecated APIs
- Backup etcd if self-managed
- Check add-on compatibility
- Test in non-production
- Confirm rollback plan
- Check monitoring/alerts

During upgrade:

- Upgrade control plane first
- Upgrade add-ons carefully
- Upgrade nodes gradually
- Respect PDBs
- Monitor workloads

After upgrade:

- Validate apps
- Check nodes
- Check system Pods
- Check logs/events
- Check autoscaling
- Check ingress/DNS

11. Deprecated API Checks

Kubernetes removes old APIs over time.

Before upgrading, check manifests for deprecated APIs.

Examples:

```
extensions/v1beta1 Ingress
apps/v1beta1 Deployment
policy/v1beta1 PodDisruptionBudget
```

Use current stable APIs.

Example current Ingress:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
```

12. Backup

Backups are required for disaster recovery.

Important backup targets:

- etcd
- Kubernetes manifests
- Helm values
- GitOps repositories
- Persistent volumes
- Databases
- Secrets/external secret stores
- Cluster configuration
- CRDs and custom resources

13. etcd Backup

For self-managed clusters, etcd backup is critical because etcd stores cluster state.

etcd stores:

- Deployments
- Services
- Secrets
- ConfigMaps
- RBAC
- CRDs
- Cluster state

Managed services usually manage control plane storage, but you still need application and persistent data backups.

14. Persistent Volume Backup

PV backup depends on storage backend.

Examples:

Storage	Backup Method
AWS EBS	EBS snapshots
AWS EFS	AWS Backup
NFS	Filesystem backup
Databases	Native DB backup
Ceph	Snapshot/replication

Do not assume PVC alone is a backup.

PVC is only a storage request.

15. Velero

Velero is a common Kubernetes backup and restore tool.

It can back up:

- Kubernetes resources
- Namespaces
- Persistent volumes, depending provider/plugin
- Cluster metadata

Use cases:

- Disaster recovery
- Namespace restore
- Migration
- Accidental deletion recovery

16. Disaster Recovery

Disaster recovery means restoring service after major failure.

Examples:

- Cluster deleted
- Region outage
- etcd corruption
- Storage failure
- Bad deployment
- Namespace accidentally deleted
- Ransomware/security incident

DR requires tested restore procedures.

IMPORTANT

Untested backups are not reliable backups.

17. RTO and RPO

Term	Meaning
RTO	Recovery Time Objective
RPO	Recovery Point Objective

RTO answers:

How quickly must we restore service?

RPO answers:

How much data loss is acceptable?

Example:

RT0: 1 hour
RPO: 15 minutes

18. Multi-Zone and Multi-Region

Production clusters commonly use multiple availability zones.

Multi-zone protects from node/AZ failure.

Multi-region protects from regional failure but is more complex.

Multi-region requires:

- Data replication
- DNS failover
- Traffic routing
- Backup strategy
- Secrets replication
- Observability
- Runbooks

19. Security Best Practices

Production Kubernetes security should include:

- RBAC least privilege
- Dedicated ServiceAccounts
- Pod Security Standards
- Non-root containers
- No privileged Pods unless required
- NetworkPolicies
- Image scanning
- Signed images where possible
- Secret management
- Encryption at rest
- Audit logs
- Regular patching
- Admission policies

20. Cost Optimization

Kubernetes can waste money if not managed.

Common waste sources:

- Over-requested CPU/memory
- Idle nodes
- Too many replicas
- Unused PVCs
- Oversized node types
- No autoscaling
- Expensive LoadBalancers
- Forgotten namespaces
- Debug workloads left running

21. Cost Optimization Practices

Use:

- Right-sized requests
- HPA
- Cluster Autoscaler or Karpenter
- Spot nodes for fault-tolerant workloads

- Namespace quotas
- Cost labels
- Scheduled scale-down for non-prod
- Cleanup policies
- PVC audits
- LoadBalancer audits
- Monitoring dashboards

22. Resource Right-Sizing

Bad:

```
requests:
  cpu: "4"
  memory: "16Gi"
```

YAML

for an app that usually uses:

200m CPU and 512Mi memory

This wastes cluster capacity.

Use real metrics to tune requests.

23. Production Namespace Standards

A production namespace should commonly have:

- ResourceQuota
- LimitRange
- NetworkPolicies
- RBAC bindings
- Pod Security labels
- Standard labels
- Monitoring rules
- Log collection
- Secret management
- Deployment ownership

24. GitOps

GitOps means Git is the source of truth for cluster desired state.

Tools:

- Argo CD
- Flux

Benefits:

- Audit trail
- Review process
- Rollback
- Drift detection
- Repeatability
- Environment consistency

PRODUCTION RULE

Avoid manual cluster changes that are not reflected in Git.

25. Production Readiness Checklist

```
[ ] Multiple replicas for critical apps
[ ] Readiness/liveness/startup probes
[ ] Resource requests and limits
[ ] PDBs for critical workloads
[ ] HPA where needed
[ ] Node autoscaling configured
[ ] Centralized logs
[ ] Metrics and dashboards
[ ] Alerts
[ ] NetworkPolicies
[ ] RBAC least privilege
[ ] Secrets managed securely
[ ] Backups configured
[ ] Restore tested
[ ] Upgrade process documented
[ ] Cost monitoring enabled
[ ] Runbooks available
```

26. Common Production Mistakes

Mistake	Impact
No resource requests	Bad scheduling
One replica	Downtime risk
No readiness probe	Bad traffic routing
No backups	No recovery
No NetworkPolicy	Lateral movement risk
Manual changes	Drift
No PDB	Maintenance outages
No log centralization	Hard debugging
Overprovisioning	High cost
No upgrade testing	Cluster outage risk

27. Interview Questions and Answers

Beginner

1. What makes Kubernetes production-ready?

HA, security, observability, backups, resource management, safe deployments, and operational runbooks.

2. What is PDB?

A PodDisruptionBudget protects minimum app availability during voluntary disruptions.

3. Why are backups important?

They allow recovery from data loss, cluster failure, or accidental deletion.

4. What is GitOps?

GitOps uses Git as the source of truth for Kubernetes desired state.

5. Why use multiple replicas?

To maintain availability if a Pod or node fails.

Intermediate

1. How do you safely upgrade a cluster?

Read release notes, test in non-prod, check deprecated APIs, back up, upgrade control plane, upgrade nodes gradually, and monitor.

2. How do you reduce Kubernetes cost?

Right-size resources, autoscale, use spot where safe, clean unused resources, audit PVCs and LoadBalancers, and use quotas.

3. What should be backed up?

etcd, manifests, Helm values, GitOps repos, PVs, databases, secrets, and CRDs.

4. Why is readiness important during deployment?

It prevents traffic from reaching Pods before they can serve requests.

5. Why avoid manual kubectl changes?

They create drift from Git and are hard to audit or reproduce.

Advanced

1. What is RTO and RPO?

RTO is how fast service must be restored. RPO is how much data loss is acceptable.

2. Why is multi-region Kubernetes complex?

Because it requires data replication, DNS failover, traffic routing, secrets, monitoring, and operational coordination.

3. How do PDBs affect node upgrades?

They may block voluntary Pod eviction if availability would drop below the defined budget.

4. How do you design DR for Kubernetes?

Back up cluster resources and data, replicate critical systems, define RTO/RPO, automate restore, and test recovery regularly.

5. Why is cost optimization tied to resource requests?

Requests determine scheduling and reserved capacity, so over-requesting wastes nodes and money.

28. Scenario-Based Interview Questions

Scenario 1: Node Upgrade Causes Downtime

EXPECTED ANSWER Check replica count, PDB, readiness probes, pod anti-affinity, and whether workloads were safely drained before node upgrade.

Scenario 2: Accidental Namespace Deletion

EXPECTED ANSWER Restore from GitOps manifests and backup tool such as Velero. Restore PV/application data if needed.

Scenario 3: Kubernetes Bill Increased

EXPECTED ANSWER Check over-requested resources, idle nodes, LoadBalancers, PVCs, autoscaling settings, non-prod workloads, and unused namespaces.

Scenario 4: Cluster Upgrade Failed

EXPECTED ANSWER Review upgrade logs/events, add-on compatibility, deprecated APIs, node status, system Pods, and rollback or fix based on failure scope.

29. Chapter Revision Notes

- Production Kubernetes requires HA, security, observability, backups, upgrades, and cost controls.
- Use multiple replicas and spread workloads.
- Use PDBs for critical apps.
- Use readiness probes for safe traffic.
- Back up both cluster state and application data.
- Test restore procedures.
- Use GitOps to prevent drift.
- Plan upgrades carefully.
- Watch deprecated APIs.

- Use autoscaling and right-sizing for cost optimization.
- Production readiness is an operational discipline, not only YAML.

30. Key Interview Statement

Memorize this:

KEY STATEMENT

Production Kubernetes requires more than running Pods. A reliable cluster needs high availability, safe rollouts, resource requests, autoscaling, observability, RBAC, NetworkPolicies, secure secrets, backups, tested disaster recovery, planned upgrades, GitOps, and continuous cost optimization.

End of Volume 02 — Chapter 16

Next chapter:

Chapter 17 — Kubernetes Final Interview Guide and Revision Cheatsheets

CLOUD NATIVE ENGINEERING MASTER SERIES · 2026
EDITION

| Volume 02 — Kubernetes Master Handbook

Final Interview Guide & Revision Cheatsheets

Chapter 17 — cheatsheets, 50 interview
questions, and architect-level scenarios.

👑 Chapter 17

Chapter Purpose

This chapter serves as the ultimate revision guide for Kubernetes.

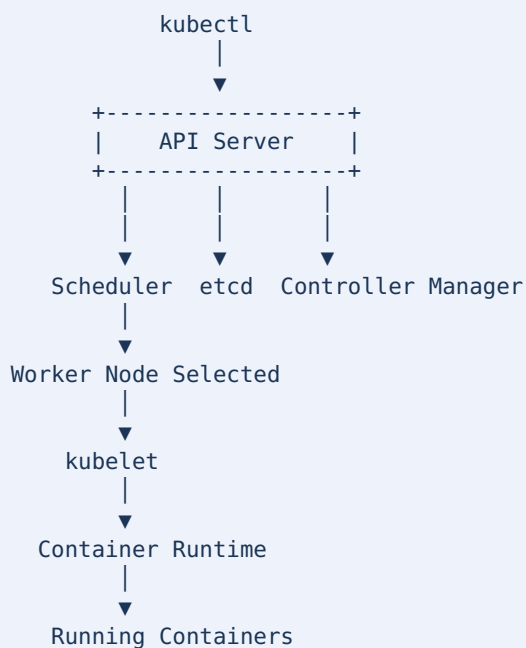
By the time you complete this chapter, you should be able to confidently answer:

- Beginner interview questions
- Mid-level DevOps interviews
- Senior Kubernetes interviews
- Platform Engineering interviews
- SRE interviews
- Cloud Native Architect interviews

This chapter is designed to be read multiple times before interviews.

Part 1 — Kubernetes Architecture Revision

Remember the complete architecture



Control Plane Components

Component	Responsibility
API Server	Front door of Kubernetes
etcd	Stores entire cluster state
Scheduler	Selects worker node
Controller Manager	Makes actual state match desired state
Cloud Controller	Talks to cloud provider

Worker Node Components

Component	Responsibility
kubelet	Runs Pods
kube-proxy	Network routing
Container Runtime	Runs containers
CNI	Networking
CSI	Storage

Part 2 — Kubernetes Objects Revision

Object	Purpose
Pod	Smallest deployable unit
ReplicaSet	Maintains replicas
Deployment	Stateless application
StatefulSet	Stateful application
DaemonSet	One Pod per node
Job	Run-to-completion task
CronJob	Scheduled Job
Service	Stable networking
Ingress	HTTP routing
Gateway API	Next-generation traffic routing
ConfigMap	Configuration
Secret	Sensitive configuration
PVC	Storage request
PV	Actual storage
StorageClass	Dynamic storage provisioning
Namespace	Logical isolation

Part 3 — Service Types Cheat Sheet

Service	Use Case
ClusterIP	Internal communication
NodePort	External via Node IP
LoadBalancer	Cloud Load Balancer
ExternalName	External DNS mapping
Headless	Stateful applications

Part 4 — Workload Selection Guide

Requirement	Use
Stateless application	Deployment
Database	StatefulSet
Logging Agent	DaemonSet
Daily Backup	CronJob
Data Migration	Job
One Pod per node	DaemonSet
Stable identity	StatefulSet
Multiple replicas	Deployment

Part 5 — Scheduling Revision

Pod Placement

Scheduler checks:

- Resources
- Labels
- Node Selector
- Affinity
- Anti-Affinity

- Taints
- Tolerations
- Storage
- Topology

NodeSelector

Simple label match.

```
SSD nodes
GPU nodes
Spot nodes
```

Node Affinity

Advanced scheduling.

- Required
- Preferred
- Multiple operators

Pod Anti-Affinity

Spread replicas.

Good example:

```
Replica 1 → Node A
Replica 2 → Node B
Replica 3 → Node C
```

Taints

Repel Pods.

```
Dedicated Database Node
```

Tolerations

Allow Pods onto tainted nodes.

Part 6 — Storage Revision

```
Application
↓
PVC
↓
PV
↓
StorageClass
↓
Cloud Storage
```

Examples

- AWS EBS
- AWS EFS
- NFS
- Ceph
- Local Storage

Part 7 — Networking Revision

Flow

```
Client
↓
Ingress
↓
Service
↓
Pod
↓
Container
```

Networking Components

- CNI
- CoreDNS
- Service
- Ingress
- Gateway API
- NetworkPolicy

Part 8 — Autoscaling Revision

Application Scaling

↓
HPA
↓
More Pods

Infrastructure Scaling

↓
Cluster Autoscaler
↓
More Nodes

Resource Optimization

↓
VPA
↓
Better Requests/Limits

Dynamic Node Provisioning

↓
Karpenter
↓
Best AWS Instance

Part 9 — Security Revision

Authentication
↓
Authorization
↓
Admission Controller
↓
RBAC
↓
Pod Security
↓
Runtime Security

Remember

Stage	Question
Authentication	Who are you?
Authorization	What can you do?
Admission	Should Kubernetes allow this request?

Part 10 — Resource Management

```
Requests
↓
Scheduler
↓
Node Selection
↓
Container Runs
↓
Limits Applied
```

Remember

Concept	Phase
Requests	Scheduling
Limits	Runtime

Part 11 — QoS Classes

QoS Class	Meaning
Guaranteed	Highest priority
Burstable	Normal applications
BestEffort	Lowest priority

BestEffort is evicted first.

Part 12 — Observability

Three Pillars

- Logs
- Metrics
- Traces

Additional

- Events
- Dashboards
- Alerts
- Probes

Golden Signals

- Latency
- Traffic
- Errors
- Saturation

Part 13 — Health Probes

Probe	Purpose
Startup	Initial startup
Readiness	Receive traffic
Liveness	Restart if unhealthy

Part 14 — Production Checklist

Every production application should have

- Multiple replicas
- Readiness Probe
- Liveness Probe
- Resource Requests
- Resource Limits
- HPA
- Monitoring
- Logging
- Alerts
- PDB
- NetworkPolicy
- RBAC
- GitOps

Part 15 — kubectl Cheat Sheet

Cluster

```
kubectl cluster-info
kubectl version
kubectl get nodes
```

SHELL

Pods

```
kubectl get pods
kubectl describe pod POD
kubectl logs POD
kubectl exec -it POD -- sh
kubectl delete pod POD
```

SHELL

Deployments

```
kubectl get deploy
kubectl rollout status deployment/app
kubectl rollout history deployment/app
kubectl rollout undo deployment/app
kubectl scale deployment app --replicas=5
```

SHELL

Services

```
kubectl get svc
kubectl describe svc SERVICE
```

SHELL

Ingress

```
kubectl get ingress
kubectl describe ingress
```

SHELL

Storage

```
kubectl get pvc
kubectl get pv
kubectl get storageclass
```

SHELL

Resources

```
kubectl top pods
kubectl top nodes
```

SHELL

Events

```
kubectl get events
kubectl get events \
--sort-by=.metadata.creationTimestamp
```

SHELL

Debug

```
kubectl describe pod POD
kubectl logs POD
kubectl logs POD --previous
```

SHELL

Part 16 — Complete Kubernetes Troubleshooting Flow

Whenever something fails

Always remember

```
Application
↓
Pod
↓
Container
↓
Logs
↓
Events
↓
Deployment
↓
ReplicaSet
↓
Service
↓
Endpoints
↓
Ingress
↓
DNS
↓
NetworkPolicy
↓
Storage
↓
Node
↓
Cluster
```

Never jump randomly.

Part 17 — 50 Most Asked Kubernetes Interview Questions

Beginner

1. What is Kubernetes?

A container orchestration platform that automates deployment, scaling, networking, and management of containerized applications.

2. What is a Pod?

The smallest deployable unit in Kubernetes.

3. Why can't containers be deployed directly?

Kubernetes manages Pods, not standalone containers, because Pods provide networking, storage, lifecycle management, and shared resources.

4. Difference between Pod and Container?

A Pod can contain one or more tightly coupled containers.

5. What is a Deployment?

A Deployment manages stateless applications and provides rolling updates and rollbacks.

6. What is ReplicaSet?

Maintains the desired number of Pod replicas.

7. What is StatefulSet?

Used for stateful applications requiring stable identity and storage.

8. What is DaemonSet?

Runs exactly one Pod on every eligible node.

9. What is Job?

Runs a task until completion.

10. What is CronJob?

Runs Jobs on a schedule.

11. What is Service?

Provides stable networking for Pods.

12. Difference between ClusterIP and NodePort?

ClusterIP is internal-only. NodePort exposes the Service via a node's IP and a static port.

13. What is Ingress?

Routes external HTTP/HTTPS traffic to Services based on rules.

14. What is ConfigMap?

Stores non-sensitive configuration.

15. What is Secret?

Stores sensitive configuration like passwords, tokens, and certificates.

16. What is PVC?

A request for persistent storage.

17. What is StorageClass?

Defines how storage is dynamically provisioned.

18. What is HPA?

Scales Pod replicas automatically based on metrics.

19. What is RBAC?

Role-Based Access Control for Kubernetes resources.

20. What is NetworkPolicy?

Controls allowed network communication between Pods.

Part 18 — Senior-Level Interview Questions

Explain Kubernetes scheduling from Pod creation until the container starts.

Expected answer:

```
Pod created
↓
API Server
↓
etcd
↓
Scheduler selects node
↓
Binding
↓
kubelet receives assignment
↓
Container runtime pulls image
↓
Container starts
↓
Readiness
↓
Service endpoints updated
```

Explain Deployment rollout.

Expected answer:

Deployment creates a new ReplicaSet, gradually increases new Pods while decreasing old Pods according to the rolling update strategy, waits for readiness, and completes when the desired state is achieved. If a problem occurs, it can be rolled back.

Explain why Pods become Pending.

Expected answer:

Pending Pods usually indicate scheduling issues such as insufficient CPU or memory, taints, affinity mismatches, unbound PVCs, quota limits, or unavailable nodes.

Explain Service discovery.

Expected answer:

CoreDNS creates DNS records for Services. Pods query DNS, receive the Service ClusterIP, and kube-proxy forwards traffic to healthy Pod endpoints.

Explain CrashLoopBackOff.

Expected answer:

The container starts successfully but exits repeatedly. Kubernetes restarts it with increasing backoff. Common causes include application bugs, bad configuration, failed dependencies, or aggressive liveness probes.

Explain Kubernetes desired state.

Expected answer:

Users declare the desired configuration in manifests. Controllers continuously compare the desired state with the actual cluster state and reconcile differences until they match.

Part 19 — Architect-Level Design Questions

Design Kubernetes for 50 million users.

Things to discuss:

- Multi-AZ cluster
- HPA
- Cluster Autoscaler/Karpenter
- Ingress Controller
- CDN
- Global Load Balancer
- Database replication
- Redis cache
- Message queues
- Observability
- GitOps
- Disaster recovery
- Security

- Cost optimization

Design a Highly Available Kubernetes Platform

Include:

- Three Availability Zones
- Multiple worker node groups
- Dedicated system node pool
- Spot node pool
- On-demand node pool
- Ingress Controller replicas
- CoreDNS replicas
- HPA
- PDB
- Monitoring stack
- Logging stack
- Velero backups
- GitOps deployment
- RBAC
- NetworkPolicies
- Admission policies

Design Secure Kubernetes Platform

Discuss:

- OIDC authentication
- RBAC
- Pod Security Standards
- NetworkPolicies
- Secrets Manager
- Image scanning
- Signed images
- Runtime security
- Audit logging
- Encryption
- GitOps

Part 20 — Final 5-Minute Revision Before Interview

Remember these keywords:


```
API Server
etcd
Scheduler
Controller Manager
kubelet
Container Runtime
CNI
CSI
CRI
Pod
Deployment
ReplicaSet
Service
Ingress
ConfigMap
Secret
PVC
StorageClass
HPA
VPA
Cluster Autoscaler
Karpenter
RBAC
ServiceAccount
NetworkPolicy
Readiness
Liveness
Startup Probe
GitOps
Rolling Update
Rollback
PDB
CoreDNS
Observability
Logs
Metrics
Traces
```

Kubernetes Interview Success Formula

For almost every interview answer, structure your response like this:

1. Definition – What it is.
2. Purpose – Why it exists.
3. Internal Working – How it works step by step.
4. Production Example – Where it is used in real systems.
5. Best Practices – Industry recommendations.
6. Common Mistakes – Pitfalls to avoid.
7. Troubleshooting – How you would debug issues.
8. Related Concepts – Mention connected Kubernetes features.

Using this structure helps you give complete, professional answers instead of short definitions.

End of Volume 02 — Kubernetes Master Handbook

CONGRATULATIONS!

You have completed Volume 02 – Kubernetes Master Handbook.

Next Volume in the Cloud Native Engineering Master Series:

Volume 03 — Helm Master Handbook (Beginner to Expert)

This volume will cover:

- Helm Architecture
- Helm Internals
- Helm Charts
- Templating Engine
- Chart Development
- OCI Helm Registries
- Helm + GitOps
- Helm + CI/CD
- Production Helm Patterns
- Enterprise Helm Best Practices
- 250+ Interview Questions
- Production Labs
- Real-world Enterprise Examples
- Complete Troubleshooting Guide

Estimated size: ~600–800 pages in full handbook format.